

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Criptografía basada en retículos

Autor
Denis Valentin Stoyanov D'Antonio

Directores
Jesús García Miranda



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, septiembre de 2024



Criptografía basada en retículos

Autor
Denis Valentin Stoyanov D'Antonio

Directores
Jesús García Miranda

Criptografía basada en retículos: El cifrado del futuro

Denis Valentin Stoyanov D'Antonio

Palabras clave: Cifrado simétrico, Cifrado asimétrico, Firma digital, Computación cuántica, Algoritmos cuánticos, Algoritmos post-cuánticos, Retículos, Mecanismo de Encapsulamiento de Claves, Python.

Resumen

La computación cuántica está cada vez más cerca de convertirse en una realidad de nuestro día a día y, aunque esto traerá muchas ventajas, también puede acarrear muchos problemas si se utiliza de forma indebida.

Ya existen algoritmos cuánticos capaces de romper cifrados que actualmente consideramos completamente seguros, tanto los simétricos como los asimétricos, además de las firmas digitales. Por suerte para nosotros, aunque la computación cuántica tiene la capacidad teórica para ello, todavía no ha conseguido llevarlo a la práctica.

Es aquí donde entra el National Institute of Standars and Technology (NIST), una institución encargada de la innovación y el avance tecnológico, la cual inició en 2017 un concurso para recabar algoritmos denominados "post-cuánticos", es decir, capaces de cifrar los datos de manera segura incluso ante los algoritmos cuánticos.

Siete años después, nos encontramos en la cuarta ronda de este concurso, habiendo avanzado mucho en algunos algoritmos muy prometedores y descartado otros muchos. Así mismo, en 2022, se estandarizaron algunos algoritmos que ya se consideran completamente seguros. Sin embargo, en pos de no dejar de avanzar en este ámbito, así como diversificar los fundamentos matemáticos de los algoritmos, el concurso sigue activo a día de hoy.

En este *Trabajo de Fin de Grado (TFG)*, vamos a realizar un ligero repaso general a cómo funciona la criptografía hoy en día, mencionando algunos de los algoritmos de cifrado más importantes. Tras esto, explicaremos en qué consiste la computación cuántica y porqué es tan peligrosa para la criptografía actual. Finalmente, hablaremos sobre el concurso del NIST, los algoritmos que se encuentran en su ronda más avanzada y aquellos que se estandarizaron en 2022.

Con todo este contexto realizado, escogeremos un algoritmo del concurso del NIST destinado al encapsulamiento de claves, denominado *Algoritmo ML-KEM*, y llevaremos a cabo su análisis e implementación. Como este algoritmo se fundamenta en los problemas basados en retículos, previamente a desarrollarlo, haremos una introducción a este campo.

Lattice-based Cryptography: The encryption of the future

Denis Valentin Stoyanov D'Antonio

Keywords: Symmetric encryption, Asymmetric encryption, Digital signature, Quantum computation, Quantum algorithms, Post-Quantum Algorithms, Reticules, Key Encapsulation Mechanism, Python.

Abstract

Quantum computing is getting closer and closer to become a reality in our everyday lives, and while this will bring many advantages, it can also bring many problems if it is misused.

There are already quantum algorithms capable of breaking encryption that we currently consider to be completely secure, both symmetric and asymmetric encryption, as well as digital signatures. Fortunately for us, although quantum computing has the theoretical capability to do this, it has not yet managed to put it into practice.

This is where the National Institute of Standards and Technology (NIST), an institution in charge of innovation and technological advancement, comes in. In 2017, it launched a competition for the called 'post-quantum' algorithms, these are algorithms capable of encrypting data securely even against quantum algorithms.

Seven years later, we are in the fourth round of this competition, having made good progress on some very promising algorithms and discarded many others. Also, in 2022, some algorithms that were considered completly safe were standardised. However, in order to continue to make progress in this area, as well as to diversify the mathematical foundations of the algorithms, the competition is still active today.

In this *Final Degree Project (FDP)*, we are going to give a brief overview of how cryptography works nowadays, mentioning some of the most important encryption algorithms. After this, we will explain what quantum computing is and why it is so dangerous for the current cryptography. Finally, we will talk about the NIST competition, the algorithms that are in the most advanced round and those that were standardised in 2022.

With all this context established, we will select an algorithm from the NIST key encapsulation competition, called the *ML-KEM Algorithm*, and proceed with its analysis and implementation. Since this algorithm is based on lattice-based problems, before developing it, we will provide an introduction to this field.

Yo, **Denis Valentin Stoyanov D'Antonio**, alumno de la titulación INGENIERÍA INFORMÁTICA de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Denis Valentin Stoyanov D'Antonio

Granada a 06 de septiembre de 2024

D. **Jesús García Miranda**, Profesor del Área de Álgebra del Departamento Álgebra de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Criptografía basada en retículos, El cifrado del futuro***, ha sido realizado bajo su supervisión por **Denis Valentin Stoyanov D'Antonio**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 06 de septiembre de 2024.

Los directores:

Jesús García Miranda

Agradecimientos

Quiero agradecerle a mi familia todo el apoyo y recursos que me han dado a lo largo de estos cuatro años. Gracias porque, aunque no lleguen a comprender del todo de qué trata este grado universitario, responder "bien" a la pregunta "¿qué tal la carrera?", les daba la seguridad necesaria de que, efectivamente, todo iba bien.

Así mismo, agradecerle a mi grupo de amigos de Águilas "La7RM(ob)" por haber estado ahí en todo momento. Aunque durante el curso apenas nos veamos, siempre los he sentido a mi lado; me han hecho creer en mí mismo cuando ni yo lo hacía, y han sabido ensalzar mis logros más de lo que yo podía ver.

Las personas que sí me han acompañado presencialmente a lo largo de estos cuatro cursos han sido las de mi grupo de amigos de Granada, "Partidazos". He tenido la suerte de que todos los integrantes estamos matriculados en esta titulación, por lo que hemos podido avanzar juntos en todo este trayecto; así como desconectar juntos de la universidad cuando había tiempo para ello.

En especial, quiero agradecerle a mi mejor amigo José Dimas todo lo que ha hecho por mí estos cuatro años. Fue la primera persona que conocí en Granada y la que me ha acompañado cada día de este proceso. Ha sabido calmarme en los momentos más tensos, animarme en los más tristes y celebrar mis triunfos como si también fuesen suyos. Te has convertido en un hermano para mí.

Por último pero no menos importante, quiero agradecerle a mi tutor, Jesús García Miranda, todo el esfuerzo que ha realizado. Le conocí en mi primer año de carrera, y desde entonces me ha acompañado hasta literalmente el último momento; orientándome cuando tenía algún problema relacionado con la universidad, y, en este caso, brindándome los recursos necesarios para la correcta realización de este trabajo.

Índice general

1. Introducción **25**

1.1. Motivación 25

1.2. Objetivos del trabajo 26

1.3. Estructura del documento 26

2. La criptografía actualmente **29**

2.1. Introducción a la criptografía actual 29

2.2. El cifrado simétrico 30

2.2.1. Introducción 30

2.2.2. Tipos de cifrado simétrico 31

2.2.3. Algunos algoritmos utilizados para el cifrado simétrico 32

2.3. El cifrado asimétrico 34

2.3.1. Introducción 34

2.3.2. Algunos algoritmos utilizados para el cifrado asimétrico 34

2.4. El cifrado híbrido 36

2.5. La firma digital 36

2.5.1. Algoritmos utilizados para la realización de firmas digitales 37

3. La computación cuántica **39**

3.1. Qué es la computación cuántica 39

3.2. Programación de los ordenadores cuánticos 40

3.3. El peligro de la llegada de los ordenadores cuánticos 41

3.3.1. Algoritmo de Shor 41

3.3.2. Algoritmo de Grover 42

4. Criptografía Post-Cuántica **43**

4.1. Concurso NIST 43

4.1.1. Algoritmos estandarizados en 2022 44

4.1.2. Algoritmos que se encuentran en la ronda 4 44

4.2. Nuevo concurso de criptografía post-cuántica para firmas digitales 45

5. Criptografía basada en retículos **47**

5.1. Qué es un retículo 47

5.1.1. Retículos q-áricos 48

5.1.2. Retículos ideales 49

5.1.3. Función Gaussiana Discreta 49

5.2. Principales problemas con retículos 50

5.2.1. Problema del vector más corto (SVP) 50

5.2.2. Problema del vector más corto aproximado (ASVP) . 51

5.2.3. GapSVP 51

5.2.4. Problema del vector más cercano (CVP) 51

5.2.5. Decodificación a Distancia Acotada (BDD) 51

5.2.6. Solución de enteros cortos (SIS) 51

5.2.7. Aprendiendo con errores (LWE) 52

5.2.8. Aprendizaje en anillo con errores (Ring-LWE) 53

5.2.9. Aprendizaje modular con errores (Module-LWE) . . . 53

6. Module-Lattice-Based Key-Encapsulation Mechanism Standar (FIPS 203) **55**

6.1. Descripción del estándar FIPS 203 55

6.1.1. Método de encapsulación de claves (KEM) 56

6.1.2. Módulo de aprendizaje con errores (Module-LWE) como base para el KEM 56

6.2. La construcción del ML-KEM 57

6.3. Características del algoritmo ML-KEM 57

6.3.1. Dominio NTT 57

6.3.2. Multiplicación en el dominio NTT 59

6.3.3. Propiedad de exactitud de un KEM 60

6.3.4. Probabilidad de fallo de desencapsulamiento 60
6.4. Los tres conjuntos de parámetros aceptados 60
6.4.1. Elegir el conjunto de parámetros apropiado 62
6.5. Diferencia de ML-KEM con CRYSTALS-Kyber 62
6.6. Diferencia entre el borrador inicial de ML-KEM y su versión final 63

7. Implementación del algoritmo ML-KEM 65
7.1. Lenguaje utilizado 65
7.2. Librerías auxiliares utilizadas 66
7.2.1. NumPy 66
7.2.2. Hashlib 67
7.3. Clase ML-KEM 67
7.3.1. Constantes globales 67
7.3.2. Variables globales 69
7.3.3. Métodos privados 71
7.3.4. Métodos públicos 95
7.4. Traza de ejecución usual 98
7.4.1. Paso 1: Generación de claves (ML-KEM KeyGen) . . 98
7.4.2. Paso 2: Generación interna de las claves (ML-KEM KeyGen Internal) 99
7.4.3. Paso 3: Encapsulación (ML-KEM Encaps) 99
7.4.4. Paso 4: Encapsulación interna (ML-KEM Encaps Internal) 99
7.4.5. Paso 5: Cifrado K-PKE (K-PKE Encrypt) 100
7.4.6. Paso 6: Desencapsulación (ML-KEM Decaps) 100
7.4.7. Paso 7: Desencapsulación interna (ML-KEM Decaps Internal) 100
7.4.8. Paso 8: Descifrado K-PKE (K-PKE Decrypt) 101
7.4.9. Paso 9: Comparación de las claves K y K' 101
7.5. Prueba de rendimiento para las distintas configuraciones de parámetros 101
7.6. Problemas encontrados durante la creación del algoritmo . . . 103

8. Interfaz del algoritmo ML-KEM **105**

8.1. Objetivos de la interfaz 105

8.2. Cómo está desarrollada la interfaz 105

8.2.1. Acción 1: Modificar el conjunto de parámetros a utilizar 107

8.2.2. Acción 2: Generar el conjunto de claves aleatoriamente 109

8.2.3. Acción 3: Asignar la clave encaps_k introducida como argumento 110

8.2.4. Acción 4: Asignar la clave decaps_k introducida como argumento 111

8.2.5. Acción 5: Realizar el proceso de encapsulamiento de un mensaje 111

8.2.6. Acción 6: Realizar el proceso de desencapsulamiento de un mensaje cifrado 112

8.2.7. Acción 7: Comparar claves secretas compartidas . . . 113

8.2.8. Acción 0: Cerrar la interfaz 114

8.3. Ejemplo de interfaz con claves aleatorias 114

8.4. Ejemplo de interfaz con claves por parámetro 117

8.5. Ejemplo de la interfaz con K, dk y c por parámetro 120

8.6. Ejemplo de la interfaz con K y K' por parámetro 122

8.7. Ejemplos de usos incorrectos de la interfaz 122

9. Conclusiones **125**

9.1. Cumplimiento de los objetivos iniciales 125

9.1.1. Analizar el estado actual de la criptografía, incluyendo los algoritmos de cifrado simétrico, cifrado asimétrico y firma digital más utilizados 125

9.1.2. Explicar los principios fundamentales de la computación cuántica y cómo esta tecnología representa una amenaza para los sistemas criptográficos actuales . . . 125

9.1.3. Investigar el concurso para la estandarización de algoritmos post-cuánticos llevado a cabo por el NIST, incluyendo una revisión de los algoritmos más prometedores 126

9.1.4. Introducir y explicar los fundamentos de los problemas basados en retículos, que son la base matemática de muchos algoritmos post-cuánticos 126

9.1.5. Seleccionar y desarrollar un algoritmo post-cuántico basado en retículos destacado del concurso del NIST, enfocado en el encapsulamiento de claves 126
9.1.6. Implementar el algoritmo seleccionado 127
9.1.7. Implementar una interfaz para el algoritmo desarrollado 127
9.1.8. Comprobar el correcto funcionamiento del algoritmo . 127
9.1.9. Sacar conclusiones de todo lo desarrollado y aprendido 127
9.2. Problemas u obstáculos encontrados durante la realización del proyecto 128
9.3. Conclusiones y aprendizaje obtenidos a raíz de este proyecto . 129

Bibliografía **131**

A. Enlace al código disponible en un repositorio de GitHub **137**

Índice de figuras

2.1. Representación de criptografía simétrica[17] 30
2.2. Representación de criptografía asimétrica[17] 34
2.3. Representación de firma digital 37

3.1. Idea de un qubit[13] 39

5.1. Ejemplo de base de un retículo[6] 48
5.2. Amarillo: CVP (inalcanzable), Morado: BDD (alcanzable)[6] . 52

6.1. Funcionamiento del mecanismo KEM[25] 56
6.2. Probabilidad de fallo en el desencapsulamiento para ML-KEM[25] 60
6.3. Conjuntos de parámetros posibles para ML-KEM[25] 61
6.4. Longitud (en bytes) de las claves y el texto cifrado en ML-KEM para cada conjunto de parámetros 61

7.1. Import de librerías: numpy y hashlib 66
7.2. Constantes globales de la clase ML-KEM 68
7.3. _ArrayBitRev y _ArrayBitRev_Multiply 68
7.4. Forma utilizada para calcular _ArrayBitRev 69
7.5. Forma utilizada para calcular _ArrayBitRev_Multiply 69
7.6. Variables establecidas por defecto para ML-KEM-768 70
7.7. Variables globales: encaps_k y decaps_k 70
7.8. Método privado: _PRF(eta, s, b) 72
7.9. Método privado: _H(s) 72
7.10. Método privado: _J(s) 73
7.11. Método privado: _G(c) 73

7.12. Método privado: _BitRev(i) 74
7.13. Método privado: _BitsToBytes(b) 74
7.14. Método privado: _BytesToBits(B) 75
7.15. Método privado: _Compress(x, d) 75
7.16. Método privado: _Decompress(x, d) 76
7.17. Método privado: _ByteEncode(F, d) 77
7.18. Método privado: _ByteDecode(B, d) 77
7.19. Método privado: _SampleNTT(B) 79
7.20. Método privado: _SamplePolyCBD(B, eta) 80
7.21. Método privado: _NTT(f) 81
7.22. Método privado: _InverseNTT(F) 81
7.23. Método privado: _MultiplyNTTs(t_hat, g_hat) 82
7.24. Método privado: _BaseCaseMultiply(a0, a1, b0, b1, gamma) 83
7.25. Método privado: _K_PKE_KeyGen(d) 84
7.26. Método privado: _K_PKE_Encrypt(ekPKE, m, r) - *Parte 1* 86
7.27. Método privado: _K_PKE_Encrypt(ekPKE, m, r) - *Parte 2* 87
7.28. Método privado: _K_PKE_Decrypt(dkPKE, c) 88
7.29. Método privado: _ML_KEM_KeyGenInternal(d, z) 89
7.30. Método privado: _ML_KEM_EncapsInternal(encaps_k, m) 90
7.31. Método privado: _ML_KEM_DecapsInternal(decaps_k, c) . 91
7.32. Método privado: _comprobarEk(encaps_k) 92
7.33. Método privado: _comprobarDk(decaps_k) 92
7.34. Método privado: _verificarC(c) 93
7.35. Método privado: _extablecerEk(encaps_k) 94
7.36. Método privado: _extablecerDk(decaps_k) 94
7.37. Método privado: _compararClavesSecretas(k1, k2) 94
7.38. Método público: extablecer_parametros(x) 95
7.39. Método público: _ML_KEM_KeyGen() 96
7.40. Método público: _ML_KEM_Encaps(encaps_k, m) 96
7.41. Método público: _ML_KEM_Decaps(decaps_k, m) 97
7.42. Método público: ML_KEM_Interfaz(encaps_k=None, decaps_k=None, K=None, K_prima=None, c=None) 97

7.43. Código para medir el rendimiento de *ML-KEM-KeyGen()* para cada una de las tres posibles configuraciones 102

7.44. Tiempo promedio en segundos que tarda en ejecutarse *ML-KEM-KeyGen()* para cada una de las tres posibles configuraciones 102

8.1. Cómo utilizar el método ML_KEM_Interfaz desde un archivo externo 105

8.2. Mensaje de bienvenida al iniciar la interfaz de ML-KEM . . . 106

8.3. Código del mensaje de bienvenida al iniciar la interfaz de ML-KEM 106

8.4. Menú de la interfaz de ML-KEM 106

8.5. Código del menú de la interfaz de ML-KEM 107

8.6. Número inválido introducido en la interfaz de ML-KEM . . . 107

8.7. Acción 1 de la interfaz: Elegir conjunto de parámetros al que cambiar partir de la interfaz 107

8.8. Conjunto de parámetros cambiado a partir de la interfaz . . . 108

8.9. Código de la acción 1 de la interfaz: Elegir conjunto de parámetros al que cambiar partir de la interfaz 109

8.10. Acción 2 de la interfaz: Generar el conjunto de claves aleatoriamente 110

8.11. Código de la acción 2 de la interfaz: Generar el conjunto de claves aleatoriamente 110

8.12. Código de la acción 3 de la interfaz: Asignar una clave de encapsulamiento pasada como argumento 111

8.13. Código de la acción 4 de la interfaz: Asignar una clave de desencapsulamiento pasada como argumento 111

8.14. Acción 5 de la interfaz: Encapsular un mensaje generado aleatoriamente 112

8.15. Código de la acción 5 de la interfaz: Encapsular un mensaje generado aleatoriamente 112

8.16. Acción 6 de la interfaz: Desencapsular un texto cifrado 113

8.17. Código de la acción 6 de la interfaz: Desencapsular un texto cifrado 113

8.18. Acción 7 de la interfaz: Comprarar dos claves secretas K y K' 113

8.19. Código de la acción 7 de la interfaz: Comprarar dos claves secretas K y K' 114
8.20. Acción 0 de la interfaz: Cerrar la interfaz 114
8.21. Llamada a la interfaz para generar claves aleatorias 114
8.22. Acción 1 de la interfaz: Elegir conjunto de parámetros al que cambiar partir de la interfaz 115
8.23. Conjunto de parámetros cambiado a partir de la interfaz . . . 115
8.24. Acción 2 de la interfaz: Generar el conjunto de claves aleatoriamente 115
8.25. Acción 5 de la interfaz: Encapsular un mensaje generado aleatoriamente 116
8.26. Acción 6 de la interfaz: Desencapsular un texto cifrado 116
8.27. Acción 7 de la interfaz: Comprarar dos claves secretas K y K' 116
8.28. Claves creadas y pasadas como argumentos a la interfaz . . . 117
8.29. La interfaz detecta las claves creadas y pasadas como argumentos a la interfaz 117
8.30. Ejemplo intentar asignar la clave de encapsulamiento para una interfaz que está configurada con otro conjunto de parámetros 117
8.31. Pasar al conjunto de parámetros correcto para la clave de encapsulamiento introducida por parámetro (ML-KEM-512 en este caso) 118
8.32. Asignar el valor de la clave de encapsulamiento introducido por parámetro como clave de encapsulamiento de la clase . . 118
8.33. Asignar el valor de la clave de desencapsulamiento introducido por parámetro como clave de desencapsulamiento de la clase . 118
8.34. Realizar el proceso de encapsulamiento con la clave encaps_k introducida por parámetro 119
8.35. Realizar el proceso de desencapsulamiento con la clave decaps_k introducida por parámetro al texto cifrado calculado en el paso anterior 119
8.36. Comparación de las claves secretas tras el proceso de encapsulamiento y desencapsulamiento a partir de las claves introducidas como parámetro 120
8.37. Claves de encapsulamiento, clave secreta y texto cifrado creados previamente y pasados como argumento 120

8.38. Pasar al conjunto de parámetros correcto para la clave de desencapsulamiento introducida por parámetro (en este caso ya nos encontramos en el conjunto correcto) 120

8.39. Asignar el valor de la clave de desencapsulamiento introducido por parámetro como clave de desencapsulamiento de la clase . 121

8.40. Realizar el proceso de desencapsulamiento con la clave decaps_k introducida por parámetro al texto cifrado introducido también como parámetro 121

8.41. Comparación de las claves secretas tras el proceso de desencapsulamiento y la la introducida inicialmente por parámetro 121

8.42. Claves K y K' precreadas y pasadas como argumento 122

8.43. Comparar las claves K y K' desde el primer momento que se accede a la interfaz 122

Capítulo 1

Introducción

1.1. Motivación

La computación cuántica representa una amenaza casi inminente para la criptografía moderna, capaz de comprometer sistemas considerados seguros. Es por esto que se desató mi curiosidad por cómo se podía contrarrestar este suceso. Investigué y di con el concurso del NIST, el cuál abordaba exactamente la duda que yo me había planteado.

Analizando el concurso del NIST, vi que ya se habían estandarizado algunos algoritmos post-cuánticos, y que con ellos, estaban apareciendo mejoras constantemente.

El objetivo de este trabajo es desarrollar uno de esos algoritmos que son mejoras a los algoritmos ya estandarizados considerados post-cuánticos, es decir, resistentes a los algoritmos cuánticos.

Antes de llegar a dicho desarrollo, habrá que pasar por un contexto previo, como de qué forma está implementada la criptografía hoy en día, qué es la computación cuántica y por qué es una amenaza, qué es la criptografía post-cuántica y, especialmente, qué son los retículos; ya que nuestro algoritmo implementado se basa en ellos.

Nuestro objetivo es contribuir al conocimiento y preparación ante los desafíos que la computación cuántica plantea. Del mismo modo, una vez llegado el día en el que todos los sistemas deban migrar a los nuevos algoritmos, ya tendremos conocimiento sobre ellos, lo que nos brindará una ventaja en cuanto a tiempo y esfuerzo.

1.2. Objetivos del trabajo

Los objetivos de este trabajo son los siguientes:

1. Analizar el estado actual de la criptografía, incluyendo los algoritmos de cifrado simétrico, cifrado asimétrico y firma digital más utilizados.
2. Explicar los principios fundamentales de la computación cuántica y cómo esta tecnología representa una amenaza para los sistemas criptográficos actuales.
3. Investigar el concurso para la estandarización de algoritmos post-cuánticos llevado a cabo por el NIST, incluyendo una revisión de los algoritmos más prometedores.
4. Introducir y explicar los fundamentos de los problemas basados en retículos, que son la base matemática de muchos algoritmos post-cuánticos.
5. Seleccionar y desarrollar un algoritmo post-cuántico basado en retículos destacado del concurso del NIST, enfocado en el encapsulamiento de claves (KEM[1]).
6. Implementar el algoritmo seleccionado.
7. Implementar una interfaz para el algoritmo desarrollado.
8. Comprobar el correcto funcionamiento del algoritmo.
9. Sacar conclusiones de todo lo desarrollado y aprendido.

1.3. Estructura del documento

Este trabajo se divide en nueve capítulos, los cuales son:

- **Capítulo 1: Introducción.** En este capítulo se explica de una manera resumida y general de qué trata el trabajo y cuales han sido las motivaciones para llevarlo a cabo, así como los objetivos y estructura que seguirá el mismo.
- **Capítulo 2: La criptografía actualmente.** En este capítulo se abordan las principales técnicas de criptografía actuales, incluyendo sus algoritmos y sus distintos usos.

[1]KEM: Key Encapsulation Mechanism.

- **Capítulo 3: La computación cuántica.** En este capítulo se introduce el concepto de computación cuántica, por qué elementos está formada, cómo se trabaja con ella y qué peligros conlleva a la criptografía actual.
- **Capítulo 4: Criptografía Post-Cuántica.** En este capítulo se habla del concurso presentado por el NIST para la estandarización de algoritmos post-cuánticos, es decir, algoritmos criptográficos resistentes a los algoritmos cuánticos.
- **Capítulo 5. Criptografía basada en retículos.** En este capítulo, con el objetivo principal del proyecto de desarrollar un algoritmo post-cuántico basado en retículos, se busca explicar de forma clara qué son los retículos y las distintas formas de trabajar con ellos.
- **Capítulo 6. Algoritmo ML-KEM.** En este capítulo se explica detalladamente y de forma teórica, de qué trata el algoritmo que vamos a desarrollar denominado ML-KEM; así como todos los aspectos a tener en cuenta relacionados con el mismo.
- **Capítulo 7. Implementación del algoritmo ML-KEM.** En este capítulo se lleva a cabo la implementación del algoritmo ML-KEM, en una clase denominada ML-KEM. Para ello, se explican cada uno de los métodos que componen dicha clase, así como sus constantes y variables globales. Además, se explica el recorrido de un uso común de dicha clase y los problemas enfrentados a la hora de implementarla.
- **Capítulo 8. Implementación de una interfaz para el algoritmo ML-KEM.** En este capítulo se explica cómo utilizar y cómo se ha desarrollado una interfaz para el uso práctico de la clase ML-KEM.
- **Capítulo 9. Conclusiones.** En este capítulo se hace una breve recapitulación de todo lo explicado, aprendido y conseguido en este trabajo.

Capítulo 2

La criptografía actualmente

2.1. Introducción a la criptografía actual

La criptografía es la práctica y el estudio que se encarga proteger información mediante el uso de algoritmos codificados, hashes y firmas. La información puede estar en reposo (como un archivo ubicado en un disco duro), en tránsito (una comunicación electrónica intercambiada entre dos o más partes) o en uso (mientras se ejecutan operaciones de computación con los datos)[2]. La criptografía tiene cuatro objetivos principales, los cuales en algunos casos se buscarán de forma independiente y, en otros casos, se buscará combinar algunos de ellos:

- **Confidencialidad.** Poner la información únicamente a disposición de usuarios autorizados, conseguido mediante el cifrado.
- **Integridad.** Asegurar que la información no se ha manipulado, mediante técnicas hash.
- **No repudio.** Evitar que un usuario deniegue compromisos o acciones previas, con el uso de la firma digital.
- **Autenticación.** Confirmar la autenticidad de la información o de la identidad de un usuario, gracias a los certificados digitales.

Para conseguir esto, la criptografía emplea principalmente el cifrado como proceso de codificación de la información. La representación oficial de la información, conocida como texto plano, se convierte en lo que se denomina texto cifrado.

Así, mientras la información viaja a través de un determinado canal, como por ejemplo internet, lo hace de forma cifrada. De esta manera, el

texto no es legible por sí mismo, sino que necesita ser descifrado, en principio por el destinatario. Decimos en principio porque hoy en día hay muchas entidades ajenas que lo que buscan es interceptar el mensaje y comprometer la seguridad del mismo. Ya sea porque saben la clave de descifrado o porque tratan de encontrarla a la fuerza con algoritmos destinados a ello.

Los dos cifrados más utilizados actualmente son: el cifrado simétrico o de clave privada; y el cifrado asimétrico o de clave pública. En algunas ocasiones, estos dos tipos de cifrado se combinan dando lugar al denominado cifrado híbrido, el cual aprovecha algunas de las ventajas de cada uno de ellos.

Que los datos estén cifrados no es mera cuestión de precaución, sino que el cifrado de datos es exigido por la legislación de la Unión Europea (UE), concretamente en el Reglamento General de Protección de Datos (RGPD), *Reglamento (UE) 2016/679*, vigente desde abril 2016.[36]

2.2. El cifrado simétrico

2.2.1. Introducción

El cifrado simétrico o de clave privada, es un método de cifrado que utiliza una sola clave o código para cifrar y descifrar mensajes entre el emisor y el receptor. Para esto, los dos comunicantes deben de establecer una clave común que utilizarán siempre que vayan a enviarse un mensaje.[35]

El problema principal en este tipo de cifrado es la tarea de establecer la clave de tal forma que solo sea conocida por los comunicantes. Esto se debe a que lo normal es que dicha clave se establezca también a través de internet, o de algún otro canal no totalmente seguro, por lo que un ciberdelincuente podría realizarnos algún tipo de ataque, como por ejemplo fishing[1], para intentar obtener dicha clave; teniendo así acceso al contenido de los mensajes que se envíen, ya que él también podría descifrarlos.

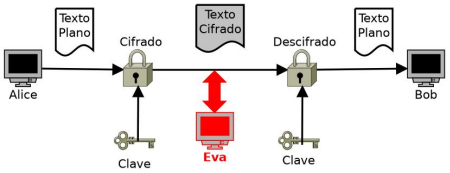


Figura 2.1: Representación de criptografía simétrica[17]

[1]Fishing: Fraude en línea para obtener información confidencial mediante engaño.

Además, hoy en día los ordenadores descifran claves a una velocidad muy alta, por lo que, para contrarrestar esto, el tamaño de la clave en el cifrado simétrico suele ser muy grande. Por suerte, generar dicha clave es un proceso rápido y sencillo, pero descifrarla a la fuerza es prácticamente imposible hoy en día, por lo que es un método muy seguro (teniendo siempre en cuenta que no nos intercepten la clave).

El cifrado simétrico, al ser rápido y eficiente para el procesamiento de grandes volúmenes de datos, se utiliza en:

- **Cifrado de archivos y discos.** Por ejemplo, BitLocker en Windows y FileVault en macOS.
- **Cifrado de comunicaciones:** Por ejemplo, SSL/TLS para navegación web segura, cifrado de mensajes en aplicaciones de mensajería (WhatsApp, Signal).
- **Cifrado de bases de datos:** Por ejemplo, Transparent Data Encryption (TDE) en SQL Server y Oracle.
- **Cifrado en redes inalámbricas:** Por ejemplo, WPA2 y WPA3 en Wi-Fi.
- **Autentificación y verificación de la integridad:** Por ejemplo, HMAC (Hashed Message Authentication Code).

2.2.2. Tipos de cifrado simétrico

Cifrados de bloque

En los cifrados de bloque, los datos de entrada se dividen en bloques, comúnmente de 64 o 128 bits y se cifran de manera independiente usando una clave secreta. Esto aumenta la seguridad y resistencia a ataques, ya que pequeños cambios en los datos de entrada afectan significativamente el resultado cifrado.[5]

Cifrados de flujo

Los cifrados de flujo no dividen los datos en bloques, sino que cifran los datos continuamente, bit a bit o byte a byte, usando una secuencia de clave generada por un generador de números pseudoaleatorios. Una técnica comúnmente empleada para esta clave es el uso de registros de desplazamiento con retroalimentación lineal (LFSR).[3]

Un LFSR consiste en una serie de flip-flops[2] conectados en cadena que almacenan bits, desplazándolos en cada ciclo de reloj. La retroalimentación lineal se logra combinando ciertos bits del registro mediante operaciones XOR, según un polinomio de retroalimentación, generando una secuencia pseudoaleatoria de bits. Esta secuencia se combina con los datos originales, también mediante XOR, para producir el texto cifrado. Es muy útil ya que puede generar secuencias largas antes de repetirse.[5]

2.2.3. Algunos algoritmos utilizados para el cifrado simétrico

Algunos de los principales algoritmos para el cifrado simétrico son:

- **Data Encryption Standard (DES):** Es un algoritmo que cifra bloques de datos de 64 bits utilizando una clave de 56 bits. El proceso de cifrado se realiza en 16 rondas de sustitución y permutación, aplicando varias transformaciones a los datos.[27] El proceso de cifrado DES incluye los siguientes pasos:

  - **Permutación Inicial (IP)**: Los 64 bits del bloque de datos se permutan según una tabla predefinida.
  - **División en mitades**: El bloque permutado se divide en dos mitades de 32 bits, llamadas L y R.
  - **Rondas de Feistel**: Cada una de las 16 rondas aplica la siguiente serie de pasos:
    - **Expansión (E)**: Los 32 bits de R se expanden a 48 bits usando una expansión predefinida.
    - **Mezcla con la clave (XOR)**: La salida expandida se combina con una subclave derivada de la clave original mediante una operación XOR.
    - **Sustitución (S-box)**: Los 48 bits resultantes se dividen en ocho bloques de 6 bits cada uno, que se sustituyen usando una S-box predefinida para obtener 32 bits.
    - **Permutación (P)**: Los 32 bits resultantes se permutan según una tabla predefinida.
    - **Intercambio (Swap)**: Los resultados se combinan con L y se intercambian L y R.
  - **Permutación Final (FP)**: Después de las 16 rondas, L y R se combinan y se aplica una permutación final inversa a la inicial.

[2]Circuitos electrónicos fundamentales en la electrónica digital que actúan como elementos de almacenamiento de un solo bit de datos.

- **Triple DES (3DES):** Este algoritmo es una versión mejorada del DES que aplica el algoritmo DES tres veces a cada bloque de datos. Esto incrementa la seguridad en comparación con el DES original, que tiene una longitud de clave considerada insuficiente para resistir ataques de fuerza bruta con la tecnología moderna.[29] El proceso de cifrado en 3DES utiliza tres claves distintas (K1, K2 y K3) y sigue estos pasos:

  - **Cifrado con K1**: El bloque de datos se cifra utilizando la primera clave (K1) mediante el algoritmo DES.
  - **Descifrado con K2**: El resultado se descifra utilizando la segunda clave (K2) mediante el algoritmo DES en modo descifrado (realmente al descifrar con otra clave, lo que hace es desordenar aún más los datos).
  - **Cifrado con K3**: El resultado se cifra nuevamente utilizando la tercera clave (K3) mediante el algoritmo DES.

  El proceso de descifrado es similar pero aplica las claves en el orden inverso:

  - **Descifrado con K3**: El bloque de datos se descifra utilizando la tercera clave (K3).
  - **Cifrado con K2**: El resultado se cifra utilizando la segunda clave (K2).
  - **Descifrado con K1**: El resultado se descifra utilizando la primera clave (K1).

- **Advanced Encryption Standard (AES):** Es un algoritmo que cifra bloques de datos de 128 bits usando claves de 128, 192 o 256 bits. Durante el proceso de cifrado, los datos se organizan en una estructura llamada matriz de estado, que es una matriz de 4x4 bytes. Esta matriz de estado es la que se modifica a lo largo de las diferentes rondas del cifrado, cuyo número depende de la longitud de la clave: 10 rondas para claves de 128 bits, 12 rondas para claves de 192 bits y 14 rondas para claves de 256 bits.[26] Cada ronda incluye una serie de pasos específicos:

  - **SubBytes**: Cada byte en la matriz de estado se sustituye por otro byte utilizando una tabla de búsqueda predefinida (S-box).
  - **ShiftRows**: Las filas de la matriz de estado se desplazan cíclicamente un número determinado de posiciones.
  - **MixColumns**: Las columnas de la matriz de estado se mezclan utilizando una multiplicación matricial en el cuerpo F256.

- **AddRoundKey**: Se realiza una operación XOR entre la matriz de estado y una subclave derivada de la clave original.

2.3. El cifrado asimétrico

2.3.1. Introducción

En el cifrado asimétrico o cifrado de clave pública, a diferencia de lo que sucede en el cifrado simétrico, cada participante dispone de dos claves, una pública y una privada.

Para llevar a cabo la comunicación, el propietario de dichas claves comparte la clave pública con el resto de participantes, la cual, de forma unidireccional, se encarga de cifrar el mensaje. De este modo, la misma clave pública no sirve para descifrar; este proceso solo puede ser realizado con la clave privada, clave secreta que no hemos tenido que compartir en ningún momento.

De este modo, este tipo de cifrado es mucho más seguro, ya que aunque una persona ajena a la conversación intercepte el mensaje y sepa la clave con la que fue cifrado, necesita la clave privada para poder descifrarlo. Además de esto, la clave privada también permite verificar las firmas digitales y autentificar a ambos participantes.

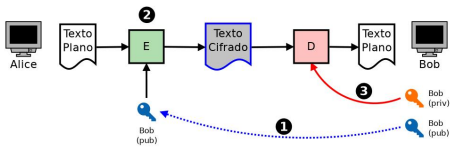


Figura 2.2: Representación de criptografía asimétrica[17]

2.3.2. Algunos algoritmos utilizados para el cifrado asimétrico

- **Rivest-Shamir-Adleman (RSA):** Es uno de los algoritmos de cifrado asimétrico más conocido y utilizado, su seguridad se basa en la dificultad de factorizar grandes números compuestos.[39]

  1. **Generación de claves.**
     - Se eligen dos números primos grandes p y q.
     - Se calcula n como el producto de p y q: $n = p \times q$.

- Se calcula $\phi(n)$ (la función de Euler) como $\phi(n) = (p-1) \times (q-1)$.
- Se elige un número e tal que $1 < e < \phi(n)$ y e sea coprimo con $\phi(n)$. Normalmente este número e tiene valor $65537 = 2^{16} + 1 = 10001$ (en hexadecimal).
- Se calcula d como el inverso multiplicativo de e módulo $\phi(n)$, es decir, $d \times e \equiv 1 \pmod{\phi(n)}$.

2. **Clave pública y privada.**
   - La clave pública es el par (n, e).
   - La clave privada es el par (n, d).
3. **Cifrado.**
   - Un mensaje m (representado como un número) se cifra utilizando la clave pública (n, e).
   - El texto cifrado c se calcula como $c = m^e \mod n$.
4. **Descifrado**
   - El texto cifrado c se descifra utilizando la clave privada (n, d).
   - El mensaje original m se recupera como $m = c^d \mod n$.

- **Elliptic Curve Cryptography (ECC):** Este algoritmo utiliza las propiedades matemáticas de las curvas elípticas para ofrecer un alto nivel de seguridad con claves más pequeñas en comparación con RSA.[22]

1. **Generación de claves.**
   - Se define una curva elíptica sobre un cuerpo finito F_p mediante una ecuación de la forma:

     $$y^2 = x^3 + ax + b$$

     donde $4a^3 + 27b^2 \neq 0$.
     Apunte: Si p es potencia de 2, la ecuación cambia a:

     $$y^2 + xy = x^3 + ax^2 + b$$

   - Se elige una clave privada k (un número aleatorio dentro del rango permitido).
   - Se calcula la clave pública como $P = kG$, donde G es un punto generador en la curva y kG representa la multiplicación escalar del punto generador G por k.
2. **Clave pública y privada**
   - La clave privada es el número k.
   - La clave pública es el punto P.

3. **Cifrado**
   - Alice quiere enviar un mensaje cifrado a Bob.
   - Bob tiene una clave privada k_B y una clave pública $P_B = k_B G$.
   - Alice codifica el mensaje m como un punto M en la curva.
   - Alice elige un número aleatorio r y calcula los puntos:

$$C_1 = rG$$

$$C_2 = M + rP_B$$

   - El par (C_1, C_2) es el texto cifrado que Alice envía a Bob.
4. **Descifrado**
   - Bob recibe el par (C_1, C_2).
   - Bob calcula:

$$M = C_2 - k_B C_1$$

   - Esto funciona porque:

$$C_2 - k_B C_1 = M + rP_B - k_B rG = M + r(k_B G) - k_B rG = M$$

2.4. El cifrado híbrido

Los criptosistemas asimétricos pueden combinarse con los simétricos. En este caso, lo que se realiza es intercambiar la clave que se va a utilizar para el cifrado simétrico mediante una comunicación inicial con cifrado asimétrico, aplicando así una seguridad mayor al punto débil del cifrado simétrico, que es el momento de compartir la clave establecida. Este sistema de cifrado híbrido se utiliza cuando los usuarios requieren la velocidad de la criptografía simétrica y la seguridad de la asimétrica.

2.5. La firma digital

Una firma digital es un mecanismo criptográfico que permite verificar la autenticidad e integridad de un mensaje, documento o software. Es análoga a una firma manuscrita o un sello, pero ofrece más seguridad inherente. Las firmas digitales aseguran que el contenido no ha sido alterado desde que fue firmado y que la firma proviene de una fuente verificada.

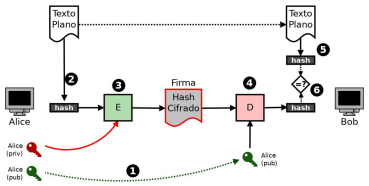


Figura 2.3: Representación de firma digital

2.5.1. Algoritmos utilizados para la realización de firmas digitales

- **Rivest-Shamir-Adleman (RSA):** En el contexto de firmas digitales, el firmante utiliza su clave privada para crear una firma, y cualquier persona con la clave pública correspondiente puede verificarla.[18]

  - **Proceso de firma.**
    1. Se calcula un hash del mensaje.
    2. Se cifra el hash con la clave privada del firmante.
    3. La firma digital es el hash cifrado.
  - **Verificación de la firma.**
    1. El receptor descifra la firma utilizando la clave pública del firmante para obtener el hash.
    2. Se calcula el hash del mensaje recibido.
    3. Si ambos hashes coinciden, la firma es válida.

- **Digital Signature Algorithm (DSA):** El DSA es un estándar para la firma digital que fue propuesto por el NIST en 1991. A diferencia del cifrado RSA, DSA se utiliza exclusivamente para la firma digital y no para el cifrado o intercambio de claves. Su seguridad se basa en la dificultad de resolver el problema del logaritmo discreto en grupos finitos.[39]

  1. **Generación de parámetros.**
     - Se elige un número primo grande p. Este número define el tamaño del grupo finito en el cual operará el algoritmo.
     - Se elige un número primo q tal que q divide a $p-1$. Típicamente, q es un número de 160 bits.
     - Se elige un número g tal que $g = h^{(p-1)/q} \mod p$ donde h es un número entero $1 < h < p-1$, asegurando que g tenga orden q en el grupo multiplicativo $\mathbb{Z}_p^*$.
     - Se elige un número aleatorio x tal que $0 < x < q$. Este número x es la clave privada.

- Se calcula y como $y = g^x \mod p$. Este valor y es la clave pública.

2. **Clave pública y privada.**
   - La clave pública es el cuarteto (p, q, g, y), donde p y q definen el grupo, g es el generador, e y es la parte pública de la clave.
   - La clave privada es el par (p, q, g, x), donde x es el número secreto que debe mantenerse confidencial.
3. **Proceso de firma.**
   a) Se elige un número aleatorio k tal que $0 < k < q$ y k es coprimo con q. Este k debe ser único para cada firma y se mantiene en secreto.
   b) Se calcula $r = (g^k \mod p) \mod q$. Este valor r es la primera parte de la firma.
   c) Se calcula $s = k^{-1} \times (\text{hash del mensaje} + x \times r) \mod q$, donde k^{-1} es el inverso multiplicativo de k módulo q. Este valor s es la segunda parte de la firma.
   d) La firma digital es el par (r, s), que se transmite junto con el mensaje.
4. **Verificación de la firma.**
   a) El receptor verifica que $0 < r < q$ y $0 < s < q$. Si alguna de estas condiciones no se cumple, la firma es inválida.
   b) Se calcula $w = s^{-1} \mod q$, el inverso multiplicativo de s módulo q.
   c) Se calcula $u_1 = (\text{hash del mensaje}) \times w \mod q$ y $u_2 = r \times w \mod q$.
   d) Se calcula $v = ((g^{u_1} \times y^{u_2}) \mod p) \mod q$.
   e) Si $v = r$, la firma es válida y se puede confiar en la autenticidad e integridad del mensaje. Si $v \neq r$, la firma es inválida y el mensaje puede haber sido alterado o no proviene del remitente esperado.

Capítulo 3

La computación cuántica

3.1. Qué es la computación cuántica

Los ordenadores clásicos funcionan con bits, que es la unidad mínima de información y pueden tener el valor 0 o 1. Así, en un momento concreto, el estado de un ordenador con N bits, será una secuencia de ceros o unos de tamaño N.

En la computación cuántica esto no sucede así, y es que aquí la unidad mínima de información no es el bit, sino el qubit. Un qubit no vale 0 o 1 (o al menos no hasta el momento en el que se mide), sino que se encuentra en una superposición de estos dos estados, es decir, vale a la vez 0 y 1.[11][30]



Figura 3.1: Idea de un qubit[13]

Esto puede parecer muy disparatado a simple vista pero, para entenderlo mejor, debemos pensar en un qubit como si se tratase del lanzamiento al aire de una moneda. Mientras esta está girando, no podemos asumir que vale ni cara, ni cruz; la moneda se encuentra en una superposición de ambos valores. Sin embargo, una vez la detenemos, solo tendrá uno de ellos, esto es sinónimo del instante en el que medimos un qubit.

Esta es la base del poder de los ordenadores cuánticos con respecto a los

clásicos, y es que, mientras un estado de N bits es una sucesión de ceros y unos de tamaño N, un estado de N qubits es la combinación de todas las posibles colecciones de N unos y ceros, teniendo así un tamaño 2^N. A su vez, cada una de estas combinaciones tiene asignado un coeficiente, que son números complejos cuyos cuadrados de sus módulos suman en total uno, y hacen referencia a la probabilidad de que suceda.[11][10][33][34]

Así, mientras con 3 bits necesitaríamos 7 iteraciones para llegar al 111, con tres qubits solo necesitamos una iteración, ya que esos tres qubits se encuentran desde el principio, en mayor o menor medida, en todos los estados posibles. A medida que aumentamos el número de qubits, la capacidad de su computación crece de forma exponencial, es decir, se duplica por cada qubit. Esto nos permite tener resultados asombrosos a nivel de reducción de complejidad de algoritmos.

Como podemos observar, los qubits, al aprovechar esta propiedad de la física cuántica, son una herramienta muy poderosa. Sin embargo, de momento tienen grandes complicaciones, entre las que destacan: los qubits necesitan estar a una temperatura cerca del cero absoluto[1] y no interaccionar con nada, ni siquiera con ellos mismos, porque pierden su propiedad de superposición.

Por esto, el avance en este campo busca crear ordenadores cuánticos con mayor número de qubits, que duren más tiempo ejecutando programas sin interactuar entre ellos y que necesiten condiciones menos extremas para funcionar.

3.2. Programación de los ordenadores cuánticos

En los ordenadores clásicos, para pasar de un estado de la máquina a otro, se realiza una operación lógica sobre los bits que definen el estado actual del ordenador. A la combinación de dichas puertas lógicas se le llama algoritmo, y, tras pasar por todas ellas, nos da la solución al problema para el que se ha programado.

En los ordenadores cuánticos esto es parecido, ya que pasamos de un estado a otro a través de operaciones lógicas cuánticas. Estas puertas lógicas cuánticas son matrices unitarias con coeficientes en los complejos que aprovechan la estructura de espacio de Hilbert[2] que tiene el cuerpo de los complejos. Algunas de ellas son: Pauli, Hadamard, CNOT o Toffoli.[11]

Un cálculo en un ordenador cuántico funciona preparando una superposición de todos los estados computacionales posibles, que son ondas con la probabilidad de cada uno de los resultados de la medición del sistema. A par-

[1] Cero absoluto: -273.15 grados celsius.

[2] Un espacio de Hilbert es una generalización del espacio euclídeo.

tir del algoritmo diseñado, se van realizando interferencias entre las superposiciones de las ondas, anulando resultados por interferencias o amplificando otros si coinciden en un resultado concreto. Estos resultados amplificados son las soluciones para el cálculo.[15]

Mientras que los ordenadores convencionales dan una solución muy buena casi siempre, los ordenadores cuánticos darán siempre la solución óptima, ya que pueden comparar entre todas las posibilidades a la vez.

3.3. El peligro de la llegada de los ordenadores cuánticos

Como hemos visto, la potencia de la computación cuántica respecto a la tradicional es exponencial. Al evaluar al mismo tiempo todas las posibilidades, es especialmente útil rompiendo claves de cifrado, ya que puede factorizar números en un tiempo mucho menor del posible hasta hoy en día. Entre estos algoritmos de "descifrado"destacan especialmente dos:

3.3.1. Algoritmo de Shor

El algoritmo de Shor, desarrollado por Peter Shor en 1994, es un algoritmo cuántico para factorizar números enteros en tiempo $O((\log N)^3)$ y espacio $O(\log N)$. El algoritmo de Shor convierte el problema de la factorización en un problema de encontrar el orden de un elemento en un grupo multiplicativo modular, siendo esto un paso crítico que permite la aplicación de la computación cuántica. Para ello, utiliza la transformada de Fourier cuántica y el principio de superposición y entrelazamiento de estados cuánticos para encontrar el período de una función, que luego se utiliza para encontrar los factores de N.[38]

Los pasos del Algoritmo de Shor son:

1. **Seleccionar un número *a*.** Elegir un número a al azar tal que $1<a<N$ y calcular el máximo común divisor (MCD) de a y N. Si el MCD es distinto de 1, se ha encontrado un factor de N.

2. **Calcular el orden *r* de *a* módulo *N*.** Este es el menor entero positivo tal que $a^r \equiv 1 \mod N$. Este paso se realiza utilizando una computadora cuántica.

3. **Comprobar si *r* es par.** Si r es impar, el proceso se repite con un nuevo a. Si es par, calcular $a^{r/2}$.

4. **Encontrar factores.** Utilizar la identidad $a^{r/2} \equiv -1 \mod N$ para encontrar los factores no triviales de N, calculando $\text{MCD}(a^{r/2} - 1, N)$ y $\text{MCD}(a^{r/2} + 1, N)$.

Un ejemplo con $N = 15$:

1. **Elegir a.** Supongamos que elegimos $a = 2$.
2. **Calcular el orden r de a módulo N**: El orden r es el menor número tal que $2^r \equiv 1 \mod 15$. Usando una computadora cuántica, encontramos que $r = 4$.
3. **Verificar r.** Como r es par, calcular $2^{r/2} = 2^2 = 4$.
4. **Calcular factores.** Los factores se encuentran como $\text{MCD}(4-1, 15) = 3$ y $\text{MCD}(4 + 1, 15) = 5$. Así, hemos factorizado 15 como 3×5.

3.3.2. Algoritmo de Grover

El algoritmo de Grover, desarrollado por Lov Grover en 1996, es un algoritmo cuántico diseñado para buscar en una base de datos no estructurada con N elementos en $O(\sqrt{N})$ tiempo, lo cual es una mejora cuadrática respecto al tiempo $O(N)$ que requeriría una búsqueda clásica. Este algoritmo se puede aplicar a una amplia variedad de problemas de búsqueda y optimización.[14][30][4]

El algoritmo utiliza la superposición y el entrelazamiento de estados cuánticos, aplicando una secuencia de operaciones cuánticas llamadas Ïteraciones de Grover"para amplificar la probabilidad del estado correcto.

Los pasos del Algoritmo de Glover son:

1. **Preparación del Estado Inicial.** Se debe preparar un registro cuántico en una superposición igual de todos los posibles estados.
2. **Operador de Oráculo.** Se aplica un oráculo cuántico que marca el estado correcto invirtiendo su amplitud.
3. **Difusión de Grover.** Se aplica una operación de difusión que amplifica la amplitud del estado correcto.
4. **Repetición de Iteraciones.** Se repiten las operaciones del oráculo y la difusión aproximadamente $O(\sqrt{N})$ veces.
5. **Medición.** Por último se debe medir el estado del registro cuántico para obtener la solución.

Capítulo 4

Criptografía Post-Cuántica

4.1. Concurso NIST

El NIST (National Institute of Standards and Technology), es una institución que promueve la innovación y la competitividad global a través del avance de la ciencia y la tecnología. Establece estándares de medición, guías de ciberseguridad, normas para la construcción y, entre otros, estándares de criptografía que son esenciales para la seguridad de la información. Su trabajo mejora la calidad de vida y la seguridad económica a nivel mundial.[40]

Debido a la preocupación respecto a los algoritmos cuánticos, los cuales podrían vulnerar a gran escala nuestros datos en un futuro, el NIST inició, en 2016, un concurso abierto al público para recoger propuestas de algoritmos post-cuánticos, es decir, algoritmos capaces de cifrar los datos de forma segura ante los ordenadores cuánticos.[28]

La inscripción a dicho concurso cerró en noviembre de 2017 y, desde entonces, los distintos algoritmos han ido avanzando por distintas rondas, descartando los menos prometedores y mejorando los que más.

En las rondas iniciales, los criptosistemas tenían atributos de diseño y fundamentos matemáticos completamente diferentes entre sí, entre los que destacaban los criptosistemas basados en multivariantes, en códigos y en retículos.[43]

Estas diferencias tan grandes complicaban la comparación directa de los candidatos, haciéndola prácticamente imposible. Para ello, durante todo este tiempo, el NIST ha llevado a cabo un análisis exhaustivo de los algoritmos presentados de forma abierta y transparente para el público, animando a la propia comunidad criptográfica a realizar también sus propios análisis y evaluaciones de estos algoritmos.[42]

A día de hoy nos encontramos con que estos algoritmos han avanzado

mucho, destacando dos puntos importantes.

El primer punto, es que en 2022 el NIST anunció la selección de cuatro de los algoritmos del concurso para su estandarización, ya que se ha considerado que estos algoritmos han pasado las respectivas fases previas de evaluación y son lo suficientemente seguros y eficientes.[46]

El segundo punto, es que actualmente nos encontramos en la ronda 4 de este concurso que inició en 2016. Ya que, a pesar de que se hayan estandarizado algunos algoritmos, el NIST busca estar en constante mejora y adaptabilidad de sus algoritmos, diversificando sus soluciones y manteniendo a la comunidad activa en este campo.[44]

4.1.1. Algoritmos estandarizados en 2022

- **CRYSTALS-Kyber.** Basado en retículos, concretamente en el problema del aprendizaje con errores (LWE[1]), es un mecanismo de establecimiento de claves(KEM). El algoritmo ML-KEM que se va a desarrollar en este trabajo es una variación de este.[9]
- **CRYSTALS-Dilithium.** Basado en retículos, concretamente en el problema del aprendizaje con errores (LWE), sirve para la firma digital.[8]
- **FALCON.** Basado en retículos, concretamente en el problema del aprendizaje con errores (LWE) y el problema del vector más corto (SVP), sirve para la firma digital.[32]
- **SPHINCS+.** Basado en árboles hash, sirve para la firma digital, y ha sido estandarizado para tener una alternativa si en algún caso fallasen los cifrados basados en retículos.[51]

4.1.2. Algoritmos que se encuentran en la ronda 4

Todos ellos son algoritmos de establecimiento de claves (KEMs).

- **BIKE (Bit Flipping Key Encapsulation)**. Está basado en códigos estructurados.[47]
- **Classic McEliece**. Está basado en códigos Goppa binarios[2], que presentan una gran seguridad pero requieren de claves públicas de gran tamaño.[48]

[1]LWE: Learning With Errors.

[2]Un código Goppa binario es un tipo de código algebraico que sirve para detectar y corregir errores en la transmisión de datos.

- **HQC (Hamming Quasi-Cyclic)**. Está basado en códigos estructurados.[49]
- **SIKE (Supersingular Isogeny Key Encapsulation)**. Está basado en curvas elípticas. Sin embargo, el propio equipo de SIKE reconoce que es inseguro y no debe de utilizarse, por lo que solo sigue en consideración debido a que sus claves y tamaños de cifrado utilizados tienen un tamaño muy pequeño.[50]

4.2. Nuevo concurso de criptografía post-cuántica para firmas digitales

En septiembre 2022, el NIST inició un nuevo concurso de criptografía post-cuántica centrado únicamente en firmas digitales. La principal razón fue que que querían diversificar aún más la base matemática de los algoritmos criptográficos y buscar firmas digitales que no se basasen en los problemas de retículos estructurados como lo hacen la mayoría de algoritmos que se estandarizaron en 2022. Una segunda razón fue que, en la ronda 4 del concurso inicial del NIST, ya no había algoritmos destinados a la firma digital, siendo todos ellos para el establecimiento de claves.[45]

Aunque buscasen diversificar, también aceptaron propuestas basadas en retículos si demostraban un rendimiento significativamente superior a CRYSTALS-Dilithium y FALCON en aplicaciones relevantes y ofrecían propiedades de seguridad sustanciales.

Así, este concurso cerró en junio de 2023, recibiendo un total de 50 propuestas, de las cuales 40 han sido consideradas completas y adecuadas para la evaluación.

Capítulo 5

Criptografía basada en retículos

En el concurso del NIST, más de la mitad de los algoritmos que se presentaron al inicio, de los que han ido avanzando por las distintas rondas e incluso de los que se estandarizaron en 2022, están basados en los retículos.

Por esto, vamos a centrarnos en qué son los retículos, comprendiéndolos un poco mejor y explicando sus funcionalidades y los problemas que se pueden crear a partir de ellos.

Además, en el siguiente capítulo, analizaremos y desarrollaremos un algoritmo para la encapsulación de claves basado en retículos modulares; por lo que esta introducción es completamente necesaria para comprenderlo. Dicho algoritmo se denomina *ML-KEM*, es una variación del algoritmo *CRYSTALS-Kyber* y es el algoritmo en el que se fundamenta el *FIPS*[1] *203*, un estándar oficial publicado por el NIST el pasado 13 de agosto de 2024. Por lo tanto, este algoritmo ya es completamente funcional y seguro contra algoritmos post-cuánticos.[23]

Como apunte importante, este trabajo comenzó a desarrollarse en base a un borrador de dicho algoritmo presentado el pasado 23 de agosto de 2023 y, tras la publicación del algoritmo final hace apenas unas semanas, se ha tenido que adaptar parte del código y de las implementaciones.[24][23]

5.1. Qué es un retículo

Un retículo en $\mathbb{R}^n$ se define como el conjunto de todas las combinaciones lineales de un conjunto de vectores linealmente independientes $b_1, ..., b_n$ con

[1]FIPS: Federal Information Processing Standars

coeficientes enteros. La base del retículo se representa como una matriz de vectores base.[52]

A partir de la base de un determinado retículo, mediante combinaciones lineares entre los vectores que la componen, podemos obtener todos los demás vectores discretos[2] que conforman ese retículo. Por esto, el vector cero siempre se encuentra en el retículo.

Estas bases de los retículos son las que hacen que los retículos sean objetos matemáticos tan buenos, ya que pueden ser descritos con "relativamente" poca cantidad de información. No existe otro conjunto con menos elementos que tenga esta propiedad. Cada retículo tiene un número infinito de bases y no todas las bases son igual de buenas para trabajar con ellas.[7]

Para definir las bases de un retículo, se busca utilizar vectores lo más cortos posibles y que sean ortogonales entre sí, ya que esto significa utilizar menos almacenamiento para describirlo. Para los retículos de dos dimensiones, esta tarea es relativamente sencilla (se puede utilizar el algoritmo de Lagrange) pero, a medida que aumentamos la dimensión, su dificultad crece rápidamente.[52][21][7]

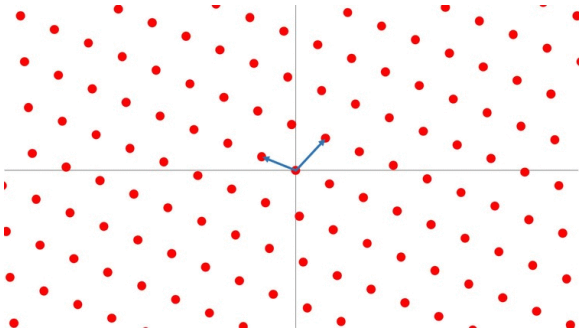


Figura 5.1: Ejemplo de base de un retículo[6]

5.1.1. Retículos q-áricos

Los retículos q-áricos (q-Ary Lattices en inglés) son una clase de retículos definidos sobre el anillo de enteros módulo q, $\mathbb{Z}_q$.

Un anillo de enteros módulo q es una estructura algebraica que consiste en los enteros cuyas operaciones suma y multiplicación se realizan bajo una relación de equivalencia módulo q.[52][21]

Para una matriz A, se definen los siguientes conjuntos:

[2] No puede haber dos vectores más cerca entre sí que una distancia λ, que es un número real positivo.

1. $\Lambda_q^{\perp}(A) = \{e \in \mathbb{Z}^m : Ae = 0 \mod q\}$
2. $\Lambda_q(A) = \{y \in \mathbb{Z}^m : \exists s \in \mathbb{Z}^n, A^T s = y \mod q\}$

Siendo $e \in \mathbb{Z}^m$ el conjunto de vectores tales que la multiplicación de la matriz A por e da como resultado el vector nulo módulo q.

Así mismo, $\Lambda_q(A)$ es el conjunto de todos los vectores y en $\mathbb{Z}^m$ que pueden expresarse como $y = A^T s \mod q$, para algún vector s en $\mathbb{Z}^n$. Este conjunto forma un retículo en $\mathbb{Z}^m$, caracterizado por las combinaciones lineales de las columnas de A^T con coeficientes en $\mathbb{Z}^n$, tomando el resultado módulo q.

Estos conjuntos son subgrupos discretos de $\mathbb{Z}^m$ y están relacionados entre sí. Bajo la escala adecuada, son duales entre sí[3] de la siguiente manera: $\Lambda_q^{\perp}(A) = q\Lambda_q(A)^*$ y $\Lambda_q(A) = q\Lambda_q^{\perp}(A)^*$, donde Λ^* denota el retículo dual de Λ.

5.1.2. Retículos ideales

Los retículos ideales (Ideal Lattices en inglés) son un tipo especial de retículos con estructuras algebraicas adicionales. Estos retículos ideales se construyen a partir de un anillo de enteros.[52][21]

Un ideal I de un anillo R es un subconjunto del anillo que es cerrado bajo adición y multiplicación por cualquier elemento del anillo. Esto significa que si $r \in R$ y $x \in I$, entonces $r \cdot x \in I$.

Dado un ideal I en un anillo R, se puede generar un retículo en un espacio vectorial a través de un proceso llamado embebimiento. El embebimiento es un proceso que toma los elementos de un ideal y los convierte en vectores de coeficientes en un espacio vectorial, como $\mathbb{R}^n$.

5.1.3. Función Gaussiana Discreta

La función gaussiana discreta es una herramienta fundamental en la criptografía basada en retículos debido a sus propiedades estadísticas, que permiten manejar el ruido y generar elementos en el retículo de manera segura. Esencialmente, esta función se utiliza para definir distribuciones de probabilidad sobre los elementos de un retículo, facilitando la construcción de problemas criptográficos difíciles de resolver, como la búsqueda de vectores cortos o la resolución de ecuaciones lineales con errores.[52][21]

[3]La dualidad entre $\Lambda_q(A)$ y $\Lambda_q^{\perp}(A)$ significa que los vectores de un retículo están alineados de forma predecible respecto a los vectores del otro mediante el producto interno, pero con un desfase o escala debido al factor q. Esta escala introduce una conexión estructural entre ambas redes de puntos en el espacio.

Definición de la Función Gaussiana Discreta

Dada una constante $s > 0$ y un vector centro $c \in \mathbb{R}^n$, la función gaussiana sobre un conjunto discreto $\Lambda \subset \mathbb{Z}^n$ se define como:

$$\rho_{s,c}(x) = \exp\left(-\frac{\pi\|x-c\|^2}{s^2}\right), \quad \text{para todo } x \in \Lambda.$$

Aquí, $\rho_{s,c}(\Lambda)$ representa la suma de $\rho_{s,c}(x)$ sobre todos los x en Λ.

Distribución Gaussiana Discreta

La distribución gaussiana discreta sobre Λ con centro c y parámetro s se define como:

$$D_{\Lambda,s,c}(y) = \frac{\rho_{s,c}(y)}{\rho_{s,c}(\Lambda)}, \quad \text{para todo } y \in \Lambda.$$

En caso de no especificarse los parámetros s y c, se asume que $s = 1$ y $c = 0$.

5.2. Principales problemas con retículos

5.2.1. Problema del vector más corto (SVP)

Dada una base de un retículo, este problema se basa en encontrar el vector más corto de la misma (el vector cero no vale).

Formalmente, se debe de encontrar un vector dentro de una determinada base de un retículo que tenga una distancia λ_1, donde λ_1 es la distancia mínima desde el origen hasta cualquier otro punto del retículo, excluyendo el vector cero.[52][21][7]

Esto se debe a que los retículos están formados por puntos distribuidos de manera discreta en el espacio, lo que implica que existe una distancia mínima entre ellos. Dicha estructura discreta garantiza que los puntos no están distribuidos de manera continua, sino que están separados por esta distancia λ_1.

Este problema se considera NP-Duro[4] para reducciones aleatorias, por lo que se ha creado una alternativa un poco más sencilla, denominada Problema del vector más corto aproximado.

[4]No existe algoritmo que lo resuelva en tiempo polinómico.

5.2.2. Problema del vector más corto aproximado (ASVP)

Esta variante al SVP ya no busca encontrar el vector más corto dentro de una determinada base, sino que acepta que el vector sea más pequeño que un valor de aproximación dado (sigue sin valer el vector cero). Este valor suele ser polinómico en relación con la dimensión del retículo y se describe como $\gamma\lambda_1$.[52][21][7]

Los algoritmos más eficientes que se conocen para resolver este problema en tiempo polinómico solo pueden manejar valores de aproximación que tienen la forma de funciones sub-exponenciales, es decir, que crecen un poco más rápido que una función polinómica, pero más lento que una función exponencial en relación con la dimensión del retículo.

5.2.3. GapSVP

En el ámbito de la criptografía, se tiene especial interés en los problemas de decisiones, por lo que también se ha tratado de convertir el SVP en un problema decisional, dando como resultado como resultado el problema GapSVP.[52][21][7]

Este problema trata de determinar si un retículo contiene un vector no nulo de longitud a lo sumo 1 o si el vector no nulo más corto tiene una longitud mayor que un parámetro dado γ. Se asume que una de estas dos condiciones es verdadera y el objetivo es decidir cuál de las dos lo es.

5.2.4. Problema del vector más cercano (CVP)

En este problema, se da un retículo definido por una base y un vector objetivo en el espacio vectorial ambiente, y la tarea es encontrar un vector en el retículo que sea el más cercano al vector objetivo.[52][21][7]

5.2.5. Decodificación a Distancia Acotada (BDD)

De manera análoga que se ha hecho con SVP y ASVP, este problema es una simplificación del CVP. Aquí lo que se asegura es que el vector objetivo se encuentra al menos a una distancia $d < \frac{\lambda_1}{2}$ del retículo, siendo λ_1 la distancia máxima entre dos retículos.[52][21][7]

5.2.6. Solución de enteros cortos (SIS)

Este problema fue introducido por Ajtai y consiste en encontrar un vector entero no nulo que satisfaga las condiciones $Ax = 0 \mod q$ y $\|x\| \leq \beta$, dados una matriz aleatoria A y un límite β.[52][21][7]

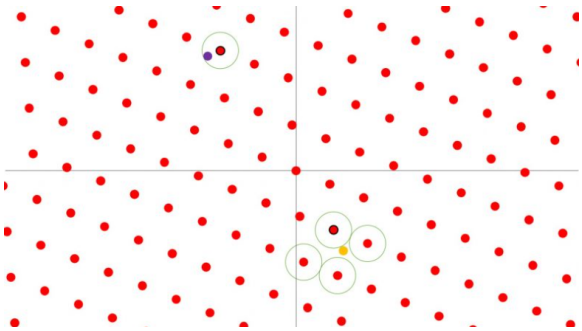


Figura 5.2: Amarillo: CVP (inalcanzable), Morado: BDD (alcanzable)[6]

Ajtai logró demostrar que resolver el problema SIS en el caso promedio es tan difícil como resolver ciertos problemas de retículos en el peor de los casos, asegurando que la seguridad del problema SIS se basa en suposiciones estándar de complejidad del peor caso.

Las soluciones de este problema forman un retículo entero, y el objetivo es encontrar un vector corto en dicho retículo, lo que es una instancia del problema del vector más corto aproximado (ASVP) con un factor de aproximación desconocido.

Este problema es útil para construir funciones hash unidireccionales y resistentes a colisiones, pero su uso para firmas digitales y esquemas de identificación no es tan bueno.

5.2.7. Aprendiendo con errores (LWE)

Este problema, introducido por Regev en 2005, trata de resolver un sistema de ecuaciones lineales que tiene ruido[5], es decir, encontrar un vector secreto a partir de múltiples observaciones de pares ruidosos.[52][21][7]

Formalmente, dado un vector secreto $\mathbf{s}$ y una distribución de ruido sobre los enteros, la distribución LWE se define al crear un vector aleatorio $\mathbf{a}$, calcularle el producto escalar con el vector secreto y después agregarle ruido, resultando en el par $(a, b = \langle a, s\rangle + e)$. La distribución de ruido suele ser una Gaussiana discretizada.

LWE como problema de búsqueda

En el problema de búsqueda, se dan muestras $(a_i, b_i) \leftarrow A_{s,\chi}$ y se debe determinar el vector secreto s.

[5]Pequeñas desviaciones o errores añadidos a los resultados esperados de las ecuaciones.

LWE como problema de decisión

En la variante de decisión, se reciben muestras $(a_i, b_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, y se tiene que decidir si provienen de la distribución LWE $A_{s,\chi}$ o de una distribución uniforme aleatoria $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

5.2.8. Aprendizaje en anillo con errores (Ring-LWE)

El cambio principal entre LWE y Ring-LWE radica en la estructura algebraica subyacente que se utiliza. Ring-LWE se basa en polinomios en un anillo $R = \mathbb{Z}[x]/(x^n + 1)$, con coeficientes en $\mathbb{Z}_q$. Las operaciones y errores se manejan utilizando polinomios, aprovechando la estructura cíclica de los anillos para optimizaciones; lo que permite realizar operaciones más eficientes y reduce significativamente el tamaño de las claves y la complejidad computacional.[52][21][7]

5.2.9. Aprendizaje modular con errores (Module-LWE)

El problema del aprendizaje modular con errores (Module-LWE) es una extensión de los problemas LWE y Ring-LWE, donde se trabaja con un módulo R_q^k, que se construye mediante el producto cartesiano k-veces de un anillo de polinomios R_q.

Específicamente, el módulo M se define sobre un anillo de polinomios de la forma $R = \mathbb{Z}_q[x]/(f(x))$, donde $f(x)$ es un polinomio irreducible y $\mathbb{Z}_q$ representa los enteros módulo q.

Así, los vectores secretos y los de ruido son elementos de este módulo, lo que introduce una mayor complejidad y seguridad respecto a *LWE* y *Ring-LWE*.

Esta estructura modular permite definir problemas en dimensiones más altas y ajustar los parámetros de seguridad de manera más precisa. Además de esto, también se aprovecha a la hora de realizar operaciones de manera eficiente y utilizar optimizaciones algebraicas, manteniendo un equilibrio entre la seguridad y la eficiencia computacional.[19][20][31]

Capítulo 6

Module-Lattice-Based Key-Encapsulation Mechanism Standar (FIPS 203)

6.1. Descripción del estándar FIPS 203

FIPS 203 es un estándar cuya versión final se presentó el 13 de agosto de 2024, mismo día en el que se estandarizó. FIPS 203 es una derivación directa del estándar CRYSTALS-Kyber, que ya fue estandarizado en 2022.

Este nuevo estándar especifica un conjunto de algoritmos para establecer, a través de un canal público, una clave secreta compartida segura frente a algoritmos cuánticos, aunque el atacante disponga de un gran número de ordenadores cuánticos (mayor potencia a la hora de ejecutar los algoritmos). Para ello, se utiliza el método de encapsulación de claves (KEM) basado en el Módulo de aprendizaje con errores (Module-LWE) de los retículos.

Para la creación de la clave secreta utilizando ML-KEM, se especifican tres conjuntos de parámetros posibles. En orden creciente de seguridad y decreciente en rendimiento, son: *ML-KEM-512*, *ML-KEM-768* y *ML-KEM-1024*.

Además, el NIST comenta que va a re-evaluar el funcionamiento y la calidad de este estándar cada cinco años; así como cada vez que haya un avance tecnológico o se descubra alguna debilidad que pueda afectar al correcto funcionamiento del estándar.

Todo la información de este capítulo está extraída del siguiente documento: [25]

6.1.1. Método de encapsulación de claves (KEM)

En un KEM, la creación de una clave secreta compartida comienza con un usuario A generando dos claves gracias a un algoritmo de generación de claves, la clave de encapsulación y la clave de desencapsulación.

A mantiene la clave de desencapsulación (DK a partir de ahora) privada para él y comparte la clave de encapsulación (EK a partir de ahora) al usuario B, que es con el que se desea establecer la clave secreta.

Con esta clave EK, mediante un algoritmo de encapsulación, B genera una copia de la clave secreta (K) que se busca establecer y un texto cifrado (c). Quedándose K para él y enviándole c al usuario A.

Finalmente, A puede utilizar el algoritmo de desencapsulación introduciéndole como parámetros DK y el texto cifrado c para obtener la misma clave secreta, denominada en este caso, por diferenciar una de otra K'.

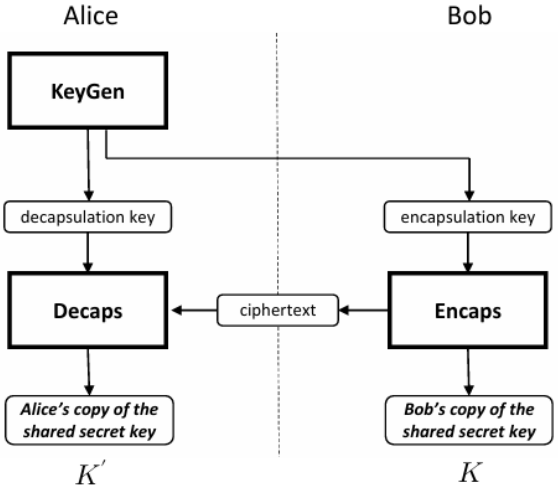


Figura 6.1: Funcionamiento del mecanismo KEM[25]

6.1.2. Módulo de aprendizaje con errores (Module-LWE) como base para el KEM

En este estándar, la seguridad del KEM se basa en la dificultad computacional del problema del Módulo de aprendizaje con errores (Module-LWE).

Como hemos visto anteriormente, el problema de Module-LWE es una extensión de los problemas LWE y Ring-LWE, que utiliza módulos construidos a partir del producto cartesiano de un anillo de polinomios en lugar de vectores simples o anillos.

Tanto los vectores secreto como los vectores de error se representan en un módulo, lo que introduce un nivel de ruido en los cálculos que dificulta significativamente la recuperación de los secretos sin la clave adecuada.

Esta estructura modular permite manejar problemas en dimensiones más altas, utilizando optimizaciones algebraicas para equilibrar la eficiencia computacional con una mayor robustez criptográfica.

6.2. La construcción del ML-KEM

La construcción del esquema ML-KEM se realiza en dos pasos.

Primero, todo lo explicado anteriormente se utiliza para construir un esquema de cifrado de clave pública (PKE) a partir del modelo de problema Module-LWE. Este esquema PKE no se considera lo suficientemente seguro por sí solo y no se debería de utilizar de manera aislada.

Segundo, este esquema PKE es convertido en un mecanismo de encapsulación de claves (KEM) utilizando la transformada de Fujisaki-Okamoto (FO).

Lo que hace esta transformada es crear un valor aleatorio a partir del mensaje a cifrar y utilizar este propio valor en el proceso de cifrado. Después, en el proceso de descifrado, se verifica que el mensaje descifrado coincida con el mensaje original a través de la generación de nuevo del valor aleatorio. Si hay una discrepancia, indica que el texto cifrado fue alterado y la clave K' se genera incorrectamente.

Debido a ciertas propiedades de la transformada FO, el ML-KEM resultante tiene un significante aumento en la seguridad ante ataques respecto al esquema PKE del primer paso. Se considera que esta seguridad satisface la seguridad IND-CCA2[1].

Sin embargo, este modelo ML-KEM, combinado con otros modelos KEM, no tiene porqué satisfacer la seguridad IND-CCA2, por lo que debería de ser supervisado y comprobado por la persona que lo implemente.

6.3. Características del algoritmo ML-KEM

6.3.1. Dominio NTT

Una característica destacada de ML-KEM es el uso de la Transformada Numérica (NTT), la cual puede verse como una versión especializada de la Transformada Discreta de Fourier. La NTT convierte un polinomio $f \in R_q$

[1]IND-CCA2: Indistinguishability under Adaptive Chosen Ciphertext Attack.

en una representación alternativa como un vector $\hat{f} \in T_q$ de polinomios lineales, lo que permite una multiplicación de polinomios significativamente más rápida. Operaciones como la suma, el redondeo y el muestreo se pueden realizar en ambas representaciones.

En ML-KEM, la NTT se utiliza para mejorar la eficiencia de la multiplicación en el anillo R_q, que está definido como $\mathbb{Z}_q[X]/(X^n + 1)$, donde los polinomios tienen la forma $f = f_0 + f_1 X + \cdots + f_{255} X^{255}$, con coeficientes $f_j \in \mathbb{Z}_q$ y operaciones aritméticas módulo $X^n + 1$.

El anillo R_q es isomorfo a otro anillo T_q, que es una suma directa de 128 extensiones cuadráticas de $\mathbb{Z}_q$. La NTT proporciona un isomorfismo[2] computacionalmente eficiente entre estos dos anillos. Al aplicarse a un polinomio $f \in R_q$, se obtiene un polinomio $\hat{f} := \mathrm{NTT}(f)$ en T_q, conocido como la representación NTT de f. La propiedad de isomorfismo garantiza que:

$$f \times_{R_q} g = \mathrm{NTT}^{-1}(\hat{f} \times_{T_q} \hat{g})$$

donde $\times_{R_q}$ y $\times_{T_q}$ son las multiplicaciones en R_q y T_q, respectivamente.

Dado que T_q es un producto de 128 anillos, cada uno compuesto por polinomios de grado a lo sumo uno, la multiplicación en T_q es mucho más eficiente que en R_q. Por esta razón, la NTT es una parte esencial de ML-KEM y no solo una optimización.

Ambos anillos, R_q y T_q, tienen estructura de espacio vectorial sobre $\mathbb{Z}_q$, lo que hace natural representar sus elementos como vectores en $\mathbb{Z}_q^n$. Por ello, la NTT y su inversa operan sobre arrays de longitud n de enteros módulo q, que representan elementos de T_q o R_q, respectivamente.

A nivel de estructura matemática, la NTT puede entenderse de la siguiente forma:

En nuestro algoritmo ML-KEM, q es el primo $3329 = 2^8 \cdot 13 + 1$, y $n = 256$. Hay 128 raíces primitivas de la unidad de orden 256 y ninguna raíz primitiva de la unidad de orden 512 en $\mathbb{Z}_q$.

Una raíz de la unidad de orden 256 es una raíz que, al elevarla a 256, da 1 en módulo q. Una raíz primitiva es es una raíz especial porque genera todas las demás raíces cuando se eleva a diferentes potencias, es decir, no repite ningún número hasta llegar a 256 donde vuelve a valer 1 en el módulo q.

Utilizaremos $\zeta = 17 \in \mathbb{Z}_q$, que es una raíz primitiva de la unidad de orden 256 módulo q. Así, $\zeta^{128} \equiv -1$.

Definimos $\mathrm{BitRev}_7(i)$ como el entero representado tras invertir el orden de sus 7 bits $i \in \{0, \ldots, 127\}$.

[2]Isomorfismo: Conversión eficiente.

El polinomio $X^{256}+1$ se factoriza en 128 polinomios de grado 2 módulo q como sigue:

$$X^{256}+1=\prod_{i=0}^{127}\left(X^2-\zeta^{2\cdot \mathrm{BitRev}_7(i)+1}\right)$$

Por lo tanto, $R_q := \mathbb{Z}_q[X]/(X^{256}+1)$ es isomorfo a una suma directa de 128 extensiones cuadráticas de campos de $\mathbb{Z}_q$ denotada T_q. Específicamente, este anillo tiene la estructura

$$T_q := \bigoplus_{i=0}^{127}\mathbb{Z}_q[X]/\left(X^2-\zeta^{2\cdot \mathrm{BitRev}_7(i)+1}\right)$$

Así, la representación NTT $\hat{f}\in T_q$ de un polinomio $f\in R_q$ es el vector que consiste en el resto de aplicarle ese módulo. Dichos restos son vectores de grado como máximo uno:

$$\hat{f} := \left(f \mod \left(X^2-\zeta^{2\cdot \mathrm{BitRev}_7(0)+1}\right), \ldots, f \mod \left(X^2-\zeta^{2\cdot \mathrm{BitRev}_7(127)+1}\right)\right)$$

Obteniendo de esta forma la siguiente igualdad:

$$f \mod \left(X^2-\zeta^{2\cdot \mathrm{BitRev}_7(i)+1}\right) = \hat{f}[2i]+\hat{f}[2i+1]X$$

para i desde 0 hasta 127.

6.3.2. Multiplicación en el dominio NTT

La suma y la multiplicación por un escalar en el anillo T_q se realiza coordenada por coordenada, es decir, cada operación se realiza de manera individual. Para la multiplicación entre dos vectores del anillo T_q esto varía un poco:

Como $\hat{f}\in T_q$ es un vector de 256 coeficientes, $\hat{f}[2i]$ para el término constante y $\hat{f}[2i+1]$ para el término X, la multiplicación en el anillo T_q consiste en multiplicaciones independientes por cada par de coeficientes entre los dos vectores. A este resultado se le aplica el módulo respecto al polinomio cuadrático correspondiente, asegurando que se mantenga como un polinomio de grado 1.

$$\hat{h}[2i]+\hat{h}[2i+1]X = (\hat{f}[2i]+\hat{f}[2i+1]X)(\hat{g}[2i]+\hat{g}[2i+1]X) \mod (X^2-\zeta^{2\mathrm{BitRev}_7(i)+1})$$

6.3.3. Propiedad de exactitud de un KEM

Este algoritmo cumple con la propiedad esencial de un KEM, la exactitud. Esta propiedad asegura que, en ausencia de corrupción o interferencia, el proceso principal descrito dará como resultado $K' = K$ con una probabilidad casi del cien por ciento.

6.3.4. Probabilidad de fallo de desencapsulamiento

Existe una probabilidad de que, aunque todos los algoritmos se ejecutasen correctamente y no hubiese corrupción o interferencias, existiese un fallo a la hora del desencapsulamiento, lo que resultaría que la clave K producida por el encapsulamiento, y la clave K' producida por el desencapsulamiento, no fuesen iguales. Estas probabilidades, para los tres distintos conjuntos de parámetros, son:

Parameter set	Decapsulation failure rate
ML-KEM-512	$2^{-138.8}$
ML-KEM-768	$2^{-164.8}$
ML-KEM-1024	$2^{-174.8}$

Figura 6.2: Probabilidad de fallo en el desencapsulamiento para ML-KEM[25]

6.4. Los tres conjuntos de parámetros aceptados

ML-KEM tiene tres conjuntos de parámetros distintos para ser ejecutado. Cada uno de estos conjuntos tiene sus propios valores para estos cinco parámetros: k, η_1, η_2, d_u, y d_v. Además, hay dos parámetros globales constantes: $n = 256$ y $q = 3329$.

- El parámetro k determina las dimensiones de la matriz $\hat{A}$, el vector s y el vector e que aparecen en el algoritmo de generación de claves de PKE; así como las dimensiones de la misma matriz $\hat{A}$, el vector e_1 y el vector y del algoritmo de cifrado de PKE.

- El parámetro η_1 especifica la distribución a la hora de generar los vectores s y e en la generación de claves, de PKE y el vector y en el algoritmo de cifrado de PKE.

- El parámetro η_2 especifica la distribución a la hora de generar los vectores e_1 y e_2 en el algoritmo de cifrado de PKE.
- Los parámetros d_u, y d_v sirven como argumentos de entrada en las funciones *Compress*, *Decompress*, *ByteEncode* y *ByteDecode*; utilizadas en los algoritmos de cifrado y descifrado de PKE.

Los tres conjuntos de parámetros posibles son los siguientes:

	n	q	k	η_1	η_2	d_u	d_v	required RBG strength (bits)
ML-KEM-512	256	3329	2	3	2	10	4	128
ML-KEM-768	256	3329	3	2	2	10	4	192
ML-KEM-1024	256	3329	4	2	2	11	5	256

Figura 6.3: Conjuntos de parámetros posibles para ML-KEM[25]

Cada conjunto muestra la longitud en bits que deben tener sus arrays de bits generados aleatoriamente.

A continuación, también se muestran los tamaños (en bytes) de las claves y el texto cifrado de ML-KEM para cada uno de los tres conjuntos de parámetros:

	encapsulation key	decapsulation key	ciphertext	shared secret key
ML-KEM-512	800	1632	768	32
ML-KEM-768	1184	2400	1088	32
ML-KEM-1024	1568	3168	1568	32

Figura 6.4: Longitud (en bytes) de las claves y el texto cifrado en ML-KEM para cada conjunto de parámetros

Cada conjunto de parámetros cumple con una categoría de seguridad definida por el NIST, en total hay 5 categorías de seguridad que van numeradas, de menor a mayor seguridad, del 1 al 5. Estas categorías no se basan en un único número, como 'x número de bits de seguridad', si no que se basan en que son al menos tan seguras como un cifrado de bloque genérico con un tamaño de clave prescrito o una función hash genérica con una longitud de salida prescrita.

Más precisamente, se basan en que los recursos computacionales necesarios para romper un algoritmo de cada categoría son mayores o iguales a los necesarios para romper el cifrado de bloque o la función hash; esos recursos computacionales se estiman utilizando cualquier modelo realista de computación. Diferentes modelos de computación pueden ser más o menos realistas y, por lo tanto, llevar a estimaciones más o menos precisas de la fortaleza de seguridad.

Se afirma que ML-KEM-512 pertenece a la categoría de seguridad 1, ML-KEM-768 a la categoría de seguridad 3, y ML-KEM-1024 a la categoría de seguridad 5.

6.4.1. Elegir el conjunto de parámetros apropiado

Si se trata de cifrar datos, se recomienda utilizar desde el principio el conjunto de parámetros más fuerte en cuanto a seguridad, el ML-KEM-1024. Esto puede traer muchas ventajas, como que si en el futuro hay un nuevo conjunto de parámetros aún más fuerte, la transición a este no será tan costosa. Sin embargo, también puede tener desventajas, como que la ejecución del algoritmo sea demasiado lenta o que objetos como llaves o textos de cifrado tengan una longitud demasiado larga.

El NIST recomienda usar ML-KEM-768 como el conjunto de parámetros predeterminado, ya que proporciona un amplio margen de seguridad a un costo de rendimiento razonable. En casos donde esto no sea práctico o se requiera una seguridad aún mayor, se pueden utilizar otros conjuntos de parámetros.

6.5. Diferencia de ML-KEM con CRYSTALS-Kyber

Como ya se ha dicho, el algoritmo ML-KEM a implementar en este trabajo es una derivación directa del algoritmo estandarizado en 2022 denominado CRYSTALS-Kyber. Por ello, vamos a nombrar, a nivel teórico, las diferencias que hay entre uno y otro.

Como todos estos algoritmos pueden ser implementados de muchas formas distintas, nos vamos a centrar en las variaciones de los inputs y outputs de los tres principales algoritmos de estos modelos: KeyGen, Encaps y Decaps.

- En CRYSTALS-Kyber, la clave secreta compartida K se trataba como un valor de longitud variable cuya longitud dependía de cómo iba a ser utilizada en la aplicación destino. En ML-KEM, la longitud de la clave secreta siempre es de 256 bits, pudiéndose utilizar directamente en aplicaciones como clave simétrica, o derivar a otras claves simétricas de otra longitud a partir de ella.
- Los algoritmos Encaps y Decaps de ML-KEM usan una variante de la transformada de Fujisaki-Okamoto diferente a la que se utiliza en CRYSTAL-Kyber. Específicamente, en este modelo, Encaps ya no incluye un hash del texto cifrado en la derivación de la clave compartida, y Decaps se ha ajustado para coincidir con este cambio.

- En CRYSTALS-Kyber, se le aplicaba un hash al mensaje antes de encapsularlo para protegerlo contra malas generaciones aleatorias. Sin embargo, como ML-KEM obliga a utilizar generaciones aleatorias aceptadas por el NIST, este paso se ha omitido.
- Se han añadido muchas comprobaciones a este modelo ML-KEM que no tenía el CRYSTALS-Kyber, como por ejemplo comprobar la longitud de algunas variables, su tipo de dato, etc.

6.6. Diferencia entre el borrador inicial de ML-KEM y su versión final

Como se ha comentado antes, este trabajo comenzó a desarrollarse sobre el borrador inicial del ML-KEM, publicado el 24 de agosto de 2023, y, tras su reciente publicación de la versión final el 13 de agosto de 2024, se ha tenido que ver modificado para reflejar las nuevas modificaciones añadidas.

La principal diferencia entre el borrador inicial y la versión final es que, en la versión final, se han añadido tres nuevos algoritmos internos.

Estos algoritmos son KeyGenInternal, EncapsInternal y DecapsInternal; y han sido creados para prevenir que las claves utilizadas para comprobar un determinado nivel de seguridad no fueran utilizadas por error en otro nivel de seguridad. Este error se podía cometer debido a que dichas claves se guardan como semillas.

Capítulo 7

Implementación del algoritmo ML-KEM

En este capítulo, se va a abordar todo lo relacionado con la implementación del algoritmo ML-KEM.

Esto es:

- Explicar qué lenguaje de programación se ha utilizado.
- Explicar qué librerías auxiliares se han utilizado.
- Explicar cómo se han estructurado todos los algoritmos para hacer funcionar el algoritmo principal ML-KEM.
- Explicar qué constantes y variables globales tiene el algoritmo.
- Explicar qué métodos privados tiene el algoritmo.
- Explicar qué métodos públicos tiene el algoritmo y cómo utilizarlos correctamente.

Además, tras esto, se explicará el recorrido que sigue una ejecución usual de este algoritmo a través de los distintos algoritmos que lo conforman.

Finalmente, se comentarán problemas y obstáculos encontrados durante la implementación del algoritmo.

7.1. Lenguaje utilizado

Para implementar este algoritmo he utilizado el lenguaje de programación Python.

Esto es debido a que es el lenguaje de programación con el que más familiarizado estoy a día de hoy. Así mismo, pienso que este lenguaje tiene muchas librerías relacionadas con el ámbito de la criptografía y las operaciones tanto matemáticas como algebraicas, lo que puede facilitar la implementación de este algoritmo.

Sin embargo, un punto a tener en cuenta, que en ocasiones puede ser beneficioso pero en otras ocasiones puede ser muy peligroso, es la conversión automática de tipos en Python, ya que podemos querer (y creer) estar trabajando con un determinado tipo de datos y que finalmente resulte que estamos manejando otro.

Es por esto que debemos ser muy cautelosos con los tipos de datos y llevar una buena trazabilidad de los inputs, outputs e incluso de las operaciones que se realizan dentro de cada algoritmo.

7.2. Librerías auxiliares utilizadas

Las librerías auxiliares que he utilizado para implementar el algoritmo son dos: *numpy* y *hashlib*.

```
import numpy as np
import hashlib
```

Figura 7.1: Import de librerías: numpy y hashlib

7.2.1. NumPy

La librería *numpy* es una de las mayores librerías para la computación científica en Python. Su principal componente es el objeto 'ndarray' (numpy array), que es una estructura de datos eficiente que permite almacenar y manipular grandes matrices y arrays multidimensionales de una manera rápida y eficiente.[1]

Además, a los objetos ndarray se les puede establecer un tipo de dato concreto, por lo que nos viene genial para solventar el problema comentado anteriormente de conversión automática de datos en Python. Los datos que utilizo son np.dtype = np.uint8 y np.dtype = np.int64.

Estos datos nos permiten asegurar que los datos tengan exactamente el tamaño de un byte o de 8 bytes, respectivamente. Esto es muy útil ya que las claves y la mayoría de elementos son arrays de bytes, mientras que las operaciones con coeficientes, al estar en el módulo 3329, las realizaremos con tamaños de 8 bytes para evitar desbordamientos.

Tanto por la razón del control de los tipos de las variables, como por todo lo que aporta la propia librería en sí a nivel de eficiencia de operaciones con vectores y matrices grandes, *numpy* es una librería muy útil para la implementación de este algoritmo.

7.2.2. Hashlib

La librería *hashlib* es una librería de Python que proporciona una interfaz para crear funciones hash seguras y eficientes. Esta librería soporta varios algoritmos de hash, como SHAKE128 y SHA-256, entre muchos otros.[12]

Como la implementación de este algoritmo requiere de muchos cifrados hash para su correcto funcionamiento, esta librería nos va a ahorrar mucho tiempo a la hora de utilizar estas funciones hash, ya que todas ellas ya vienen creadas.

7.3. Clase ML-KEM

Para crear este algoritmo, he decidido crear una clase llamada ML-KEM.

Dicha clase encapsula todas las constantes y variables del conjunto de parámetros, todos los métodos privados necesarios para el correcto funcionamiento de los tres principales métodos públicos (KeyGen, Encaps y Decaps), así como algunos métodos públicos más que he visto necesarios y/o apropiados para el algoritmo ML-KEM.

Ahora, voy a proceder a explicar cada una de sus partes.

7.3.1. Constantes globales

Esta clase tiene dos constantes globales, que son n y q.

- n es el entero 256 y se utiliza para diversas cosas a lo largo del documento, como definir cuántos elementos tiene cada posición de un array.
- q es el entero primo $3329 = 2^8 \cdot 13 + 1$ y se utiliza para definir el módulo sobre el que vamos a trabajar.

Así mismo, también tiene dos arrays constantes globales denominados *_ArrayBitRev* y *_ArrayBitRev_Multiply*. Estos arrays son numpy arrays de longitud 128 y almacenan datos del tipo *np.int64*.

```
# CONSTANTES GLOBALES
n = 256
q = 3329
```

Figura 7.2: Constantes globales de la clase ML-KEM

```
# Array necesario para calcular tanto NTT como NTT^-1
_ArrayBitRev = np.array([
    1, 1729, 2580, 3289, 2642, 630, 1897, 848, 1062, 1919, 193, 797, 2786, 3260, 569,
    1746, 296, 2447, 1339, 1476, 3046, 56, 2240, 1333, 1426, 2094, 535, 2882, 2393, 2879,
    1974, 821, 289, 331, 3253, 1756, 1197, 2304, 2277, 2055, 650, 1977, 2513, 632, 2865,
    33, 1320, 1915, 2319, 1435, 807, 452, 1438, 2868, 1534, 2402, 2647, 2617, 1481, 648,
    2474, 3110, 1227, 910, 17, 2761, 583, 2649, 1637, 723, 2288, 1100, 1409, 2662, 3281,
    233, 756, 2156, 3015, 3050, 1703, 1651, 2789, 1789, 1847, 952, 1461, 2687, 939, 2308,
    2437, 2388, 733, 2337, 268, 641, 1584, 2298, 2037, 3220, 375, 2549, 2090, 1645, 1063,
    319, 2773, 757, 2099, 561, 2466, 2594, 2804, 1092, 403, 1026, 1143, 2150, 2775, 886,
    1722, 1212, 1874, 1029, 2110, 2935, 885, 2154
], dtype=np.int64)

# Array necesario para calcular la multiplicación entre dos representaciones NTT
_ArrayBitRev_Multiply = np.array([
    17, 3312, 2761, 568, 583, 2746, 2649, 680, 1637, 1692, 723, 2606, 2288, 1041, 1100,
    2229, 1409, 1920, 2662, 667, 3281, 48, 233, 3096, 756, 2573, 2156, 1173, 3015, 314,
    3050, 279, 1703, 1626, 1651, 1678, 2789, 540, 1789, 1540, 1847, 1482, 952, 2377, 1461,
    1868, 2687, 642, 939, 2390, 2308, 1021, 2437, 892, 2388, 941, 733, 2596, 2337, 992,
    268, 3061, 641, 2688, 1584, 1745, 2298, 1031, 2037, 1292, 3220, 109, 375, 2954, 2549,
    780, 2090, 1239, 1645, 1684, 1063, 2266, 319, 3010, 2773, 556, 757, 2572, 2099, 1230,
    561, 2768, 2466, 863, 2594, 735, 2804, 525, 1092, 2237, 403, 2926, 1026, 2303, 1143,
    2186, 2150, 1179, 2775, 554, 886, 2443, 1722, 1607, 1212, 2117, 1874, 1455, 1029, 2300,
    2110, 1219, 2935, 394, 885, 2444, 2154, 1175
], dtype=np.int64)
```

Figura 7.3: _ArrayBitRev y _ArrayBitRev_Multiply

_ArrayBitRev

Como hemos visto a la hora de explicar la representación NTT, dicha representación necesita los parámetros:

$$\zeta^{BitRev_7(i)} \mod q$$

para $i \in \{0, 1, 2, \ldots, 127\}$

Estos parámetros son necesarios para calcular tanto la representación NTT a partir de un polinomio en R_q como para realizar la inversa NTT a un polinomio en T_q.

El estándar recomienda tener este array pre-calculado para que, a la hora de utilizar sus valores, no se tengan que estar constantemente generando, sino que directamente se extraigan de un array.

La forma para calcular este array ha sido la siguiente:

Si no quisiésemos calcular los valores de dicho array, el propio estándar cuenta con un apéndice (Apéndice A) en el que aparecen los datos de dicho array ya calculados.

```
ArrayBitRev = []
for i in range(128):
    ArrayBitRev.append(pow(17, BitRev7(i)) % q)
```

Figura 7.4: Forma utilizada para calcular _ArrayBitRev

_ArrayBitRev_Multiply

Como hemos visto a la hora de explicar la multiplicación de dos representaciones NTT, para calcular dicha multiplicación se necesita de los parámetros:

$$\zeta^{2 \cdot BitRev_7(i)+1} \mod q$$

para $i \in \{0, 1, 2, \ldots, 127\}$

El estándar recomienda tener este array pre-calculado para que, a la hora de utilizar sus valores, no se tengan que estar constantemente generando, sino que directamente se extraigan de un array.

La forma para calcular este array ha sido la siguiente:

```
ArrayBitRev_Multiply = []
for i in range(128):
    ArrayBitRev_Multiply.append(pow(17, 2*BitRev7(i) + 1) % q)
```

Figura 7.5: Forma utilizada para calcular _ArrayBitRev_Multiply

Si no quisiésemos calcular los valores de dicho array, el propio estándar cuenta con un apéndice (Apéndice A) en el que aparecen los datos de dicho array ya calculados.

7.3.2. Variables globales

Las variables globales de la clase son los ya mencionados parámetros de los tres distintos conjuntos de parámetros aceptados por este algoritmo. Los parámetros para el ML-KEM-512, para el ML-KEM-768 y para el ML-KEM-1024.

Recordando qué hacían estos parámetros:

- El parámetro k determina las dimensiones de la matriz $\hat{A}$, el vector s y el vector e que aparecen en el algoritmo de generación de claves de PKE; así como las dimensiones de la misma matriz $\hat{A}$, el vector e_1 y el vector y del algoritmo de cifrado de PKE.
- El parámetro η_1 especifica la distribución a la hora de generar los

vectores s y e en la generación de claves, de PKE y el vector y en el algoritmo de cifrado de PKE.

- El parámetro η_2 especifica la distribución a la hora de generar los vectores e_1 y e_2 en el algoritmo de cifrado de PKE.
- Los parámetros d_u, y d_v sirven como argumentos de entrada en las funciones *Compress*, *Decompress*, *ByteEncode* y *ByteDecode*; utilizadas en los algoritmos de cifrado y descifrado de PKE.

Por defecto, vienen establecidos los parámetros para la ejecución de ML-KEM-768, tal y como recomienda el NIST: $k = 3$, $\eta_1 = 2$, $\eta_2 = 2$, $d_u = 10$ y $d_v = 4$.

Estos parámetros se pueden modificar para alternar el modelo ML-KEM mediante una función denominada *_establecer_parametros*(x), cuyo funcionamiento explicaremos más adelante.

```
# VARIABLES GLOBALES: Por defecto ML-KEM-768
k = 3
eta_1 = 2
eta_2 = 2
d_u = 10
d_v = 4
```

Figura 7.6: Variables establecidas por defecto para ML-KEM-768

Además, hay dos variables globales más, que son la clave de encapsulamiento *encaps_k* y la clave de desencapsulamiento *decaps_k*. Estas variables sirven por si el usuario, en lugar de generar las claves de forma interna, ya dispone de un conjunto de claves y quiere realizar el encapsulamiento o desencapsulamiento con ellas.

Estas claves se inicializan como numpy arrays de tipo np.uint8 (un byte) rellenos de ceros y con longitudes 384*k+32 y 768*k+96 respectivamente. Ya comenzamos a apreciar la gran relevancia de las variables de los conjuntos de parámetros aceptados, ya que la longitud de dichas claves varía en función de k.

```
# CLAVES DE ENCAPSULAMIENTO Y DESENCAPSULAMIENTO DE LA EJECUCIÓN
encaps_k = np.zeros(384*k+32, dtype=np.uint8)
decaps_k = np.zeros(768*k+96, dtype=np.uint8)
```

Figura 7.7: Variables globales: encaps_k y decaps_k

Todas las demás variables que se manejen a lo largo de todos los algoritmos serán variables locales, es decir, no se utilizará en ningún momento el paso por referencia.

7.3.3. Métodos privados

Todos los nombres de los métodos privados comienzan por '_' debido a que esto es lo que se utiliza en Python para representar que son métodos privados.

_PRF(eta, s, b)

Este método calcula el hash *SHAKE256* de un array de 33 bytes y genera una salida pseudorandom[1] cuya longitud será de $64 * \eta$ bytes.

Aquí podemos ver una vez más la importancia del conjunto de parámetros definido inicialmente, y así será a lo largo de la mayoría de los algoritmos de ML-KEM (no se hará alusión más veces a dicho suceso).

Siempre que hablemos de inputs u outputs de tipo byte, estaremos hablando de un *numpy array* con *dtype=np.uint8*.

Para realizar el hash SHAKE256 nos apoyamos en la librería *hashlib*. Gracias a esta librería y su función *shake_256*, podemos introducirle los datos de entrada (el array de 33 bytes) y producir una salida que tenga la longitud deseada gracias a otro método de la librería hashlib denominado *digest()*.

El método *digest()* producirá una salida cuya longitud será siempre el número de bytes que produzca la función hash utilizada, a menos que le especifiquemos otra longitud concreta.

En nuestro caso, como la longitud de salida es variable (depende de η), creamos una variable con dicha longitud y la utilizamos en el método *digest(longitud_variable_necesaria)*.

Finalmente, como las funciones de la librería *hashlib* producen datos de tipo byte en dicha representación, debemos convertirlos de nuevo a números decimales de tipo *np.uint8*. Esto lo conseguimos gracias al método *np.frombuffer(datos, dtype al que convertir)* de la librería *numpy*. Gracias a este método, podemos mantener una consistencia en los datos utilizados en todo momento.

[1]Pseudorandom: Un output que a vistas de quien no conoce el cálculo interno parece aleatorio, pero que, realmente, es determinístico; esto quiere decir que a una mismo input, siempre generará el mismo output.

```python
@staticmethod
def _PRF(eta, s, b):
    # Concatenamos los datos de entrada para obtener un solo array de 33 bytes (32+1)
    datos_entrada = np.concatenate((s, b))

    # Calculamos la longitud deseada de salida gracias al parámetro eta
    longitud_salida = 64 * eta

    # Calculamos el hash SHAKE256 y generamos la salida con la longitud deseada
    shake = hashlib.shake_256(datos_entrada).digest(longitud_salida)

    #Convertimos el resultado a np.uint8
    shaked_bytes = np.frombuffer(shake, dtype=np.uint8)

    return shaked_bytes
```

Figura 7.8: Método privado: _PRF(eta, s, b)

_H(s)

Calcula el hash *SHA3-256* a un array de bytes de longitud variable e imprime un array de 32 bytes de longitud.

De nuevo, utilizamos un método de la librería *hashlib*, en este caso el *sha3_256*. Como este método produce 256 bits (32 bytes), y es justo la longitud que deseamos obtener, no tenemos que pasarle ningún parámetro a la función *digest()*, ya que automáticamente extraerá todos los bytes (32).

Devolvemos el valor calculado convertido a tipo *np.uint8* como hemos visto anteriormente.

```python
@staticmethod
def _H(s):
    # Calculamos el hash SHA3-256 y generamos la salida de 32 bytes
    hash_output = hashlib.sha3_256(s).digest()

    return np.frombuffer(hash_output, dtype=np.uint8)
```

Figura 7.9: Método privado: _H(s)

_J(s)

Calcula el hash *SHAKE256* a un array de bytes de longitud variable e imprime un array de 32 bytes de longitud.

De nuevo, utilizamos un método de la librería *hashlib*, en este caso el *shake_256*. A este método sí que debemos especificarle que queremos una longitud de 32 bytes ya que es un método XOF(Extendable-Output Function), por lo que lo indicamos en el método *digest(32)*.

Devolvemos el valor calculado convertido a tipo *np.uint8*.

```python
@staticmethod
def _J(s):
    # Calculamos el hash SHAKE256 y generamos la salida de 32 bytes
    hash_output = hashlib.shake_256(s).digest(32)

    return np.frombuffer(hash_output, dtype=np.uint8)
```

Figura 7.10: Método privado: _J(s)

_G(c)

Calcula el hash SHA3-512 a una entrada de bytes de longitud variable y produce como salida dos arrays de 32 bytes cada uno.

De nuevo, utilizamos un método de la librería *hashlib*, en este caso el *sha_512*. Al generar 512bits, genera 64bytes. Para obtener dos arrays de 32 bytes cada uno, se nos indica que debemos coger para el primer elemento los primeros 32 bytes de *sha_512* y, para el segundo, los últimos 32 bytes.

Devolvemos ambos valores convertidos a tipo *np.uint8*.

```python
@staticmethod
def _G(c):
    # Calculamos el hash SHA3-512
    hash_output = hashlib.sha3_512(c).digest()

    # Generamos las dos salidas de 32 bytes
    a = np.frombuffer(hash_output[:32], dtype=np.uint8)
    b = np.frombuffer(hash_output[32:], dtype=np.uint8)

    return a, b
```

Figura 7.11: Método privado: _G(c)

_BitRev7(i)

Esta función toma un entero *i* en el rango *0, ..., 127*, invierte el orden de sus bits y devuelve el entero resultante de dicha inversión.

Para ello, pasamos el número a representación de 7 bits, revertimos el orden de su cadena de bits y lo volvemos a convertir a un número entero gracias a tres funciones básicas introducidas por defecto en Python.

Como ya hemos visto, esta función es esencial para calcular los arrays *_ArrayBitReverse* y *_ArrayBitReverse_Multilply* utilizados para las representaciones NTT y la mutiplicación de dichas representaciones, respectivamente.

```python
def _BitRev7(i):
    # Verificamos que el valor a invertir esté en el rango 0-127 que es lo que se puede represntar con 7 bits
    if not (0 <= i < 128):
        raise ValueError("El valor de entrada debe estar entre 0 y 127 (7 bits)")

    # Pasamos de un entero a representación de 7 bits
    bit_string = f'{i:07b}'

    #Invertimos el orden del string de bits
    bits_invertidos = bit_string[::-1]

    # Convertimos el string de bits a un entero
    bits_invertidos = int(bits_invertidos, 2)

    return bits_invertidos
```

Figura 7.12: Método privado: _BitRev(i)

_BitsToBytes(b)

Convierte un array de bits (longitud 8*x) introducido como argumento en un array de bytes (longitud x). El array de bits también es un *numpy array* de tipo *np.uint8*.

Para ello, creamos un array bytes de longitud 8//x[2] y le añadimos en cada posición el valor correspondiente de sumar los valores de sus 8 bits correspondientes. Para ello, recorremos todos los bits, y, como 8 bits son 1 byte, nos mantenemos en B[i//8] durante 8 iteraciones.

En cada iteración, el valor del byte se suma o no dependiendo de si es 0 o 1, ya que va multiplicado por ese valor. En caso de ser 1, al valor actual del byte (inicializalizado en 0), se le suma el valor $2^{(i \bmod 8)}$.

Finalmente devolvemos el array de bytes calculado.

```python
@staticmethod
def _BitsToBytes(b):
    # Calculamos la longitud que tendrá el array de bytes
    x = len(b) // 8

    # Inicializamos el array de bytes de longitud n con ceros
    B = np.zeros(x, dtype=np.uint8)

    # Calculamos el valor de cada byte
    for i in range(len(b)):
        B[i//8] = B[i//8] + b[i] * pow(2, i % 8) # Si b[i] es 1, suma el valor de esa posición del bit

    return B
```

Figura 7.13: Método privado: _BitsToBytes(b)

_BytesToBits(B)

Convierte un array de bytes (longitud x) introducido como argumento en un array de bits (longitud 8*x). El array de bits también es un *numpy array* de tipo *np.uint8*.

Para ello, creamos un array bits de longitud 8*x y, para acceder dentro de un determinado byte a la posición de cada uno de los 8 bits, avanzamos

[2] // para redondear automáticamente al entero más bajo

con dos bucles distintos: uno para determinar el byte actual y otro para determinar el bit actual.

El bit se pone a 0 o a 1 en función del valor del byte módulo 2 (solo puede generar 0 o 1, en función de si es par o impar). Conseguimos que esto funcione gracias a que en cada iteración dividimos (redondeando) entre 2, lo que hace que vaya navegando entre las distintas posiciones de bits: 2^8, 2^7...

Finalmente devolvemos el array de bits calculado.

```python
@staticmethod
def _BytesToBits(B):
    # Extraemos la longitud del array de bytes y creamos el array de bits con esa longitud*8
    n = len(B)
    b = np.zeros(n * 8, dtype=np.uint8)

    # Copiamos el array para su modificación
    C = B.copy()

    # Ponemos el bit a 0 o 1 dependiendo del %2 del valor del byte en ese momento y nos desplazamos dividiendolo entre 2
    for i in range(n):
        for j in range(8):
            b[8*i + j] = C[i] % 2
            C[i] = C[i]//2
    return b
```

Figura 7.14: Método privado: _BytesToBits(B)

_Compress(x, d)

Comprime un valor de $\mathbb{Z}_q$ a un valor en $\mathbb{Z}_{2^d}$, en función de un valor de compresión d que se encuentra en el rango de valores $1 \leq d < 12$.

Como para representar q necesitamos 12 bits, podemos comprimir valores a representaciones en un número menor de bits. Para ello, debemos utilizar la siguiente fórmula:

$$x = \left\lceil \frac{2^d}{q} \cdot x \right\rfloor \text{ mód } 2^d$$

El valor x que entra, y el nuevo valor x que sale, son de tipo *np.int64*, ya que en un tipo *np.uint8* no nos caben enteros superiores al valor 255.

```python
@staticmethod
def _Compress(x, d):
    return round(((2**d) / ML_KEM.q) * x) % (2**d)
```

Figura 7.15: Método privado: _Compress(x, d)

_Decompress(y, d)

Descomprime un valor de $\mathbb{Z}_{2^d}$ a un valor en $\mathbb{Z}_q$, en función de un valor de compresión d que se encuentra en el rango de valores $1 \leq d < 12$.

No podemos utilizar d = 12 ya que eso permitiría valores hasta 4096 (superiores a 3329). Para realizar la descompresión, debemos utilizar la siguiente fórmula:

$$y = \left\lceil \left(\frac{q}{2^d} \right) \cdot y \right\rfloor$$

Los algoritmos de Compresión y Descompresión satisfacen dos propiedades importantes.

- La descompresión seguida de la compresión preserva el input inicial. Es decir, $\text{Compress}_d(\text{Decompress}_d(y)) = y$ para todo $y \in \mathbb{Z}_{2^d}$ y todo $d < 12$.
- Si d es grande (es decir, cercano a 12), la compresión seguida de la descompresión no altera significativamente el valor.

```
@staticmethod
def _Decompress(y, d):
    return round((ML_KEM.q / (2**d)) * y)
```

Figura 7.16: Método privado: _Decompress(x, d)

_ByteEncode(F, d)

Convierte un array de 256 enteros módulo m (*np.uint64*) a un array de bytes (*np.uint8*) que utiliza segmentos de d bits para su representación. Como d define los bits, trabajaremos con ellos y, solo al final, utilizaremos el método *_BitsToBytes* explicado anteriormente para generar el array de bytes que queremos como output.

Los parámetros aceptados para d son $1 \leq d \leq 12$.

Primero, crea un array de bits de longitud n (256 enteros del array de coeficientes) * d (bits utilizados para codificar cada entero) para almacenar todos los bits que se van a generar.

Tras esto, va operando con cada entero del array pasado como argumento convirtiéndolo a d bits representativos y almacenándolos en el array de bits creado.

Una vez se tiene el array de bits completo, se convierte a un array de longitud 32*d bytes de tipo *np.uint8*, ya que 256 enteros * d bits = 32 bytes * d.

```python
@staticmethod
def _ByteEncode(F, d):
    # Creamos un array con el número de bits que vamos a necesitar
    b = np.zeros(ML_KEM.n * d, dtype=np.uint8)

    # Calculamos su representación en bits parecido a BitsToBytes pero en lugar de con 8, con d
    for i in range(ML_KEM.n):
        a = F[i]
        for j in range(d):
            b[i*d + j] = a % 2
            a = (a - b[i*d + j]) // 2

    # Finalmente convertimos el array de bits obtenido a bytes (de 8 en 8)
    B = np.zeros((ML_KEM.n * d) // 8, dtype=np.uint8)
    B = ML_KEM._BitsToBytes(b)

    return B
```

Figura 7.17: Método privado: _ByteEncode(F, d)

_ByteDecode(B,d)

Convierte un array de 32*d bytes (*np.uint8*) en un array de 256 enteros módulo m (*np.int64*). Para ello debemos operar directamente con los bits, por lo que, al principio, pasaremos el array de bytes a bits.

Los parámetros aceptados para d son $1 \leq d \leq 12$.

Primero, convierte el array de bytes introducido por parámetro a un array de bits para poder trabajar de d en d bits.

Segundo, calcula el módulo con el que trabajar, si $d \leq 11$, $m = d$; sin embargo, si $d = 12$, se debe truncar $m = q$ para no generar valores fuera del rango con el que estamos trabajando.

Finalmente, de manera parecida a cuando transformábamos bits a bytes, vamos viendo para cada entero F[i], si el valor de sus d bits es 0 o 1, añadiéndole dicho valor a su valor total o no. Este array F que se devuelve como output es de tipo *np.int64*.

```python
@staticmethod
def _ByteDecode(B, d):
    # Primero convertimos el array a bits para poder trabajar de d en d bits
    b = ML_KEM._BytesToBits(B)

    # Si d es <12, podemos decodificar hasta 2^d, si no, debemos truncar en q
    m = 2**d if d < 12 else ML_KEM.q

    # Regeneramos el valor de F[i] igual que haciamos para regenerar el valor de los bytes en BitsToBytes
    F = np.zeros(ML_KEM.n, dtype=np.int64)
    for i in range(ML_KEM.n):
        for j in range(d):
            F[i] = (F[i] + b[i*d + j] * pow(2, j)) % m

    return F
```

Figura 7.18: Método privado: _ByteDecode(B, d)

_SampleNTT(B)

Convierte un array de 34 bytes (una semilla de 32 bytes + 2 índices de 1 byte cada uno) en un polinomio del dominio NTT. Si la semilla es uniformemente aleatoria, el resultado será un polinomio pseudoaleatorio que tendrá una distribución computacionalmente indistinguible de la distribución T_q.

El output de esta función es un array de 256 coeficientes del elemento muestreado T_q.

Para ello, lo que hace este algoritmo es primero crear un hash de tipo *SHAKE128* con el array de bytes de longitud 34 pasado como argumento. Esto se utiliza para crear una secuencia más grande de bytes pseudoaleatorios a partir de una semilla inicial de menor tamaño.

Ahora, generamos 840 bytes pseudoaleatorios a partir del método digest para luego operar con ellos dentro del bucle while para el cálculo de coeficientes del array $\hat{a}$. El tener que crear los 840 inicialmente y luego ir extrayendo de 3 en 3 es debido al funcionamiento del método digest, ya que si realizásemos digest(3) en cada iteración del bucle while, siempre nos generaría los mismos 3 bytes. El número 840 se debe a un límite de 280 iteraciones máximas que se explicará más adelante.

Con los 840 bytes pseudoaleatorios generados, entra en un bucle while del que solo saldrá cuando *j*, inicializada a 0, valga 256. Dentro de este bucle while, se extraen de 3 en 3 los bytes creados anteriormente y, con ellos, se calculan los coeficientes *d1* y *d2*.

Si alguno de estos dos valores (coeficientes) es menor que q, se añade en la posición j del array $\hat{a}$ y se incrementa j en 1. Este proceso de repite hasta que j valga 256, que significa que $\hat{a}$ está completo.

El número de iteraciones máximo comentado anteriormente es una consideración opcional recogida en el estándar. Esta opción de limitar las iteraciones se puede utilizar por si acaso se tardase, normalmente, demasiado en generar los 256 valores, es decir, que constantemente d_1 y d_2 fuesen mayores que q.

El estándar no recomienda establecer dicho límite pero, en caso de ser necesario, el valor límite que recomiendan es 280, ya que la probabilidad de necesitar más de 280 iteraciones para rellenar todo $\hat{a}$ es ínfima.

Finalmente, se devuelve el array $\hat{a}$ que contiene 256 coeficientes del elemento muestreado T_q de tipo *np.int64*.

```python
@staticmethod
def _SampleNTT(B):
    # Calcula el hash shake_128 con el array de bytes B
    ctx = hashlib.shake_128()
    ctx.update(B.tobytes())

    # Generamos el máximo de posibles bytes necesarios, 280 iteraciones máxima * 3 bytes utilizados por iteración
    C_total = ctx.digest(840)

    # Rellena a_hat con los valores d1 y d2 calculados a partir de los valores aleatorios extraidos en C
    a_hat = np.zeros(256, dtype=np.int64)
    j = 0
    i = 0
    while j < 256:
        # Extraemos 3 bytes pseudoaleatorios
        C = C_total[i:i+3]

        # Calculamos los coeficientes d1 y d2
        d1 = C[0] + 256 * (C[1] % 16)
        d2 = (C[1] // 16) + 16 * C[2]

        if d1 < ML_KEM.q:
            a_hat[j] = np.int64(d1)
            j += 1

        if d2 < ML_KEM.q and j < 256:
            a_hat[j] = np.int64(d2)
            j += 1

        # Avanzamos a los siguientes 3 bytes pseudoaleatorios
        i = i+3

    return a_hat
```

Figura 7.19: Método privado: _SampleNTT(B)

_SamplePolyCBD(B, eta)

ML-KEM hace uso de una distribución especial $D_\eta(R_q)$ de polinomios en R_q con coeficientes pequeños. Estos polinomios son los que denominaremos "errores" o "ruido". La distribución está parametrizada por un entero $\eta \in \{2, 3\}$. Para muestrear un polinomio de $D_\eta(R_q)$, cada uno de sus coeficientes se muestrea de forma independiente a partir de una cierta distribución binomial centrada (CBD) en $\mathbb{Z}_q$.

Este algoritmo muestrea el array de coeficientes de un polinomio $f \in R_q$ de acuerdo con la distribución $D_\eta(R_q)$, siempre que su entrada sea un flujo de bytes aleatorios uniformemente distribuidos.

Para ello, recibe un array de bytes de longitud 64*η y lo convierte en bits. Tras esto, va calculando el valor tanto de x como de y, que se calcula sumando η veces el valor de las posiciones de sus bits (0 o 1), por lo que la suma máxima es η en caso de que todos los bits estén a 1.

Finalmente, se realiza la diferencia x-y para obtener el valor objetivo. Como x e y tienen valores entre 0 y η, el resultado de la diferencia estará en el rango entre $(0 - \eta, 0 + \eta$. Como trabajamos en el módulo q, $0 - \eta$ es equivalente a $q - \eta$.

Finalmente, devolvemos f, que es un array de 256 coeficientes entre $q - \eta$ y $0 + \eta$ de tipo *np.int64*.

```python
@staticmethod
def _SamplePolyCBD(B, eta):
    # Transformamos el array de bytes a bits
    b = ML_KEM._BytesToBits(B)
    f = np.zeros(256, dtype=np.int64)

    # f[i] tiene el valor (x-y)%q y x e y solo pueden alcanzar 'eta' valor cada vez
    for i in range(256):
        x = 0
        y = 0
        for j in range(eta):
            x += b[2 * eta * i + j]
            y += b[2 * eta * i + eta + j]
        f[i] = (x - y) % ML_KEM.q

    return f
```

Figura 7.20: Método privado: _SamplePolyCBD(B, eta)

_NTT(f)

Como hemos explicado anteriormente, en nuestro algoritmo ML-KEM, utilizamos latransformada numérica (NTT) de un polinomio $f \in R_q$, que da como resultado $\hat{f} \in T_q$, para realizar operaciones con polinomios de manera más rápida y eficiente.

El algoritmo _NTT(f) organiza el array de entrada f, que contiene los 256 coeficientes de f en R_q, en segmentos de tamaño inicialmente 128 y reduciéndolos a la mitad en cada iteración: primero es 128, luego 64, luego 32, y así sucesivamente hasta llegar a 2.

En cada iteración, se selecciona un valor ζ del array *_ArrayBitRev* que, como ya sabemos, es una raíz primitiva precomputada. Este valor ζ se utiliza en un bucle anidado para combinar y transformar los coeficientes del array $\hat{f}$ mediante operaciones de suma y resta, siguiendo la estructura del anillo. Esta estructura permite realizar multiplicaciones y sumas de manera eficiente, tomando siempre los resultados módulo q.

El array final $\hat{f}$, producido tras la ejecución del algoritmo, es de tipo *np.int64* con longitud 256 (igual que el array de entrada), y contiene la representación del polinomio f en el dominio NTT.

```python
@staticmethod
def _NTT(f):
    # Copiamos el contenido del array para crear f_hat
    f_hat = f.copy()

    # Organizamos el array en segmentos de longitud len_ y procesamos los valores de las posiciones por pares
    i = 1
    len_ = 128
    while len_ >= 2:
        for start in range(0, 256, 2 * len_):
            zeta = ML_KEM._ArrayBitRev[i]
            i += 1
            for j in range(start, start + len_):
                t = (zeta * f_hat[j + len_]) % ML_KEM.q
                f_hat[j + len_] = (f_hat[j] - t) % ML_KEM.q
                f_hat[j] = (f_hat[j] + t) % ML_KEM.q
        len_ = len_ // 2

    return f_hat
```

Figura 7.21: Método privado: _NTT(f)

_InverseNTT(f_hat)

Este algoritmo, como su nombre indica, realiza la operación inversa a NTT. A partir de un array $\hat{f}$ que contiene sus 256 coeficientes en la representación NTT (T_q), transforma dicho array de vuelta al anillo R_q, con sus respectivos 256 coeficientes originales.

Para conseguir esto, se realiza un proceso inverso al aplicado en el método *_NTT*. Aquí, en lugar de comenzar con *len_ = 128* e ir dividiendo entre 2, comenzamos con *len_ = 2* y vamos duplicando su valor en cada iteración.

Además, existe un paso extra donde se multiplica cada coeficiente del array por 3303 mód q. Esto se debe a que 3303 es el inverso multiplicativo de 128 módulo 3329 ($128 \cdot 3303 \equiv 1 \mod 3329$), lo que permite recuperar los valores correctos de los coeficientes originales en f.

El array f producido es de tipo *np.int64* y tiene longitud 256 (igual que el array de entrada).

```python
@staticmethod
def _InverseNTT(f_hat):
    # Copiamos el contenido del array para crear f_hat
    f = f_hat.copy()

    # Organizamos el array en segmentos de longitud len_ y procesamos los valores de las posiciones, aplicando las operaciones inversas a la NTT
    i = 127
    len_ = 2
    while len_ <= 128:
        for start in range(0, 256, 2 * len_):
            zeta = ML_KEM._ArrayBitRev[i]
            i -= 1
            for j in range(start, start + len_):
                t = f[j]
                f[j] = (t + f[j + len_]) % ML_KEM.q
                f[j + len_] = (zeta * (f[j + len_] - t)) % ML_KEM.q
        len_ = len_ * 2
    for i in range(256):
        f[i] = (f[i] * 3303) % ML_KEM.q

    return f
```

Figura 7.22: Método privado: _InverseNTT(F)

_MultiplyNTTs(f_hat, g_hat)

Este algoritmo calcula la multiplicación de dos representaciones NTT $\hat{f}$ y $\hat{g}$. Para ello, calcula las posiciones de $\hat{h}$ (resultado de la multiplicación) de dos en dos hasta $i = 128$, utilizando una función auxiliar denominada *BaseCaseMultiply(a0, a1, b0, b1, gamma)*.

Aquí, $a0$ y $a1$ son los coeficientes $\hat{f}[2 * i]$ y $\hat{f}[2 * i + 1]$, mientras que $b0$ y $b1$ son los coeficientes $\hat{g}[2 * i]$ y $\hat{g}[2 * i + 1]$. La variable γ corresponde a los valores precomputados del array _ArrayBitRev_Multiply.

La función *BaseCaseMultiply(a0, a1, b0, b1, gamma)* devuelve los coeficientes resultado de la multiplicación y estos se asignan a $\hat{h}[2*i]$ y $\hat{h}[2*i+1]$, respectivamente. El funcionamiento interno de esta función se basa en el método _BaseCaseMultiply(a0, a1, b0, b1, gamma) explicado en el siguiente apartado.

El array $\hat{h}$ devuelto es de tipo *np.int64* con longitud 256.

```python
def _MultiplyNTTs(f_hat, g_hat):
    # Creamos el array en el que se va a almacenar la multiplicación
    h_hat = np.zeros(256, dtype=np.int64)

    # Para cada 2 posiciones, extraemos los valores de ambos arrays, así como el valor gamma
    # Tras esto, obtenemos los valores de la multiplicación gracias a la función BaseCaseMultiply
    for i in range(128):
        f0, f1 = f_hat[2*i], f_hat[2*i + 1]
        g0, g1 = g_hat[2*i], g_hat[2*i + 1]
        gamma = ML_KEM._ArrayBitRev_Multiply[i]
        h0, h1 = ML_KEM._BaseCaseMultiply(f0, f1, g0, g1, gamma)
        h_hat[2*i], h_hat[2*i + 1] = h0, h1

    return h_hat
```

Figura 7.23: Método privado: _MultiplyNTTs(t_hat, g_hat)

_BaseCaseMultiply(a0, a1, b0, b1, gamma)

Este algoritmo es la base de la multiplicación de representaciones NTT, y lo que hace es calcular el producto de dos polinomios de grado uno con respecto a un módulo cuadrático.

El primer valor, c_0, se calcula como $a_0 \cdot b_0 + a_1 \cdot b_1 \cdot \gamma$. Aquí, el término $a_1 \cdot b_1$ se multiplica por γ porque produciría X^2, y como estamos trabajando con polinomios de grado máximo X, es necesario aplicar la reducción $X^2 = \gamma$ para mantener la consistencia con el módulo cuadrático.

El segundo valor, c_1, se calcula como $a_0 \cdot b_1 + a_1 \cdot b_0$, y no necesita multiplicarse por γ ya que no genera términos de grado X^2.

Ambos valores c_0 y c_1 son tomados módulo q, asegurando que sus valores estén dentro del rango permitido por $\mathbb{Z}_q$.

Esta función devuelve los valores c_0 y c_1, que son de tipo *np.int64*.

```
def _BaseCaseMultiply(a0, a1, b0, b1, gamma):
    c0 = (a0 * b0 + a1 * b1 * gamma) % ML_KEM.q
    c1 = (a0 * b1 + a1 * b0) % ML_KEM.q

    return c0, c1
```

Figura 7.24: Método privado: _BaseCaseMultiply(a0, a1, b0, b1, gamma)

_K_PKE_KeyGen(d)

Este método genera las claves de cifrado y descifrado, *ekPKE* y *dkPKE* respectivamente, a partir de un única semilla (un array de 32 bytes generado aleatoriamente) inicial introducida como parámetro a la función. Los prefijos *ek* y *dk* hacen referencia a *Encryption Key* y *Decryption Key*. Este algoritmo no realiza comprobaciones sobre el input que recibe ya que, al ser una subrutina del método *ML-KEM KeyGen* principal, se asume que estas comprobaciones ya han sido realizadas previamente.

Al igual que con las claves de encapsulamiento y desencapsulamiento, la *ekPKE* puede hacerse pública mientras que la *dkPKE* debe manetenerse privada, ya que esta última contiene el vector secreto s, que es esencial para el descifrado.

La clave de cifrado $\hat{t}$ se obtiene al aplicar una colección de ecuaciones lineales ruidosas al vector secreto s.

Para ello, primero se generan dos semillas ρ y σ, consecuencia de aplicar la función hash *SHA-512* al array obtenido tras concatenar la semilla d introducida como parámetro y el valor de la variable global k (en total es un array de 33 bytes).

Utilizando la semilla ρ, se genera una matriz pseudoaleatoria $\hat{A}$ de tamaño $k \times k$, donde cada posición contiene un array de 256 elementos en $\mathbb{Z}_q$ obtenidos mediante la función *SampleNTT*.

La matriz $\hat{A}$ se multiplica por el vector secreto s, resultando en un nuevo vector intermedio. A continuación, se suma este vector con otro vector de ruido e, el cual es generado de manera independiente y pseudoaleatoria a partir de la semilla σ, también con la función *SampleNTT*.

Esta suma introduce una perturbación en los valores del vector secreto, lo que añade un nivel de seguridad adicional. El vector resultante es la clave $\hat{t}$, que luego se concatena con la semilla ρ para formar la clave de cifrado *ekPKE*. La semilla ρ se concatena para poder regenerar la matriz $\hat{A}$ en otro lugar.

Finalmente, las dos claves *ekPKE* y *dkPKE* son devueltas como salida, siendo *ekPKE* pública y *dkPKE* privada. Ambas claves son numpy arrays de

tipo np.uint8 con longitud *384*k + 32* para ekPKE y *384*k* para dkPKE.

```
def _K_PKE_KeyGen(d):
    # Transformamos la variable global k a un array de bytes para poder concatenarlo con la semilla d
    k_byte = np.array([ML_KEM.k], dtype=np.uint8)
    array_dk = np.concatenate((d, k_byte))

    # Mediante la función _G obtenemos 2 arrays de 32 bytes
    rho, sigma = ML_KEM._G(array_dk)

    # Creamos una matriz A_hat de dimensión k*k a partir de la semilla rho
    # Los valores de A_hat son el resultado del sample NTT a la semilla + 2 bytes de índices
    N = 0
    A_hat = np.zeros((ML_KEM.k, ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        i_byte = np.array([i], dtype=np.uint8)
        for j in range (ML_KEM.k):
            j_byte = np.array([j], dtype=np.uint8)
            combined_bytes = np.concatenate((rho, i_byte, j_byte))
            A_hat[i, j] = ML_KEM._SampleNTT(combined_bytes)

    # Creamos el vector de coeficientes que contendrá las variables secretas
    s = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        N_byte = np.array([N], dtype=np.uint8)
        s[i] = ML_KEM._SamplePolyCBD(ML_KEM._PRF(ML_KEM.eta_1, sigma, N_byte), ML_KEM.eta_1)
        N += 1

    # Creamos el vector de coeficientes que se utilizará como ruido
    e = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        N_byte = np.array([N], dtype=np.uint8)
        e[i] = ML_KEM._SamplePolyCBD(ML_KEM._PRF(ML_KEM.eta_1, sigma, N_byte), ML_KEM.eta_1)
        N += 1

    # Pasamos ambos vectores a la representación NTT
    s_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    e_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        s_hat[i] = ML_KEM._NTT(s[i])
        e_hat[i] = ML_KEM._NTT(e[i])

    # Creamos t_hat (A_hat*s_hat + e_hat) que será el vector utilizado para crear ekPKE
    t_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        for j in range(ML_KEM.k):
            t_hat[i] = (t_hat[i] + ML_KEM._MultiplyNTTs(A_hat[i, j], s_hat[j])) % ML_KEM.q

    for i in range(ML_KEM.k):
        t_hat[i] = (t_hat[i] + e_hat[i]) % ML_KEM.q

    # ekPKE es el vector t_hat aplicándole una codificación apropiada
    ekPKE = np.zeros(384*ML_KEM.k, dtype=np.uint8)
    for i in range(ML_KEM.k):
        t_encoded = ML_KEM._ByteEncode(t_hat[i], 12)
        for j in range(384):
            ekPKE[384*i+j] = t_encoded[j]

    # Además a ekPKE se le concatena la semilla rho utilizada para crear A_hat
    ekPKE = np.concatenate((ekPKE, rho))

    # La clave de descifrado es el vector de claves secretas s_hat con una codificación apropiada
    dkPKE = np.zeros(384*ML_KEM.k, dtype=np.uint8)
    for i in range(ML_KEM.k):
        s_encoded = ML_KEM._ByteEncode(s_hat[i], 12)
        for j in range(384):
            dkPKE[384*i+j] = s_encoded[j]

    return ekPKE, dkPKE
```

Figura 7.25: Método privado: _K_PKE_KeyGen(d)

_K_PKE_Encrypt(ekPKE, m, r)

El método de cifrado K-PKE Encrypt recibe una clave de cifrado *ekPKE*, un mensaje plano de 32 bytes denominado m y una aleatoriedad r de 32 bytes, y produce un texto cifrado c. Este algoritmo no realiza comprobaciones sobre los inputs que recibe, ya que, al ser una subrutina del método *ML-KEM Encaps* principal, se asume que estas comprobaciones ya han sido realizadas previamente.

A partir de la clave de cifrado, se recupera la semilla utilizada para generar la matriz $\hat{A}$, ya que dicha semilla fue almacenada en *ekPKE* durante su generación. Junto con la semilla, también se extrae el vector $\hat{t}$.

En este punto, se genera un vector $\mathbf{y} \in R_q^k$ y los términos de ruido $\mathbf{e_1} \in R_q^k$ y $e_2 \in R_q$, todos ellos muestreados de la distribución binomial centrada utilizando una función pseudoaleatoria (PRF) a partir de la aleatoriedad de entrada r.

Luego, se calculan u como $u = \hat{A}^\top \mathbf{y} + \mathbf{e_1}$, y v como $v = \hat{t}^\top \mathbf{y} + e_2 + \mu$, donde μ es el mensaje m previamente codificado con d=1. Todos estos cálculos se realizan en el dominio NTT. Una vez realizados estas operaciones, para que u y v estén en R_q, se les aplica la transformada inversa NTT con el método *inverseNTT*.

Finalmente, a u y v se les aplican las compresiones y codificaciones apropiadas gracias a las funciones *Compress* y ByteEncode utilizando las variables globales d_u y d_v como parámetros. Finalmente, u y v se concatenan, dando lugar al texto cifrado c.

```python
def _K_PKE_Encrypt(ekPKE, m, r):

    # Inicializamos t_hat para su posterior recuperación a partir de la clave de cifrado ekPKE
    N = 0
    t_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        t_hat[i] = ML_KEM._ByteDecode(ekPKE[384*i:384*i+384], 12)

    # Extraemos la semilla de ekPKE y recreamos la matriz A_hat con ella
    rho = ekPKE[384*ML_KEM.k:384*ML_KEM.k + 32]
    A_hat = np.zeros((ML_KEM.k, ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        i_byte = np.array([i], dtype=np.uint8)
        for j in range (ML_KEM.k):
            j_byte = np.array([j], dtype=np.uint8)
            combined_bytes = np.concatenate((rho, i_byte, j_byte))
            A_hat[i, j] = ML_KEM._SampleNTT(combined_bytes)

    # Creamos un vector de ruido y
    y = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        N_byte = np.array([N], dtype=np.uint8)
        y[i] = ML_KEM._SamplePolyCBD(ML_KEM._PRF(ML_KEM.eta_1, r, N_byte), ML_KEM.eta_1)
        N += 1

    # Creamos un vector de ruido e1
    e1 = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        N_byte = np.array([N], dtype=np.uint8)
        e1[i] = ML_KEM._SamplePolyCBD(ML_KEM._PRF(ML_KEM.eta_2, r, N_byte), ML_KEM.eta_2)
        N += 1

    # Creamos un valor (256 coeficientes) de ruido
    N_byte = np.array([N], dtype=np.uint8)
    e2 = ML_KEM._SamplePolyCBD(ML_KEM._PRF(ML_KEM.eta_2, r, N_byte), ML_KEM.eta_2)

    # Convertirmos el vector y a su representación NTT
    y_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        y_hat[i] = ML_KEM._NTT(y[i])

    # Callculamos u_hat como (A_hat^T*y_hat)
    u_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        for j in range(ML_KEM.k):
            u_hat[i] = (u_hat[i] + ML_KEM._MultiplyNTTs(A_hat[j, i], y_hat[j])) % ML_KEM.q

    # Pasamos u_hat al anillo R_q
    u = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        u[i] = ML_KEM._InverseNTT(u_hat[i])

    # Le añadimos el ruido e1 a u
    for i in range(ML_KEM.k):
        u[i] = (u[i] + e1[i]) % ML_KEM.q
```

Figura 7.26: Método privado: _K_PKE_Encrypt(ekPKE, m, r) - *Parte 1*

```python
# Decodificamos m (de bytes a 256 enteros)
m_decoded = ML_KEM._ByteDecode(m, 1)

# Y descomprimimos el valor de m_decoded
mu = np.zeros(ML_KEM.n, dtype=np.int64)
for i in range(ML_KEM.n):
    mu[i] = ML_KEM._Decompress(m_decoded[i], 1)

# Calculamos la multiplicación entre (t_hat^T y y_hat)
resultado_multiplicacion = np.zeros(ML_KEM.n, dtype=np.int64)
for i in range(ML_KEM.k):
    resultado_multiplicacion = (resultado_multiplicacion + ML_KEM._MultiplyNTTs(t_hat[i], y_hat[i])) % ML_KEM.q

# Pasamos el resultado de la multiplicación al anillo R_q
mult_invertida = ML_KEM._InverseNTT(resultado_multiplicacion)

# Creamos v como resultado de la inversa de la multiplicación + ruido e2 + mu
v = np.zeros(ML_KEM.n, dtype=np.int64)
v = (v + mult_invertida + e2 + mu) % ML_KEM.q

# Una vez tenemos u y v, comprimimos cada uno de sus valores y los codificamos
# Comprimimos y codificamos u
u_compressed = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
for i in range(ML_KEM.k):
    for j in range(ML_KEM.n):
        u_compressed[i, j] = ML_KEM._Compress(u[i, j], ML_KEM.d_u)
c1 = np.zeros(32*ML_KEM.d_u*ML_KEM.k, dtype=np.uint8)

for i in range(ML_KEM.k):
    u_encoded = ML_KEM._ByteEncode(u_compressed[i], ML_KEM.d_u)
    for j in range(32*ML_KEM.d_u):
        c1[32*ML_KEM.d_u*i+j] = u_encoded[j]

# Comprimimos y codificamos v
v_compressed = np.zeros(ML_KEM.n, dtype=np.int64)
for i in range(ML_KEM.n):
    v_compressed[i] = ML_KEM._Compress(v[i], ML_KEM.d_v)
c2 = ML_KEM._ByteEncode(v_compressed, ML_KEM.d_v)

# Concatenamos u y v para dar así como resultado el texto cifrado
c = np.concatenate((c1, c2))

return c
```

Figura 7.27: Método privado: _K_PKE_Encrypt(ekPKE, m, r) - *Parte 2*

_K_PKE_Decrypt(dkPKE, c)

El método de descifrado K-PKE Decrypt recibe una clave de descifrado *dkPKE* y un texto cifrado c, y aplicando dicha clave al texto cifrado, se recupera un mensaje en texto plano m. Este algoritmo no realiza comprobaciones sobre los inputs que recibe ya que, al ser una subrutina del método *ML-KEM Decaps* principal, se asume que estas comprobaciones ya han sido realizadas previamente.

Primero, el texto cifrado se divide en dos partes: u y v, las cuales están codificadas. Mediante los métodos *Decompress* y *ByteDecode* con sus parámetros d_u y d_v correspondientes, se recuperan u' y v', que son las versiones no codificadas de estas partes.

Como la clave *dkPKE* contiene el vector de variables secretas s (en representación NTT), podemos multiplicarlo por u' (transformado a NTT) y, tras esto, aplicarle la inversa de la transformada NTT para obtener el resultado en R_q, que llamaremos v.

Finalmente, se calcula $v' - v$. Esta diferencia nos da un valor que, tras aplicar los métodos de decodificación, nos permite recuperar m, que es el mensaje original en texto plano.

Este mensaje m es un array de 32 bytes.

```
def _K_PKE_Decrypt(dkPKE, c):

    # Extraemos c1 y c2 creados del texto cifrado
    c1 = c[0:32*ML_KEM.d_u*ML_KEM.k]
    c2 = c[32*ML_KEM.d_u*ML_KEM.k:32*(ML_KEM.d_u*ML_KEM.k + ML_KEM.d_v)]

    # Obtenemos u_prima realizandole las apropiadas decodificaciones y compresiones a c1
    u_prima = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        u_prima[i] = ML_KEM._ByteDecode(c1[32*ML_KEM.d_u*i:32*ML_KEM.d_u*i+32*ML_KEM.d_u], ML_KEM.d_u)

    for i in range(ML_KEM.k):
        for j in range(ML_KEM.n):
            u_prima[i, j] = ML_KEM._Decompress(u_prima[i, j], ML_KEM.d_u)

    # Obtenemos v_prima realizandole las apropiadas decodificaciones y compresiones a c2
    v_prima = np.zeros(ML_KEM.n, dtype=np.int64)
    v_decoded = ML_KEM._ByteDecode(c2, ML_KEM.d_v)
    for i in range(ML_KEM.n):
        v_prima[i] = ML_KEM._Decompress(v_decoded[i], ML_KEM.d_v)

    # Recosntruimos el vector de claves secretas apartir de decodificar la dkPKE
    s_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        s_hat[i] = ML_KEM._ByteDecode(dkPKE[384*i:384*i+384], 12)

    # Queremos conseguir v-inicial, que es la inversa de s_hat^T * NTT(u_prima)
    u_hat = np.zeros((ML_KEM.k, ML_KEM.n), dtype=np.int64)
    for i in range(ML_KEM.k):
        u_hat[i] = ML_KEM._NTT(u_prima[i])

    resultado_multiplicacion = np.zeros(ML_KEM.n, dtype=np.int64)
    for i in range(ML_KEM.k):
        resultado_multiplicacion = (resultado_multiplicacion + ML_KEM._MultiplyNTTs(s_hat[i], u_hat[i])) % ML_KEM.q

    v_inicial = ML_KEM._InverseNTT(resultado_multiplicacion)

    # Calculamos w que es v_prima - v_inicial
    w = (v_prima - v_inicial) % ML_KEM.q

    # Comprimimos y codificamos este mensaje para recuperar el mensaje inicial
    m = np.zeros(32, dtype=np.uint8)
    w_compressed = np.zeros(ML_KEM.n, dtype=np.int64)

    for i in range(ML_KEM.n):
        w_compressed[i] = ML_KEM._Compress(w[i], 1)

    m = ML_KEM._ByteEncode(w_compressed, 1)

    return m
```

Figura 7.28: Método privado: _K_PKE_Decrypt(dkPKE, c)

_ML_KEM_KeyGenInternal(d, z)

Este método recibe dos semillas de 32 bytes cada una generadas aleatoriamente y produce las claves de encapsulación (encaps_k) y de desencapsulación (decaps_k) a partir de estas.

Este método está diseñado para ser un proceso intermedio privado, para así mantener los procesos de generación de claves ocultos al usuario, al ocurrir estos en este método y no directamente en ML_KEM_KeyGen.

Este método es determinístico, es decir, para un determinado input, siempre va a producir el mismo output. Además, este método tampoco realiza comprobaciones sobre los inputs que recibe, ya que, al ser una subrutina del método *ML-KEM KeyGen* principal, se asume que estas comprobaciones ya han sido realizadas previamente.

Lo que hace es utilizar el método de generación de claves de PKE con

el primer array aleatorio que recibe como argumento para obtener así una clave de cifrado *ekPKE* y una clave de descifrado *dkPKE*.

La clave de encapsulación es directamene la propia clave ekPKE.

La clave de desencapsulación es el array de bytes resultante de la concatenación de la clave dkPKE, la clave ekPKE, un hash *SHA3-256* aplicado a ekPKE y un valor aleatorio que corresponde al segundo array de 32 bytes pasado por argumento.

Así, la clave de encapsulación tendrá longitud de *384*k+32* bytes y la clave de desencapsulación tendrá longitud de *768*k+96* bytes.

```
def _ML_KEM_KeyGenInternal(d, z):

    # Se crean las claves de cifrado y descifrado PKE a partir de d
    ekPKE, dkPKE = ML_KEM._K_PKE_KeyGen(d)

    # La clave de encapsulamiento es directamente la clave de cifrado
    encaps_k = ekPKE

    # La clave de desencapsulamiento se calcula de la siguiente manera
    decaps_k = np.concatenate((dkPKE, encaps_k, ML_KEM._H(encaps_k), z))

    return encaps_k, decaps_k
```

Figura 7.29: Método privado: _ML_KEM_KeyGenInternal(d, z)

_ML_KEM_EncapsInternal(encaps_k, m)

Este método utiliza la clave de encapsulamiento y un mensaje m (generado aleatoriamente) para generar la clave secreta K y un texto cifrado.

Este método está diseñado para ser un proceso intermedio privado, para así mantener los procesos de generación de claves ocultos al usuario, al ocurrir estos en este método y no directamente en ML_KEM_Encaps.

Este método es determinístico, es decir, para un determinado input, siempre va a producir el mismo output. Además, este método tampoco realiza comprobaciones sobre los inputs que recibe ya que, al ser una subrutina del método *ML-KEM Encaps* principal, se asume que estas comprobaciones ya han sido realizadas previamente.

Para ello, primero calcula el hash SHA3-512 a la concatenación de m con el resultado del cálculo hash SHA-256 a la clave de encapsulamiento, lo que devuelve dos arrays de 32 bytes, K y r.

Así, K será la clave secreta compartida, y r es un array de 32 bytes que se utilizará como aletoriedad para el cifrado.

Luego, se genera un texto cifrado c utilizando el método K-PKE Encrypt, empleando la clave de encapsulamiento (que recordamos que su valor es el mismo que el de la clave de cifrado ekPKE) , el valor m, y la aleatoriedad r.

El texto cifrado c es crucial para la etapa de desencapsulamiento, ya que contiene los parámetros necesarios (la clave de encapsulamiento y el mensaje m) para que la otra parte pueda crear correctamente la clave secreta K durante el proceso de desencapsulación.

Esta función devuelve la clave secreta K y un texto cifrado c, de 32 y $32^*(d_u * k + d_v)$ bytes respectivamente.

```
def _ML_KEM_EncapsInternal(encaps_k, m):
    # Obtener la clave secreta K y un valor de aletoriedad r
    K, r = ML_KEM._G(np.concatenate((m, ML_KEM._H(encaps_k))))

    # Crear un texto cifrado c
    c = ML_KEM._K_PKE_Encrypt(encaps_k, m, r)

    return K, c
```

Figura 7.30: Método privado: _ML_KEM_EncapsInternal(encaps_k, m)

_ML_KEM_DecapsInternal(decaps_k, c

Este método obtiene la clave secreta K a partir de una clave de desencapsulamiento y un texto cifrado. Si no ha habido interferencias o corrupción en el canal de comunicación, la clave secreta será correcta. De lo contrario, se generará una clave K', que será diferente de la original.

Este método está diseñado para ser un proceso intermedio privado, para así mantener los procesos de generación de claves ocultos al usuario, al ocurrir estos en este método y no directamente en ML_KEM_Decaps.

Este método es determinístico, es decir, para un determinado input, siempre va a producir el mismo output. Además, este método tampoco realiza comprobaciones sobre los inputs que recibe ya que, al ser una subrutina del método *ML-KEM Decaps* principal, se asume que estas comprobaciones ya han sido realizadas previamente.

El primer paso que realiza este algoritmo es serparar la clave de desencapsulamiento dk en las cuatro partes que ya sabemos que la componen: *dkPKE*, *ekPKE*, un hash *SHA-256* de *ekPKE*, y un valor aleatorio z.

Luego, el texto cifrado c se descifra utilizando *dkPKE*, obteniendo el valor m'. A partir de m' y el hash extraído de la clave de desencapsulamiento, se calcula una clave secreta candidata K' y una nueva aleatoriedad r'.

Para verificar si K' es correcta, se vuelve a cifrar m' utilizando la clave *ekPKE* y la aleatoriedad r', generando un nuevo texto cifrado c'. Si c' coincide con el texto cifrado original c, significa que la clave secreta K' es la correcta, y se devuelve como resultado.

Si c' no coincide con c, el método asume que ha habido interferencias o

corrupción. En este caso, se genera una clave secreta incorrecta K, calculada aplicando el hash SHAKE256 a la concatenación de z y c, y esta es la clave que se devuelve.

```python
def _ML_KEM_DecapsInternal(decaps_k, c):
    # Extraemos los 4 arrays que conforman la clave de desencapsulamiento
    dkPKE = decaps_k[0:384*ML_KEM.k]
    ekPKE = decaps_k[384*ML_KEM.k:768*ML_KEM.k+64]
    h = decaps_k[768*ML_KEM.k+32:768*ML_KEM.k+64]
    z = decaps_k[768*ML_KEM.k+64:768*ML_KEM.k+96]

    # Desciframos el mensaje
    m_prima = ML_KEM._K_PKE_Decrypt(dkPKE, c)

    # Generamos K de la misma forma que en el proceso de encapsulamiento
    K_prima, r_prima = ML_KEM._G(np.concatenate((m_prima, h)))

    # Generamos una K_falsa por si los mensajes cifrados no coinciden
    K_falsa = ML_KEM._J(np.concatenate((z, c)))

    # Generamos un texto cifrado de la misma forma que el pasado por argumento
    c_prima = ML_KEM._K_PKE_Encrypt(ekPKE, m_prima, r_prima)

    # Comparamos los dos textos cifrados, en caso de no ser iguales, la K devuelta será la K falsa
    if not np.array_equal(c, c_prima):
        K_prima = K_falsa

    return K_prima
```

Figura 7.31: Método privado: _ML_KEM_DecapsInternal(decaps_k, c)

_comprobarEkValida(encaps_k)

Esta función lo que hace es verificar si una determinada clave de encapsulación tiene el formato válido antes de trabajar con ella.

Para ello, primero verifica si tiene la longitud que debería tener en base a las variables globales que se están utilizando *384*k+32*, así como si su tipo es *np.uint8*. En caso de que esto no sea así, devuelve un mensaje de error.

Segundo, debe comprobar que sus valores, después de ser decodificados y después codificados, deben ser iguales a los valores antes de realizar dichas operaciones. En caso de que esto no sea así, devuelve un mensaje de error.

En caso de que la clave de encapsulación supere ambas pruebas, significa que tiene un formato válido para ser utilizada y el método devuelve True, en caso contrario, devuelve False.

```python
def _comprobarEkValida(encaps_k):
    # Verificar la longitud y el tipo de la clave de encapsulación
    expected_length = 384*ML_KEM.k + 32
    if (len(encaps_k) != expected_length) or (encaps_k.dtype != np.uint8):
        print(f"\nError: Fallo en la longitud o el tipo de Ek.")
        return False

    # Comprueba que los arrays Codificados(Decodificados), tienen el mismo valor que antes de hacerlo
    for i in range(ML_KEM.k):
        test = ML_KEM._ByteEncode(ML_KEM._ByteDecode(encaps_k[384*i:384*i+384], 12), 12)
        if not np.array_equal(test, encaps_k[384*i:384*i+384]):
            print("\nError: Los valores codificados no coinciden con los originales.")
            return False

    return True
```

Figura 7.32: Método privado: _comprobarEk(encaps_k)

Que una clave de encapsulamiento supere esta comprobación no significa que sea una buena clave o una clave fiable, solo afirma que tiene un formato válido.

_comprobarDkValida(decaps_k)

Esta función lo que hace es verificar si una determinada clave de desencapsulación tiene el formato válido antes de trabajar con ella.

Para ello, primero verifica si tiene la longitud que debería tener en base a las variables globales que se están utilizando *768*k+96*, así como si su tipo es *np.uint8*. En caso de que esto no sea así, devuelve un mensaje de error.

Segundo, debe comprobar que, al aplicar el hash SHA-256 sobre los valores de la clave de desencapsulación entre las posiciones *384*k* y *768*k+32* es igual a los valores de la clave de desencapsulación entre las posiciones *768*k+32* y *768*k+64*. Esto se debe a que la clave de desencapsulación, como ya se ha explicado, tiene, en el el primer rango mencionado, la clave ekPKE y en el segundo rango mencionado el hash de la clave ekPKE. En caso de que esto no sea así, devuelve un mensaje de error.

En caso de que la clave de desencapsulación supere ambas pruebas, significa que tiene un formato válido para ser utilizada y el método devuelve True, en caso contrario, devuelve False.

```python
def _comprobarDkValida(decaps_k):
    # Verificar la longitud y el tipo de clave de descapsulación
    expected_length = 768*ML_KEM.k + 96
    if (len(decaps_k) != expected_length) or (decaps_k.dtype != np.uint8):
        print("\nError: Fallo en la longitud o el tipo de Dk.")
        return False

    # Verifica si aplicarle el hash a las posiciones extraídas es el mismo que ya se tiene en dk
    test = ML_KEM._H(decaps_k[384*ML_KEM.k:768*ML_KEM.k+32])
    if not np.array_equal(test, decaps_k[768*ML_KEM.k+32:768*ML_KEM.k+64]):
        print("\nError: Fallo en la verfificación del hash de la clave Dk")
        return False

    return True
```

Figura 7.33: Método privado: _comprobarDk(decaps_k)

Que una clave de desencapsulamiento supere esta comprobación no significa que sea una buena clave o una clave fiable, solo afirma que tiene un formato válido.

__verificarC(c)

Verifica si un determinado texto cifrado tiene el formato válido para ser utilizado.

Los requisitos que debe cumplir el texto cifrado *c* para ser válidos son que su longitud sea igual a *32*d_u*k + d_v* y que sea de tipo *np.uint8*.

Si supera ambas comprobaciones, significa que el texto cifrado *c* tiene un formato válido, devolviendo True, en caso contrario, devuelve False.

Esto es útil cuando el texto cifrado no lo ha generado nuestro proceso de encapsulamiento sino que se ha obtenido mediante otras fuentes.

```
def _verificarC(c):
    # Verificar la longitud y el tipo del texto cifrado
    expected_length = 32*(ML_KEM.d_u*ML_KEM.k + ML_KEM.d_v)
    if (len(c) != expected_length) or (c.dtype != np.uint8):
        print("\nError: Longitud o tipo del texto cifrado inválido.")
        return False

    return True
```

Figura 7.34: Método privado: _verificarC(c)

__establecerEk(encaps_k)

Establece el valor de la clave de encapsulamiento de la clase a una clave introducida por parámetro.

Para ello debe pasar la comprobación de _comprobarEkValida(encaps_k). Aun así, esto no asegura que esa clave sea útil o correcta, simplemente afirma que tiene un formato válido, por lo que su uso es responsabilidad del usuario.

Esto puede ser útil cuando una persona tiene una clave de encapsulamiento suya o de otra fuente y quiere utilizarla para encapsular un mensaje. Por ejemplo, si tengo la clave de encapsulamiento que me ha proporcionado otro usuario y quiero enviarle un mensaje cifrado para que el lo descifre con su clave de desencapsulamiento y que así definamos una clave secreta compartida por ambos.

```python
@staticmethod
def _establecerEk(encaps_k):
    if(ML_KEM._comprobarEkValida(encaps_k)):
        ML_KEM.encaps_k = encaps_k
        print("\nNueva clave de encapsulamiento establecida.")
        return True

    return False
```

Figura 7.35: Método privado: _extablecerEk(encaps_k)

_establecerDk(decaps_k)

Establece el valor de la clave de desencapsulamiento de la clase a una clave introducida por parámetro.

Para ello debe pasar la comprobación de _comprobarDkValida(decaps_k). Aun así, esto no asegura que esa clave sea útil o correcta, simplemente se puede utilizar y es responsabilidad del usuario.

Esto puede ser útil cuando una persona tiene un texto cifrado y quiere desencapsularlo para obtener la clave secreta K, para así poder establecer una clave secreta compartida para intercambiar mesnajes con otro usuario de forma segura.

```python
@staticmethod
def _establecerDk(decaps_k):
    if(ML_KEM._comprobarDkValida(decaps_k)):
        ML_KEM.decaps_k = decaps_k
        print("\nNueva clave de desencapsulamiento establecida.")
        return True

    return False
```

Figura 7.36: Método privado: _extablecerDk(decaps_k)

_compararClavesSecretas(k1, k2)

Este método sirve para comparar si dos claves secretas k1 y k2 son idénticas, devolviendo True en caso de que sí lo sean y False en el caso contrario.

```python
@staticmethod
def _compararClavesSecretas(k1, k2):
    if np.array_equal(k1, k2):
        return True

    return False
```

Figura 7.37: Método privado: _compararClavesSecretas(k1, k2)

7.3.4. Métodos públicos

establecer_parametros(x)

Este método permite variar las variables globales entre los tres tipos de conjuntos de parámetros posibles.

Esto se consigue en función del argumento de entrada: para el valor 1 se establece la configuración *ML-KEM-512*, para el valor 2 se establece la configuración *ML-KEM-768* y para el valor 3 se establece la configuración *ML-KEM-1024*.

Además de modificar las variables globales, se vuelven a crear claves de encapsulación y desencapsulación rellenas de ceros para su posible futura asignación, ahora con su nuevo tamaño (ya que este depende de k).

```
@staticmethod
def establecer_parametros(x):
    # ML-KEM-512
    if x == 1:
        ML_KEM.k = 2
        ML_KEM.eta_1 = 3
        ML_KEM.eta_2 = 2
        ML_KEM.d_u = 10
        ML_KEM.d_v = 4
    # ML-KEM-768
    elif x == 2:
        ML_KEM.k = 3
        ML_KEM.eta_1 = 2
        ML_KEM.eta_2 = 2
        ML_KEM.d_u = 10
        ML_KEM.d_v = 4
    # ML-KEM-1024
    elif x == 3:
        ML_KEM.k = 4
        ML_KEM.eta_1 = 2
        ML_KEM.eta_2 = 2
        ML_KEM.d_u = 11
        ML_KEM.d_v = 5
    else:
        raise ValueError("Valor introducido inválido, debe de ser 1, 2 o 3.")

    ML_KEM.encaps_k = np.zeros(384*ML_KEM.k+32, dtype=np.uint8)
    ML_KEM.decaps_k = np.zeros(768*ML_KEM.k+96, dtype=np.uint8)
```

Figura 7.38: Método público: extablecer_parametros(x)

ML_KEM_KeyGen

El algoritmo de generación de claves *KeyGen* es el algoritmo principal de nuestro esquema ML-KEM, a partir de él generamos las claves de encapsulación y desencapsulación utilizadas para el resto de la comunicación.

Este método no utiliza inputs externos, asegurando que sus procesos sean seguros y aleatorios, sin depender de datos externos. Su funcionamiento se basa en generar dos arrays aleatorios de 32 bytes , d y z, y si no ha habido ningún problema durante su creación, llama al método KeyGenInternal

pasándolos como parámetro para crear las claves de encapsulamiento y de desencapsulamiento.

Estas claves de encapsulamiento y desencapsulamiento son el output del método.

```python
def ML_KEM_KeyGen():
    # Genera aleatoriamente d y z, cada uno de 32 bytes
    d = np.random.randint(0, 256, size=32, dtype=np.uint8)
    z = np.random.randint(0, 256, size=32, dtype=np.uint8)

    # Si d o z se ha generado mal, mostrar error
    if d is None or z is None:
        raise ValueError("\nFallo al generar d o z")

    # Llamar al método interno de generación de claves para generar encaps_k y decaps_k
    encaps_k, decaps_k = ML_KEM._ML_KEM_KeyGenInternal(d, z)

    return encaps_k, decaps_k
```

Figura 7.39: Método público: _ML_KEM_KeyGen()

ML_KEM_Encaps

El algoritmo de encapsulamiento solo recibe como input una clave de encapsulamiento.

Lo que hace es primero generar un mensaje aleatorio m de 32 bytes y, tras esto, utiliza el método EncapsInternal con la clave de encapsulamiento pasada por argumento y el mensaje que se acaba de crear para generar una clave secreta compartida K y un texto cifrado c.

Los outputs del algoritmo son K y c.

Aunque exista el método para comprobar si una clave de encapsulamiento tiene un formato válido, se indica que esto no es responsabilidad del que vaya a usar la clave, sino del que la creó. Por esto, se escpecifica que no se debe comprobar cada vez que se utilice este método, sino que se deben buscar alternativas para comprobar su validez previamente a su uso.

```python
def ML_KEM_Encaps(encaps_k):
    # Genera un mesnaje aleatorio de 32 bytes
    m = np.random.randint(0, 256, size=32, dtype=np.uint8)

    # Si m se ha generado mal, mostrar error
    if m is None:
        raise ValueError("\nFallo al generar m")

    # Llamar al método interno de encapsulamiento para generar la clave secreta K y el texto cifrado c
    K, c = ML_KEM._ML_KEM_EncapsInternal(encaps_k, m)

    return K, c
```

Figura 7.40: Método público: _ML_KEM_Encaps(encaps_k, m)

ML_KEM_Decaps

El algoritmo de desencapsulamiento recibe como input una clave de desencapsulamiento y un texto cifrado.

Lo que hace este algoritmo es llamar al método DecapsInternal con la clave de desencapsulamiento y el texto cifrado introducidos por parámetro para obtener la clave secreta K'. Esta clave será igual a la que tiene el usuario con el que estamos realizando la comunicación siempre y cuando el canal no haya sufrido de interferencias o currupción. En otro caso, dicha clave no será igual a la del usuario y no podremos utilizar estas dos claves para mantener una transmisión segura.

Este método realiza una comprobación previa para verificar si el texto cifrado tiene el formato necesario. Aunque la clave de desencapsulamiento también se debe comprobar que tenga el formato correcto, se indica que esto no es responsabilidad del que vaya a usar la clave, sino del que la creó. Por esto, se escpecifica que no se debe comprobar cada vez que se utilice este método, sino que se deben buscar alternativas para comprobar su validez previamente a su uso.

```
def ML_KEM_Decaps(decaps_k, c):
    if(ML_KEM._verificarC(c)):

        # Llamar al método interno de desencapsulamiento para generar la clave secreta K a partir del texto cifrado c
        K_prima = ML_KEM._ML_KEM_DecapsInternal(decaps_k, c)

        return K_prima
    else:
        raise ValueError("\nEl texto cifrado 'c' no es válido.")
```

Figura 7.41: Método público: _ML_KEM_Decaps(decaps_k, m)

ML_KEM_Interfaz(encaps_k=None, decaps_k=None, K=None, K_prima=None, c=None)

Este método es en sí la interfaz interactiva para utilizar todas las funcionalidades de la clase ML-KEM. Como este método es muy extenso y tiene 7 tipos de acciones distintas, su implementación y uso correcto será explicado en el siguiente capítulo.

```
@staticmethod
def ML_KEM_Interfaz(encaps_k=None, decaps_k=None, K=None, K_prima=None, c=None):

    print("\nBienvenido a la interfaz de usuario del modelo KEM: ML-KEM.")
    print("\nEsta interfaz ha sido desarrollada por Denis Valentin Stoyanov D'Antonio")
    parameter_set = "ML-KEM-768"
    ML_KEM.establecer_parametros(2)
    print(f"\nEl conjunto de parámetros establecido por defecto es: {parameter_set}.")
```

Figura 7.42: Método público: ML_KEM_Interfaz(encaps_k=None, decaps_k=None, K=None, K_prima=None, c=None)

7.4. Traza de ejecución usual

Una traza de ejecución usual sería primero crear el par de claves de encapsulamiento y desencapsulamiento de forma aleatoria, es decir, no conseguirlas de ninguna fuente externa.

A partir de ellas, se realizaría el proceso de encapsulamiento, generando la clave secreta compartida K y el texto cifrado c. Gracias a este c generado en esta etapa, podemos pasar al siguiente paso, el proceso de desencapsulamiento.

Realizamos el desencapsulamiento gracias a la clave de desencapsulamiento y el texto cifrado c obtenido del método anterior, consiguiendo así la clave secreta compartida *K'*.

Finalmente, solo nos queda comprobar si estas dos claves, K y *K'*, son iguales. Lo serán siempre y cuando no haya habido interferencias o corrupción en el canal utilizado, en caso de que se detecte alguna de estas dos amenazas, las claves no serán iguales ya que, como hemos visto, el proceso de desencapsulamiento genera una clave K' distinta para estos casos.

La ejecución explicada en el párrafo anterior puede entenderse como la sucesión de los siguientes 9 pasos:

7.4.1. Paso 1: Generación de claves (ML-KEM KeyGen)

Función utilizada: *ML-KEM.KeyGen()*

El proceso comienza con la generación de las claves necesarias para la encapsulación y desencapsulación. El usuario llama a la función *ML-KEM.KeyGen()*, que es el punto de partida para la generación de claves en el esquema ML-KEM. Esta función:

1. Genera dos arrays aleatorios de 32 bytes denominados d y z. Estos son los parámetros necesarios para crear las claves de encapsulamiento y desencapsulación.

2. Si d o z son nulos, se devuelve un error, garantizando que los valores generados sean seguros y aleatorios.

3. Luego, la función llama a *ML-KEM.KeyGen_internal(d, z)*, que es una subrutina encargada de realizar el trabajo interno de generación de claves.

7.4.2. Paso 2: Generación interna de las claves (ML-KEM KeyGen Internal)

Función utilizada: *ML-KEM.KeyGen_internal(d, z)*

La subrutina *ML-KEM.KeyGen_internal(d, z)* recibe las semillas d y z generadas en el paso anterior. Utiliza estas dos semillas para generar una clave de encapsulamiento $encaps_k$ y una clave de desencapsulamiento $decaps_k$. Los pasos clave son:

1. Invoca el proceso de generación de claves de K-PKE llamando a *K-PKE.KeyGen(d)*, que genera una clave de cifrado pública $ekPKE$ y una clave de descifrado privada $dkPKE$.

2. La clave de encapsulamiento *encaps_k* se deriva directamente de $ekPKE$, mientras que la clave de desencapsulamiento *decaps_k* se construye concatenando $dkPKE$, $ekPKE$, un hash $H(ekPKE)$, y el valor aleatorio z.

3. Estas dos claves, *encaps_k* y *decaps_k*, se devuelven al método *ML-KEM.KeyGen()*.

7.4.3. Paso 3: Encapsulación (ML-KEM Encaps)

Función utilizada: *ML-KEM.Encaps(encaps_k)*

Ahora que tenemos la clave de encapsulamiento *encaps_k*, el siguiente paso es encapsular una clave secreta compartida. Este proceso lo realiza la función *ML-KEM.Encaps(encaps_k)*:

1. Primero, genera internamente un valor aleatorio m de 32 bytes. Este valor m es esencial para generar la clave secreta compartida y el texto cifrado asociado.

2. Luego, llama a *ML-KEM.Encaps_internal(ek, m)*, pasando la clave de encapsulamiento *encaps_k* y el valor aleatorio m.

7.4.4. Paso 4: Encapsulación interna (ML-KEM Encaps Internal)

Función utilizada: *ML-KEM.Encaps_internal(ek, m)*

La subrutina *ML-KEM.Encaps_internal(encaps_k, m)* es responsable de generar una clave secreta K y un texto cifrado c:

1. Crea la clave secreta K y la aleatoriedad r utilizando el hash $G(m||H(ek))$, donde G es una función de hash y $H(ek)$ es un hash de la clave de encapsulamiento.

2. Utiliza el valor r como aleatoriedad para cifrar m utilizando la clave de encapsulamiento *encaps_k*, llamando a *K-PKE.Encrypt(ek, m, r)*, lo que genera el texto cifrado c.

3. La función devuelve tanto la clave secreta K como el texto cifrado c al método *ML-KEM.Encaps()*.

7.4.5. Paso 5: Cifrado K-PKE (K-PKE Encrypt)

Función utilizada: *K-PKE.Encrypt(ekPKE, m, r)*

Este método es invocado dentro de *ML-KEM.Encaps_internal* y se utiliza para cifrar el mensaje m utilizando la clave de cifrado $ekPKE$ y la aleatoriedad r. Los pasos son:

1. Generar un vector aleatorio y y términos de ruido e_1 y e_2, que se utilizarán para cifrar el mensaje.

2. Calcular los valores cifrados u y v, utilizando y, e_1, e_2, y $ekPKE$.

3. Devolver el texto cifrado c, que es la concatenación de u y v, a la función *ML-KEM.Encaps_internal*.

7.4.6. Paso 6: Desencapsulación (ML-KEM Decaps)

Función utilizada: *ML-KEM.Decaps(decaps_k, c)*

Después de que la parte que encapsuló la clave secreta K envía el texto cifrado c al otro usuario, la parte receptora puede utilizar la clave de desencapsulamiento *decaps_k* y el texto cifrado c para recuperar la clave secreta compartida K'. Esto se realiza llamando a la función *ML-KEM.Decaps(dk, c)*, que:

1. Llama al método *ML-KEM.Decaps_internal(decanps_k, c)*, que es responsable del trabajo interno de desencapsulación.

7.4.7. Paso 7: Desencapsulación interna (ML-KEM Decaps Internal)

Función utilizada: *ML-KEM.Decaps_internal(decaps_k, c)*

La subrutina *ML-KEM.Decaps_internal(decaps_k, c)* es responsable de recuperar la clave secreta compartida K':

1. Descifra el mensaje m' utilizando la clave de descifrado $dkPKE$ y el texto cifrado c, llamando a *K-PKE.Decrypt(dkPKE, c)*.
2. Crea una nueva clave secreta K' y una nueva aleatoriedad r' a partir de m' y $H(ek)$, utilizando el mismo proceso que en la encapsulación.
3. Genera un nuevo texto cifrado c' utilizando m' y r' para verificar que coincide con el c original. Si coincide, la clave secreta K' es correcta. Si no, el método genera una clave secreta incorrecta K', derivada de un hash SHAKE256 aplicado a la concatenación del valor aleatorio z y el texto cifrado c.

7.4.8. Paso 8: Descifrado K-PKE (K-PKE Decrypt)

Función utilizada: *K-PKE.Decrypt(dkPKE, c)*

Este método descifra el mensaje m' utilizando la clave de descifrado $dkPKE$ y el texto cifrado c, devolviendo el mensaje descifrado m', que luego se utiliza en el proceso de creación de K' en *ML-KEM.Decaps_internal*.

7.4.9. Paso 9: Comparación de las claves K y K'

Función utilizada: *Comparación directa*

Al final del proceso, las dos partes que han utilizado ML-KEM deberían haber generado la misma clave secreta compartida K en el lado del encapsulamiento y K' en el lado del desencapsulamiento. Si todo ha ido correctamente, K y K' serán iguales, permitiendo así una comunicación segura. Si K y K' no coinciden, esto indica que ha habido una interferencia o corrupción en el canal de comunicación.

7.5. Prueba de rendimiento para las distintas configuraciones de parámetros

He realizado tres mediciones de tiempo, una para cada conjunto de parámetros, ML-KEM-512, ML-KEM-768 y ML-KEM-1024; para comprobar el tiempo promedio que se tarda en generar las claves de encapsulamiento y desencapsulamiento.

Para ello he utilizado la librería timeit, la cual permite ejecutar una determinada función un número establecido de veces.

Yo he ejecutado la función ML_KEM_KeyGen un total de 100 veces para cada configuración. Dichas configuraciones son establecidas gracias a que, antes de medir el tiempo, utilizo el método establecer_parametros(x), siendo x: 1 para ML-KEM-512, 2 para ML-KEM-768 y 3 para ML-KEM-1024.

El código para realizar la prueba es el siguiente:

```python
# Importar la clase ML_KEM desde el archivo ml_kem.py
from ml_kem import ML_KEM
import timeit

# Main creado para utilizar la interfaz de la clase ML_KEM
if __name__ == "__main__":

    # Prueba de rendimiento de la generación de claves para las distintas configuraciones
    def medir_ml_kem_keygen():
        clave_encaps, clave_decaps = ML_KEM.ML_KEM_KeyGen()

    # Para el conjunto de parámetros ML-KEM-512
    ML_KEM.establecer_parametros(1)

    tiempo = timeit.timeit('medir_ml_kem_keygen()', globals=globals(), number=100)
    print(f"Tiempo promedio de ejecución (100 repeticiones) para ML-KEM-512: {tiempo/100} segundos")

    # Prueba de rendimiento de la generación de claves para las distintas configuraciones
    # Para el conjunto de parámetros ML-KEM-768
    ML_KEM.establecer_parametros(2)

    tiempo = timeit.timeit('medir_ml_kem_keygen()', globals=globals(), number=100)
    print(f"Tiempo promedio de ejecución (100 repeticiones) para ML-KEM-768: {tiempo/100} segundos")

    # Prueba de rendimiento de la generación de claves para las distintas configuraciones
    # Para el conjunto de parámetros ML-KEM-1024
    ML_KEM.establecer_parametros(3)

    tiempo = timeit.timeit('medir_ml_kem_keygen()', globals=globals(), number=100)
    print(f"Tiempo promedio de ejecución (100 repeticiones) para ML-KEM-1024: {tiempo/100} segundos")
```

Figura 7.43: Código para medir el rendimiento de *ML-KEM-KeyGen()* para cada una de las tres posibles configuraciones

Los resultados obtenidos para estas pruebas son:

```
denisvs@denisvs-ASUS:~/Escritorio/TFG_DenisStoyanovFinal/Algoritmo_ML-KEM$ python3 ml_kem_main.py
Tiempo promedio de ejecución (100 repeticiones) para ML-KEM-512: 0.06250214063000044 segundos
Tiempo promedio de ejecución (100 repeticiones) para ML-KEM-768: 0.08338018153999997 segundos
Tiempo promedio de ejecución (100 repeticiones) para ML-KEM-1024: 0.1117058832500004 segundos
denisvs@denisvs-ASUS:~/Escritorio/TFG_DenisStoyanovFinal/Algoritmo_ML-KEM$
```

Figura 7.44: Tiempo promedio en segundos que tarda en ejecutarse *ML-KEM-KeyGen()* para cada una de las tres posibles configuraciones

A partir de estos resultados, podemos comprobar claramente como conforme utilizamos un conjunto de parámetros mayor, lo que supone mayor tamaño de claves y, por ende, mayor seguridad; también se incrementa el tiempo de ejecución.

7.6. Problemas encontrados durante la creación del algoritmo

Algunos de los problemas afrontados para la creación e implementación del algoritmo ML-KEM en el que se fundamenta el estándar FIPS 203 han sido los siguientes:

En el método sampleNTT(B), la forma de obtener los 3 bytes pseudo-aleatorios especificada en el pseudocódigo de la documentación era generar estos 3 bytes en cada iteración. Sin embargo, al utilizar el método .digest(3) siempre se producían los mismos 3 valores. Esto resultaba en dos grandes problemas:

1. Si ambos coeficientes calculados eran mayores que q, ninguno de los dos se utilizaba para rellenar el vector a_hat y, como no cambiaban, esto producía un bucle infinito.

2. En caso de que uno (o los dos) coeficientes calculados fuesen menores que q, al no cambiar de valor al pasar de iteración, generaban una matriz llena de valores repetidos; lo cuál no era válido.

Para solucionar esto, lo que he hecho ha sido realizar el digest con salida de 840 bytes, que es el número máximo de iteraciones (280) * el número de bytes utilizado en cada iteración (3). Así, teniéndolos precalculados, solo debo ir extrayendo de 3 en 3, asegurando así que todos los tríos de valores son distintos entre sí y que nunca se producen bucles infinitos.

Otro problema que tuve fue que, a la hora de querer asignar claves de encapsulamiento o desencapsulamiento a partir de una fuente externa, no tenía donde almacenarlas para realizar otras operaciones sin perderlas. Por ello, tuve que crear dos variables globales denominadas encaps_k y decaps_k con sus respectivas longitudes adecuadas en función del conjunto de parámetros que utilizase.

Finalmente, un error muy grave que realizaba al comienzo de la implementación del algoritmo hasta que comprendí correctamente su uso a partir de la documentación fue aplicarle algunas funciones directamente a un vector en lugar de a cada una de sus posiciones. Funciones como por ejemplo NTT, InverseNTT, Compress, Decompress, etc.

Capítulo 8

Interfaz del algoritmo ML-KEM

8.1. Objetivos de la interfaz

Esta interfaz está diseñada para que cualquier usuario pueda utilizar correctamente este tipo de esquema de encapsulamiento de claves sin necesidad de tener ningún tipo de conocimientos previos. Evidentemente, si se tienen, se le podrá sacar más provecho a dicha interfaz.

Para utilizar dicha interfaz, debemos importar la clase ML_KEM y hacer la debida llamada a la función ML_KEM_Interfaz.

```python
# Importar la clase ML_KEM desde el archivo ml_kem.py
from ml_kem import ML_KEM

# Main creado para utilizar la interfaz de la clase ML_KEM
if __name__ == "__main__":

    # Utilizar la interfaz por defecto
    ML_KEM.ML_KEM_Interfaz()
```

Figura 8.1: Cómo utilizar el método ML_KEM_Interfaz desde un archivo externo

8.2. Cómo está desarrollada la interfaz

La interfaz cuenta con un menú inicial donde se da la bienvenida, se hace referencia a su creador (yo) y se advierte del conjunto de parámetros establecido por defecto, que como recomienda el NIST es el ML-KEM-768.

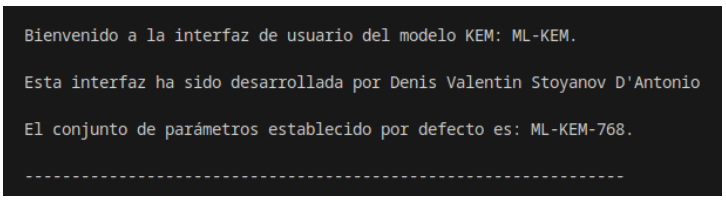


Figura 8.2: Mensaje de bienvenida al iniciar la interfaz de ML-KEM

```
@staticmethod
def ML_KEM_Interfaz(encaps_k=None, decaps_k=None, K=None, K_prima=None, c=None):

    print("\nBienvenido a la interfaz de usuario del modelo KEM: ML-KEM.")
    print("\nEsta interfaz ha sido desarrollada por Denis Valentin Stoyanov D'Antonio")
    parameter_set = "ML-KEM-768"
    ML_KEM.establecer_parametros(2)
    print(f"\nEl conjunto de parámetros establecido por defecto es: {parameter_set}.")
```

Figura 8.3: Código del mensaje de bienvenida al iniciar la interfaz de ML-KEM

Tras esto, entra en un menú interactivo con siete opciones principales más una para cerrar la interfaz. Además, cada vez que se muestre este menú principal, también se mostrará qué conjunto de parámetros hay seleccionado en ese momento y cuáles de los cinco datos fundamentales para llevar a cabo las funciones de encapsular y desencapsular se encuentran establecidos y cuáles no.

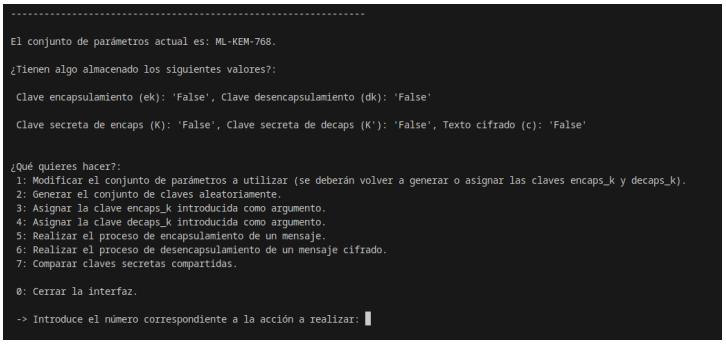


Figura 8.4: Menú de la interfaz de ML-KEM

```
print("\n----------------------------------------------------------------")
print(f"\nEl conjunto de parámetros actual es: {parameter_set}.")
print("\n¿Tienen algo almacenado los siguientes valores?: ")
print(f"\n Clave encapsulamiento (ek): '{encaps_k is not None}', Clave desencapsulamiento (dk): '{decaps_k is not None}'")
print(f"\n Clave secreta de encaps (K): '{K is not None}', Clave secreta de decaps (K'): '{K_prima is not None}', Texto cifrado (c): '{c is not None}'")

print("\n\n¿Qué quieres hacer?: ")
print(" 1: Modificar el conjunto de parámetros a utilizar (se deberán volver a generar o asignar las claves encaps_k y decaps_k).")
print(" 2: Generar el conjunto de claves aleatoriamente.")
print(" 3: Asignar la clave encaps_k introducida como argumento.")
print(" 4: Asignar la clave decaps_k introducida como argumento.")
print(" 5: Realizar el proceso de encapsulamiento de un mensaje.")
print(" 6: Realizar el proceso de desencapsulamiento de un mensaje cifrado.")
print(" 7: Comparar claves secretas compartidas.")
print("\n 0: Cerrar la interfaz.")

x = int(input("\n -> Introduce el número correspondiente a la acción a realizar: "))
if(0 <= x <= 7):
    seleccion = x
else:
    print("\nEl número introducido no tiene un valor correcto.")
```

Figura 8.5: Código del menú de la interfaz de ML-KEM

Para elegir una opción de este menú habrá que introducir un valor del 0 al 7, cualquier otro valor no será aceptado y se volverá a pedir que se introduzca un número válido.

```
 -> Introduce el número correspondiente a la acción a realizar: 9

El número introducido no tiene un valor correcto.
```

Figura 8.6: Número inválido introducido en la interfaz de ML-KEM

Las siete acciones proporcionadas por el menú principal son:

8.2.1. Acción 1: Modificar el conjunto de parámetros a utilizar

El conjunto de parámetros por defecto es ML-KEM-768 pero, como ya sabemos, existen también ML-KEM-512 y ML-KEM-1024. Así, mediante un menú interactivo, podemos variar entre ellos.

```
 -> Introduce el número correspondiente a la acción a realizar: 1

----------------------------------------------------------------

¿A qué conjunto de parámetros quieres pasar?:
 1: ML-KEM-512 (mayor rendimiento)
 2: ML-KEM-768 (equilibrado)
 3: ML-KEM-1024 (mayor seguridad)

 0: Volver al menú principal.

 -> Introduce el número correspondiente al conjunto de parámetros a utilizar:
```

Figura 8.7: Acción 1 de la interfaz: Elegir conjunto de parámetros al que cambiar partir de la interfaz

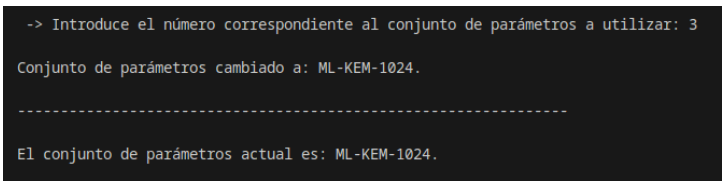


Figura 8.8: Conjunto de parámetros cambiado a partir de la interfaz

Hay dos consideraciones muy importantes a tener en cuenta:

- Si generamos un conjunto de claves con un determinado conjunto de parámetros y luego cambiamos a otro, tendremos que volver a generar las claves, ya que los métodos de encapsulación y desencapsulación utilizarían unas comprobaciones con unos parámetros distintos a los utilizados para crear las claves.

- Si utilizamos claves pre-creadas introduciéndolas en la interfaz, debemos de asegurarnos de cambiar al conjunto de parámetros para el que el formato de dichas claves es válido.

El funcionamiento interno de esta función es el siguiente:

Se solicita que el usuario seleccione el valor acorde al conjunto al que se desea cambiar: 1 para ML-KEM-512, 2 para ML-KEM-768 y 3 para ML-KEM-1024; o el 0 para volver atrás. Si se introduce un número fuera del rango 0-3 se advertirá y se volverá a solicitar que se introduzca un número válido.

Si se selecciona el mismo con el que ya se está trabajando, se advertirá por pantalla y no hará nada más, regresando al menú principal para poder seguir escogiendo otras acciones.

Si se selecciona uno distinto al actual, se llamará a la función establecer_parámetros(x) con el número introducido como input, estableciendo el conjunto correspondiente.

```
# Modificar parámeter set
if(seleccion == 1):
    print("\n-----------------------------------------------------------------")
    print("\n¿A qué conjunto de parámetros quieres pasar?: ")
    print(" 1: ML-KEM-512 (mayor rendimiento)")
    print(" 2: ML-KEM-768 (equilibrado)")
    print(" 3: ML-KEM-1024 (mayor seguridad)")
    print("\n 0: Volver al menú principal.")

    # Asegurar que el valor introducido es correcto
    x = int(input("\n -> Introduce el número correspondiente al conjunto de parámetros a utilizar: "))
    while (x != 0 and x != 1 and x != 2 and x != 3):
        print("\nEl número introducido no tiene un valor correcto.")
        x = int(input("\n -> Introduce el número correspondiente al conjunto de parámetros a utilizar: "))

    # Modificar el conjunto de parámetros o volver al menú principal
    if(x == 1):
        if(parameter_set == "ML-KEM-512"):
            print("\nYa tienes establecido el conjunto de parámetros ML-KEM-512")
        else:
            ML_KEM.establecer_parametros(x)
            print("\nConjunto de parámetros cambiado a: ML-KEM-512.")
            parameter_set = "ML-KEM-512"
            bloqueo = 2
            seleccion = -1
    elif(x == 2):
        if(parameter_set == "ML-KEM-768"):
            print("\nYa tienes establecido el conjunto de parámetros ML-KEM-768")
        else:
            ML_KEM.establecer_parametros(x)
            print("\nConjunto de parámetros cambiado a: ML-KEM-768.")
            parameter_set = "ML-KEM-768"
            bloqueo = 2
            seleccion = -1
    elif(x == 3):
        if(parameter_set == "ML-KEM-1024"):
            print("\nYa tienes establecido el conjunto de parámetros ML-KEM-1024")
        else:
            ML_KEM.establecer_parametros(x)
            print("\nConjunto de parámetros cambiado a: ML-KEM-1024.")
            parameter_set = "ML-KEM-1024"
            bloqueo = 2
            seleccion = -1
    else:
        seleccion = -1
```

Figura 8.9: Código de la acción 1 de la interfaz: Elegir conjunto de parámetros al que cambiar partir de la interfaz

8.2.2. Acción 2: Generar el conjunto de claves aleatoriamente

Esta acción genera un conjunto de claves desde cero, una de encapsulamiento y otra desencapsulamiento.

Para ello llama al método público ML_KEM_KeyGen() que genera dichas claves y, tras esto, establece sus valores a las dos variables globales encaps_k y decaps_k de la clase ML-KEM para poder trabajar con ellas desde cualquier método.

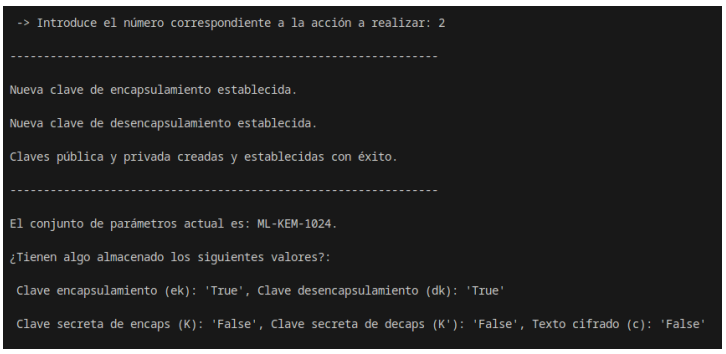


Figura 8.10: Acción 2 de la interfaz: Generar el conjunto de claves aleatoriamente

```python
# Generar claves aleatorias
if(seleccion == 2):
    print("\n----------------------------------------------------------------")
    while(seleccion == 2): # Para cuando de error el squeeze
        encaps_k, decaps_k = ML_KEM.ML_KEM_KeyGen()
        if(ML_KEM._establecerEk(encaps_k) and ML_KEM._establecerDk(decaps_k)):
            print("\nClaves pública y privada creadas y establecidas con éxito.")
            bloqueo = 0
            seleccion = -1
```

Figura 8.11: Código de la acción 2 de la interfaz: Generar el conjunto de claves aleatoriamente

8.2.3. Acción 3: Asignar la clave encaps_k introducida como argumento

Como ya hemos comentado repetidas veces, quizás el usuario ya dispone de una clave de encapsulamiento y desea directamente encapsular un mensaje con ella, es decir, que no necesita crearla previamente.

Esta interfaz dispone de la acción de asignar una clave introducida como argumento a la variable global encaps_k de la clase ML_KEM para así poder trabajar con ella directamente.

Para ello se llama al método establecerEk, el cual, internamente, tiene la comprobación de si encaps_k tiene un formato válido.

Cabe recordar que quizás la clave introducida si es válida e incluso funcional, pero el método puede darnos error si no estamos en el mismo conjunto de parámetros con el que se creó dicha clave.

Foto del código.

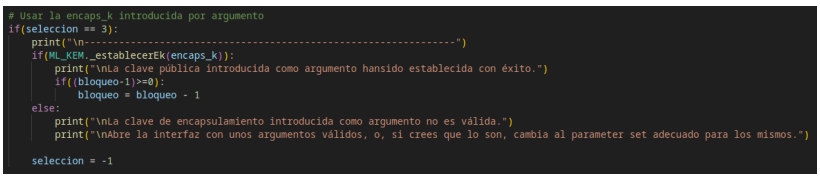


Figura 8.12: Código de la acción 3 de la interfaz: Asignar una clave de encapsulamiento pasada como argumento

8.2.4. Acción 4: Asignar la clave decaps_k introducida como argumento

Como ya hemos comentado repetidas veces, quizás el usuario ya dispone de una clave de desencapsulamiento y un texto cifrado y desea desencapsular este texto cifrado directamente, sin realizar ningún otro tipo de acción previa.

Esta interfaz dispone de la acción de asignar una clave introducida como argumento a la variable global decaps_k de la clase ML_KEM para así poder trabajar con ella directamente.

Para ello se llama al método establecerDk, el cual, internamente, tiene la comprobación de si decaps_k tiene un formato válido.

Cabe recordar que quizás la clave introducida si es válida e incluso funcional, pero el método puede darnos error si no estamos en el mismo conjunto de parámetros con el que se creó dicha clave.

Foto del código.

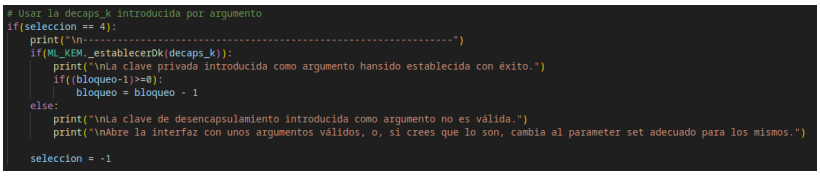


Figura 8.13: Código de la acción 4 de la interfaz: Asignar una clave de desencapsulamiento pasada como argumento

8.2.5. Acción 5: Realizar el proceso de encapsulamiento de un mensaje

Siempre y cuando haya establecida una clave de encapsulamiento en la interfaz, independientemente de si ha sido generada a partir de la propia interfaz o introducida como parámetro, se realizará el proceso de encapsulamiento, generando una clave secreta K y un texto cifrado c.

Esto se realiza llamando a la función ML_KEM_Encaps utilizando como parámetro el valor encaps_k que tiene asignado la interfaz actualmente.

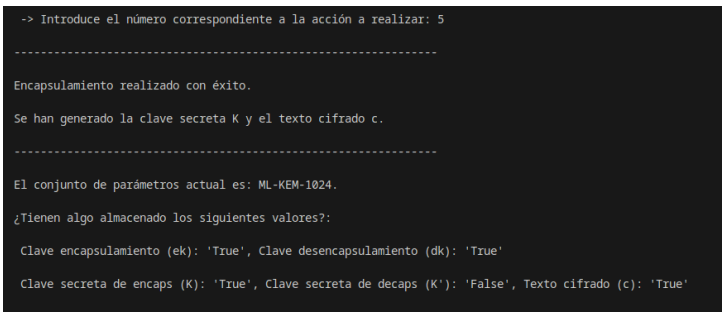


Figura 8.14: Acción 5 de la interfaz: Encapsular un mensaje generado aleatoriamente

```python
# Proceso de encapsulamiento
if(seleccion == 5):
    print("\n----------")
    if(bloqueo == 0):
        if(encaps_k is not None):
            K, c = ML_KEM.ML_KEM_Encaps(encaps_k)
            print("\nEncapsulamiento realizado con éxito.")
            print("\nSe han generado la clave secreta K y el texto cifrado c.")
        else:
            print("\nError: No hay ninguna clave de encapsulamiento para usar actualmente.")
    else:
        print("\nDespués de modificar los parámetros, tienes que volver a crear o asignar las claves encaps_k y decaps_k")
    seleccion = -1
```

Figura 8.15: Código de la acción 5 de la interfaz: Encapsular un mensaje generado aleatoriamente

.

8.2.6. Acción 6: Realizar el proceso de desencapsulamiento de un mensaje cifrado

Siempre y cuando se tengan establecidos una clave de desencapsulamiento y un texto cifrado en la interfaz, independientemente de si han sido generados a partir de la acción de encapsular de la propia interfaz, o si han sido introducidos por parámetro, se podrá realizar la función de desencapsulamiento a dicho texto cifrado gracias a esta clave de desencapsulamiento.

Para ello, se llama el método ML_KEM_Decaps con la clave decaps_k y el texto cifrado que tiene asignados la interfaz en ese momento, devolviendo una clave secreta K.

```
 -> Introduce el número correspondiente a la acción a realizar: 6

------------------------------------------------------------------

Desencapsulamiento realizado con éxito.

Se ha obtenido la clave secreta K_prima.

------------------------------------------------------------------

El conjunto de parámetros actual es: ML-KEM-1024.

¿Tienen algo almacenado los siguientes valores?:

 Clave encapsulamiento (ek): 'True', Clave desencapsulamiento (dk): 'True'

 Clave secreta de encaps (K): 'True', Clave secreta de decaps (K'): 'True', Texto cifrado (c): 'True'
```

Figura 8.16: Acción 6 de la interfaz: Desencapsular un texto cifrado

```
# Proceso de desencapsulamiento
if(seleccion == 6):
    print("\n------------------------------------------------------------------")
    if(bloqueo == 0):
        if(decaps_k is not None and c is not None):
                K_prima = ML_KEM.ML_KEM_Decaps(decaps_k, c)
                print("\nDesencapsulamiento realizado con éxito.")
                print("\nSe ha obtenido la clave secreta K_prima.")
        else:
            print("\nError: No hay ninguna clave de desencapsulamiento para usar actualmente.")
    else:
        print("\nDespués de modificar los parámetros, tienes que volver a crear o asignar las claves encaps_k y decaps_k")
    seleccion = -1
```

Figura 8.17: Código de la acción 6 de la interfaz: Desencapsular un texto cifrado

8.2.7. Acción 7: Comparar claves secretas compartidas

Esta acción compara si dos claves secretas K y K' son la misma o no, en caso de serlo, significa que esas dos claves se pueden utilizar como clave secreta segura para una comunicación.

Estas dos claves K y K' se pueden obtener a partir de realizar un encapsulamiento y de realizar un desencapsulamiento, respectivamente; o de ser introducidas como parámetro si alguien desea directamente saber si son iguales o no sin mirar su composición interna.

```
 -> Introduce el número correspondiente a la acción a realizar: 7

------------------------------------------------------------------

Se han comparado las claves secretas generadas durante el encapsulamiento y el desencapsulamiento y son iguales.

Por lo tanto, el modelo ML-KEM funciona correctamente con este conjunto de claves y parámetros.

------------------------------------------------------------------
```

Figura 8.18: Acción 7 de la interfaz: Comprarar dos claves secretas K y K'

```
# Comparar las claves K y K_prima
if(seleccion == 7):
    print("\n-----------------------------------------------------------------")
    if(K is not None and K_prima is not None):
        if ML_KEM._compararClavesSecretas(K, K_prima):
                print("\nSe han comparado las claves secretas generadas durante el encapsulamiento y el desencapsulamiento y son iguales.")
                print("\nPor lo tanto, el modelo ML-KEM funciona correctamente con este conjunto de claves y parámetros.")
        else:
            print("\nError: Las claves secretas K y K_prima no coinciden.")
            print(f"Clave K generada en el proceso de encapsulamiento: {K}")
            print(f"Clave K' generada en el proceso de desencapsulamiento:: {K_prima}")
    elif(K is None):
        print("\nLa clave K generada en el proceso de encapsulamiento no tiene valor.")
    else:
        print("\nLa clave K' generada en el proceso de desencapsulamiento no tiene valor.")

    seleccion = -1
```

Figura 8.19: Código de la acción 7 de la interfaz: Comprarar dos claves secretas K y K'

8.2.8. Acción 0: Cerrar la interfaz

Como su propio nombre indica, esta acción cierra la interfaz, mostrando con ello un mensaje de despediada.

```
 -> Introduce el número correspondiente a la acción a realizar: 0

-----------------------------------------------------------------

Gracias por utilizar la interfaz de ML_KEM, hasta pronto.

-----------------------------------------------------------------
```

Figura 8.20: Acción 0 de la interfaz: Cerrar la interfaz

8.3. Ejemplo de interfaz con claves aleatorias

Un usuario que quiera generar una clave secreta compartida K desde cero debería seguir los siguientes pasos en la interfaz:

Para llamar a la interfaz con este objetivo, no necesitamos introducirle ningún parámetro.

```
if __name__ == "__main__":
    ML_KEM.ML_KEM_Interfaz()
```

Figura 8.21: Llamada a la interfaz para generar claves aleatorias

Primero, establecer el conjunto de parámetros deseado (acción 1).

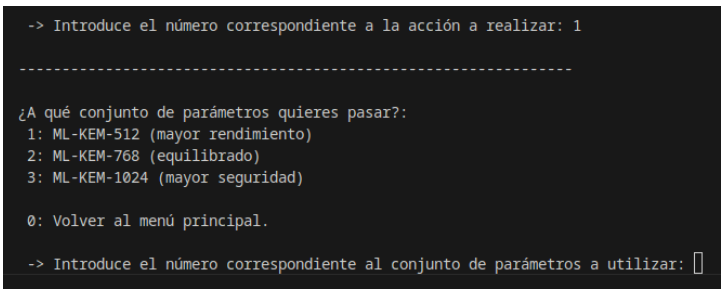


Figura 8.22: Acción 1 de la interfaz: Elegir conjunto de parámetros al que cambiar partir de la interfaz

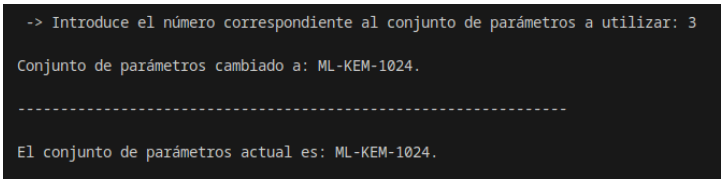


Figura 8.23: Conjunto de parámetros cambiado a partir de la interfaz

Segundo, generar el conjunto de claves aleatoriamente (acción 2).

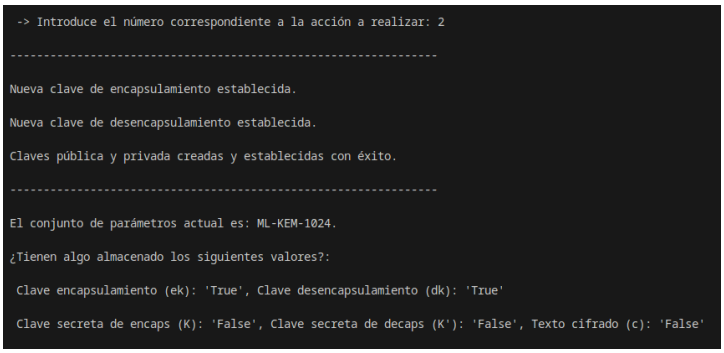


Figura 8.24: Acción 2 de la interfaz: Generar el conjunto de claves aleatoriamente

Tercero, realizar el proceso de encapsulamiento de un mensaje (acción 5).

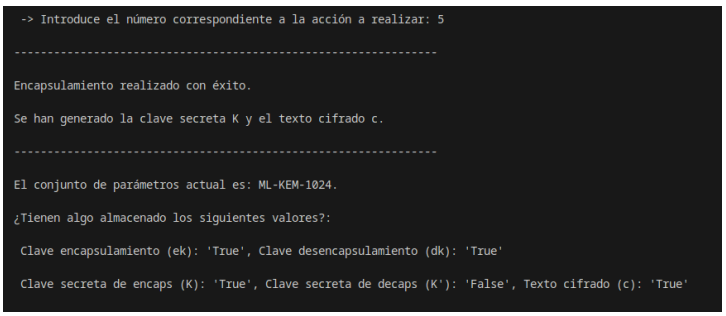


Figura 8.25: Acción 5 de la interfaz: Encapsular un mensaje generado aleatoriamente

Cuarto, realizar el proceso de desencapsulamiento de un mensaje cifrado (acción 6).

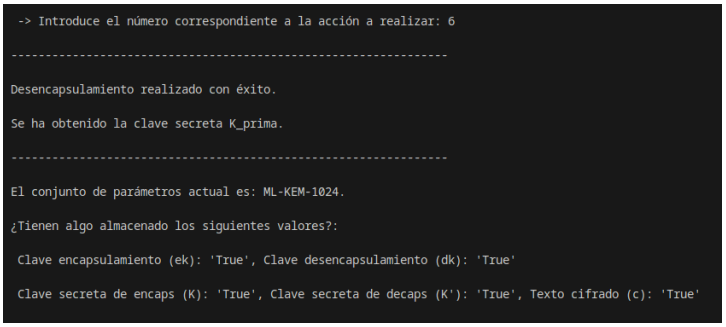


Figura 8.26: Acción 6 de la interfaz: Desencapsular un texto cifrado

Finalmente, comparar si las claves obtenidas en las acciones 5 y 6 son la misma (acción 7).

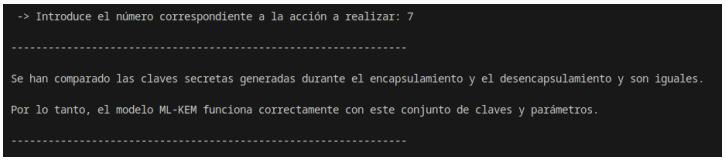


Figura 8.27: Acción 7 de la interfaz: Comprarar dos claves secretas K y K'

Toda la ejecución se ha realizado correctamente y hemos conseguido establecer una clave secreta K para una comunicación segura entre dos usuarios.

8.4. Ejemplo de interfaz con claves por parámetro

Si un usuario quiere utilizar un conjunto de claves de encapsulamiento y de desenacapsulamiento introducidas por parámetro, es decir, que ya las tiene de alguna otra parte, los pasos que debería de seguir son:

```
if __name__ == "__main__":

    # tilizar la interfaz con un conjunto de claves encaps-decaps precreadas
    ML_KEM.establecer_parametros(1)
    clave_encaps, clave_decaps = ML_KEM.ML_KEM_KeyGen()
    ML_KEM.ML_KEM_Interfaz(encaps_k=clave_encaps, decaps_k=clave_decaps)
```

Figura 8.28: Claves creadas y pasadas como argumentos a la interfaz

Comprobamos que la interfaz haya detectado que se le han pasado las claves de encapsulamiento y desencapsulamiento por parámetro.

```
------------------------------------------------------------------

El conjunto de parámetros actual es: ML-KEM-768.

¿Tienen algo almacenado los siguientes valores?:

 Clave encapsulamiento (encaps_k): 'True', Clave desencapsulamiento (decaps_k): 'True'

 Clave secreta de encaps (K): 'False', Clave secreta de decaps (K'): 'False', Texto cifrado (c): 'False'
```

Figura 8.29: La interfaz detecta las claves creadas y pasadas como argumentos a la interfaz

Primero, debemos pasar al conjunto de parámetros adecuado para dichas claves si no lo está ya (acción 1). En este caso, debemos pasar de ML-KEM-768 a ML-KEM-512, que es el conjunto de parámetros en el que se han creado.

```
 -> Introduce el número correspondiente a la acción a realizar: 3

------------------------------------------------------------------

Error: Fallo en la longitud o el tipo de Ek.

La clave de encapsulamiento introducida como argumento no es válida.

Abre la interfaz con unos argumentos válidos, o, si crees que lo son, cambia al parameter set adecuado para los mismos.

------------------------------------------------------------------

El conjunto de parámetros actual es: ML-KEM-768.
```

Figura 8.30: Ejemplo intentar asignar la clave de encapsulamiento para una interfaz que está configurada con otro conjunto de parámetros

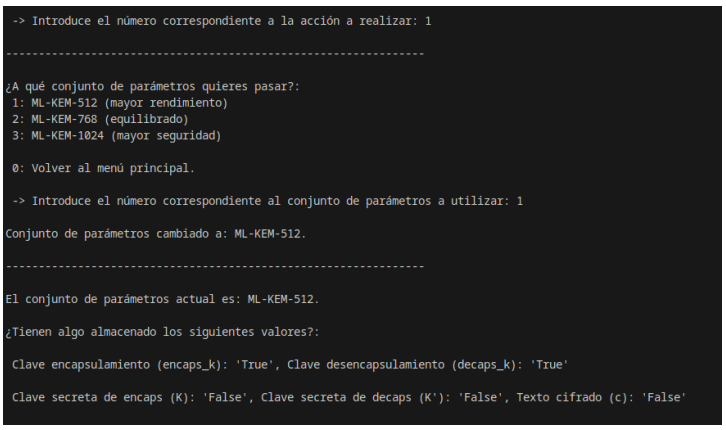


Figura 8.31: Pasar al conjunto de parámetros correcto para la clave de encapsulamiento introducida por parámetro (ML-KEM-512 en este caso)

Segundo, asignar la clave encaps_k introducida como argumento (acción 3).

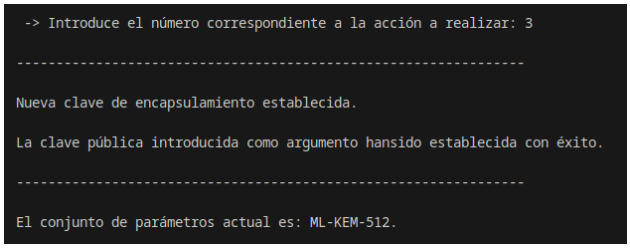


Figura 8.32: Asignar el valor de la clave de encapsulamiento introducido por parámetro como clave de encapsulamiento de la clase

Tercero, asignar la clave decaps_k introducida como argumento (acción 4).

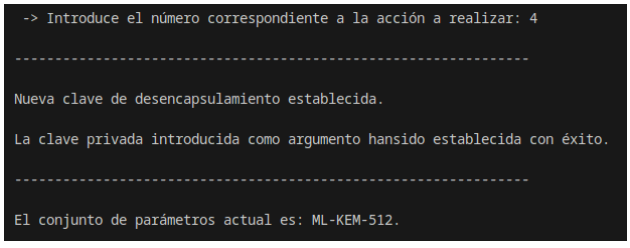


Figura 8.33: Asignar el valor de la clave de desencapsulamiento introducido por parámetro como clave de desencapsulamiento de la clase

Cuarto, realizar el proceso de encapsulamiento de un mensaje (acción

5). Vemos como la interfaz detecta que tras esta acción se han establecido los valores de K y c.

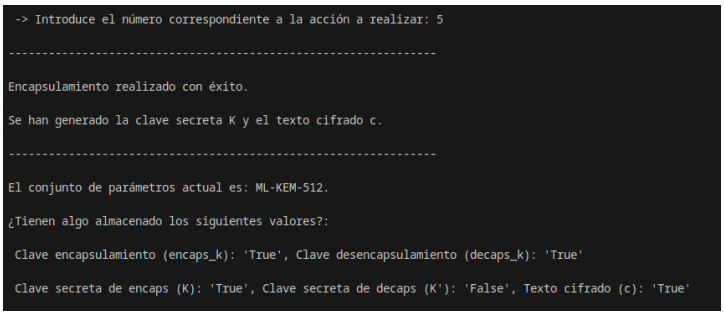


Figura 8.34: Realizar el proceso de encapsulamiento con la clave encaps_k introducida por parámetro

Quinto, realizar el proceso de desencapsulamiento de un mensaje cifrado (acción 6). Vemos como la interfaz detecta que tras esta acción se han establecido el valor de K'.

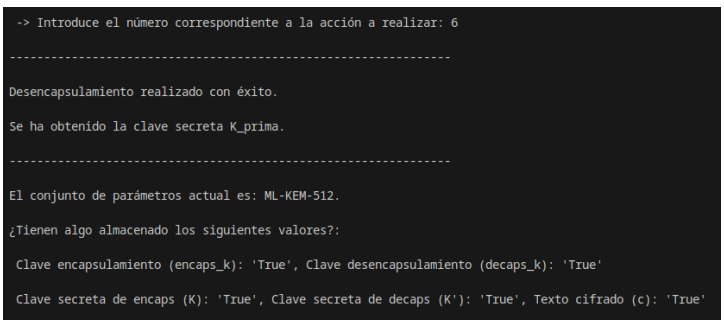


Figura 8.35: Realizar el proceso de desencapsulamiento con la clave decaps_k introducida por parámetro al texto cifrado calculado en el paso anterior

Finalmente, comparar si las claves obtenidas en las acciones 5 y 6 son la misma (acción 7).

```
 -> Introduce el número correspondiente a la acción a realizar: 7

------------------------------------------------------------

Se han comparado las claves secretas generadas durante el encapsulamiento y el desencapsulamiento y son iguales.

Por lo tanto, el modelo ML-KEM funciona correctamente con este conjunto de claves y parámetros.

------------------------------------------------------------

El conjunto de parámetros actual es: ML-KEM-512.
```

Figura 8.36: Comparación de las claves secretas tras el proceso de encapsulamiento y desencapsulamiento a partir de las claves introducidas como parámetro

Toda la ejecución se ha realizado correctamente y hemos conseguido establecer una clave secreta K para una comunicación segura entre dos usuarios.

8.5. Ejemplo de la interfaz con K, dk y c por parámetro

Si un usuario lo que quiere es directamente desencapsular un texto cifrado que tiene, gracias a una clave de desencapsulamiento que también tiene ya, y finalmente comprarla con una clave secreta K para ver si el resultado es el mismo, lo que debe de hacer es lo siguiente:

```python
if __name__ == "__main__":

    clave_encaps, clave_decaps = ML_KEM.ML_KEM_KeyGen()
    clave_secreta, texto_cifrado = ML_KEM.ML_KEM_Encaps(clave_encaps)
    ML_KEM.ML_KEM_Interfaz(decaps_k=clave_decaps, K=clave_secreta, c=texto_cifrado)
```

Figura 8.37: Claves de encapsulamiento, clave secreta y texto cifrado creados previamente y pasados como argumento

Primero, pasar al conjunto de parámetros adecuado para la clave de desencapsulamiento si no lo está ya (acción 1). Además, comprobar que la interfaz reconoce todos los parámetros que le hemos introducido.

```
------------------------------------------------------------

El conjunto de parámetros actual es: ML-KEM-768.

¿Tienen algo almacenado los siguientes valores?:

 Clave encapsulamiento (encaps_k): 'True', Clave desencapsulamiento (decaps_k): 'True'

 Clave secreta de encaps (K): 'True', Clave secreta de decaps (K'): 'False', Texto cifrado (c): 'True'
```

Figura 8.38: Pasar al conjunto de parámetros correcto para la clave de desencapsulamiento introducida por parámetro (en este caso ya nos encontramos en el conjunto correcto)

Segundo, asignar la clave decaps_k introducida como argumento (acción 4).

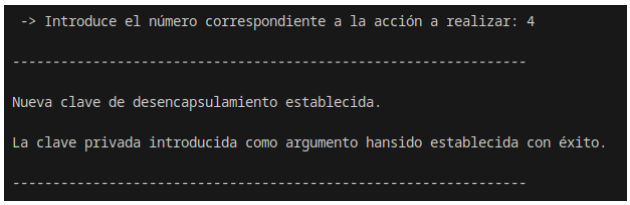


Figura 8.39: Asignar el valor de la clave de desencapsulamiento introducido por parámetro como clave de desencapsulamiento de la clase

Tercero, realizar el proceso de desencapsulamiento de un mensaje cifrado (acción 6). Ver que la interfaz detecta la clave K' tras este proceso.

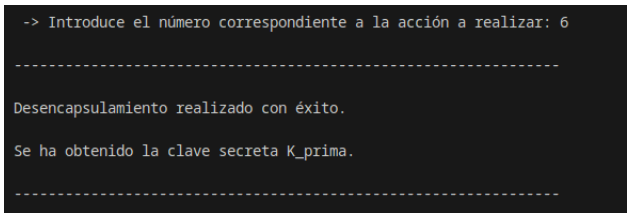


Figura 8.40: Realizar el proceso de desencapsulamiento con la clave decaps_k introducida por parámetro al texto cifrado introducido también como parámetro

Finalmente, comparar si las claves obtenida del desencapsulamiento es la misma que la introducida por parámetro (acción 7).

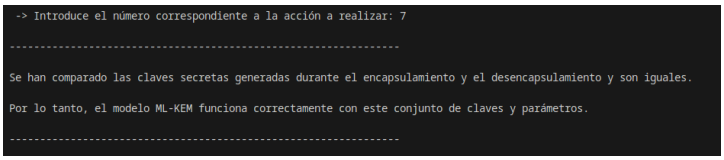


Figura 8.41: Comparación de las claves secretas tras el proceso de desencapsulamiento y la la introducida inicialmente por parámetro

Ambas claves coinciden por lo que hemos comprobado que podrían utilizarse para establecer una comunicación segura entre dos usuarios.

8.6. Ejemplo de la interfaz con K y K' por parámetro

Si se tienen dos claves secretas y se quiere comprobar si son iguales para su posterior uso, el único paso que hay que realizar es la acción 7, es decir, comparar si dos claves K y K' son iguales.

```
if __name__ == "__main__":

    clave_encaps, clave_decaps = ML_KEM.ML_KEM_KeyGen()
    clave_secreta, texto_cifrado = ML_KEM.ML_KEM_Encaps(clave_encaps)
    clave_secreta_prima = ML_KEM.ML_KEM_Decaps(clave_decaps, texto_cifrado)
    ML_KEM.ML_KEM_Interfaz(K=clave_secreta, c=texto_cifrado)
```

Figura 8.42: Claves K y K' precreadas y pasadas como argumento

```
Bienvenido a la interfaz de usuario del modelo KEM: ML-KEM.

Esta interfaz ha sido desarrollada por Denis Valentin Stoyanov D'Antonio

El conjunto de parámetros establecido por defecto es: ML-KEM-768.

------------------------------------------------------------

El conjunto de parámetros actual es: ML-KEM-768.

¿Tienen algo almacenado los siguientes valores?:

 Clave encapsulamiento (encaps_k): 'False', Clave desencapsulamiento (decaps_k): 'False'

 Clave secreta de encaps (K): 'True', Clave secreta de decaps (K'): 'True', Texto cifrado (c): 'False'


¿Qué quieres hacer?:
 1: Modificar el conjunto de parámetros a utilizar (se deberán volver a generar o asignar las claves encaps_k y decaps_k).
 2: Generar el conjunto de claves aleatoriamente.
 3: Asignar la clave encaps_k introducida como argumento.
 4: Asignar la clave decaps_k introducida como argumento.
 5: Realizar el proceso de encapsulamiento de un mensaje.
 6: Realizar el proceso de desencapsulamiento de un mensaje cifrado.
 7: Comparar claves secretas compartidas.

 0: Cerrar la interfaz.

 -> Introduce el número correspondiente a la acción a realizar: 7

------------------------------------------------------------

Se han comparado las claves secretas generadas durante el encapsulamiento y el desencapsulamiento y son iguales.

Por lo tanto, el modelo ML-KEM funciona correctamente con este conjunto de claves y parámetros.

------------------------------------------------------------
```

Figura 8.43: Comparar las claves K y K' desde el primer momento que se accede a la interfaz

Ambas claves coinciden por lo que hemos comprobado que podrían utilizarse para establecer una comunicación segura entre dos usuarios.

8.7. Ejemplos de usos incorrectos de la interfaz

Hay algunas comprobaciones y errores preestablecidos:

- Si se introducen números fuera de rango, se solicita que se vuelva a introducir un número.
- Siempre que se modifique el conjunto de parámetros, se deberán de volver a establecer las claves de encapsulamiento y desencapsulamiento.
- No se puede asingar un valor encaps_k a partir de un argumento si no se ha pasado una clave de encapsulamiento como argumento.
- No se puede asignar un valor encaps_k a partir de un argumento si el conjunto de parámetros no es el adecuado para dicha clave.
- No se puede asingar un valor decaps_k a partir de un argumento si no se ha pasado una clave de desencapsulamiento como argumento.
- No se puede asignar un valor decaps_k a partir de un argumento si el conjunto de parámetros no es el adecuado para dicha clave.
- No se puede realizar una acción de encapsulamiento si no hay ningún valor encaps_k asignado a la interfaz.
- No se puede realizar una acción de desencapsulamiento si no hay una clave de encapsulamiento y un texto cifrado asignados a la interfaz. |item No se puede realizar una comprobación de si dos claves K y K' son iguales si no están asignadas a la interfaz cada una de ellas.

Capítulo 9

Conclusiones

En este capítulo, vamos a comprobar cuáles eran los objetivos iniciales y cuáles hemos terminado cumpliendo, qué problemas u obstáculos se han tenido durante la realización de este proyecto y qué conclusiones y aprendizajes nos llevamos de todo este desarrollo.

9.1. Cumplimiento de los objetivos iniciales

Vamos a recapitular los objetivos fijados en el capítulo 1 y ver si se han cumplido o no. En caso afirmativo, explicaremos qué se ha hecho para lograrlo; en caso negativo, por qué no se ha podido realizar correctamente.

9.1.1. Analizar el estado actual de la criptografía, incluyendo los algoritmos de cifrado simétrico, cifrado asimétrico y firma digital más utilizados

Estado: cumplido.

Este objetivo se ha cumplido ya que, como podemos apreciar en el capítulo 2, se ha dedicado un capítulo entero a analizar la criptografía actual, con sus diferentes métodos de llevarla a cabo; así como, los algoritmos más importantes y/o utilizados para cada método.

9.1.2. Explicar los principios fundamentales de la computación cuántica y cómo esta tecnología representa una amenaza para los sistemas criptográficos actuales

Estado: cumplido.

Para lograr este objetivo también se ha dedicado un capítulo entero, específicamente el 3, en el que se explica de una forma detallada pero comprensible en qué se basa la computación cuántica, para qué cosas es muy útil y eficiente, y por qué esto supone una amenaza para la criptografía actual.

9.1.3. Investigar el concurso para la estandarización de algoritmos post-cuánticos llevado a cabo por el NIST, incluyendo una revisión de los algoritmos más prometedores

Estado: cumplido.

El objetivo inicial era analizar los algoritmos post-cuánticos más prometedores, es decir, los que están en las fases más avanzadas del concurso.

Finalmente, realizamos un mayor análisis al planteado inicialmente, ya que, al investigar en la página web oficial del NIST, nos encontramos con que en 2022 ya se estandarizaron cuatro algoritmos; además de que se ha lanzado un nuevo concurso para recabar propuestas sobre nuevos algoritmos post-cuánticos para firmas digitales.

El objetivo inicial y todas las cosas extra que se han comentado quedan recogidas en el capítulo 4 de este trabajo.

9.1.4. Introducir y explicar los fundamentos de los problemas basados en retículos, que son la base matemática de muchos algoritmos post-cuánticos

Estado: cumplido.

Todo el capítulo 5 se destina a explicar el concepto de retículo, los tipos de retículos existen y qué problemas se pueden crear a partir de ellos.

9.1.5. Seleccionar y desarrollar un algoritmo post-cuántico basado en retículos destacado del concurso del NIST, enfocado en el encapsulamiento de claves

Estado: cumplido.

En el capítulo 6 se desarrolla de forma teórica todo lo relacionado con nuestro algoritmo de encapsulamiento de claves basado en el problema Module-LWE de los retículos, denominado ML-KEM, en el que se basa el estándar FIPS 203.

9.1.6. Implementar el algoritmo seleccionado

Estado: cumplido.

En el capítulo 7 se lleva a cabo la implementación del algoritmo ML-KEM. Para ello, se explica detalladamente el lenguaje utilizado y todas las constantes, variables y métodos de los que dispone la clase ML-KEM.

Además, se explica de forma teórica la traza usual que sigue una ejecución de los tres métodos principales (ML_KEM_KeyGen, ML_KEM_Encaps y ML_KEM_Decaps) para generar un nuevo par de claves de encapsulamiento y desencapsulamiento, llevar a cabo la encapsulación y, con el texto cifrado obtenido, llevar a cabo la desencapsulación; comprobando para finalizar si las claves secretas compartidas obtenidas para cada usuario son iguales o no.

Finalmente, también se comentan problemas encontrados a la hora de implementar correctamente el código del algoritmo.

9.1.7. Implementar una interfaz para el algoritmo desarrollado

Estado: cumplido.

Para el correcto e intuitivo uso de la clase ML-KEM se ha desarrollado, en el capítulo 8, una interfaz por consola interactiva que permite realizar las siete acciones principales para las que ha sido creada la clase.

9.1.8. Comprobar el correcto funcionamiento del algoritmo

Estado: cumplido.

Se han realizado pruebas para medir el tiempo promedio de ejecución para cada uno de los tres conjuntos de parámetros, así como pruebas a través de la interfaz, comprobando que funciona correctamente y que se muestran los errores pertinentes en caso de que sucedan.

9.1.9. Sacar conclusiones de todo lo desarrollado y aprendido

Estado: cumplido.

Este propio capítulo es el destinado para alcanzar este objetivo.

9.2. Problemas u obstáculos encontrados durante la realización del proyecto

En este apartado vamos a comentar los problemas u obstáculos que se han encontrado para realizar este proyecto; esto es, no solo las dificultades para implementar correctamente el código, sino cualquier ámbito que haya podido interferir en el trabajo.

El principal problema ha sido que este algoritmo de encapsulamiento de claves, al basarse en el problema de Module-LWE, requiere un gran conocimiento previo de matemáticas, álgebra y de los propios retículos. Esto es un requisito evidente, pero personalmente no estaba muy familiarizado con el ámbito de desarrollo de algoritmos de cifrado, por lo que ha sido una barrera difícil de superar. Además, hasta que no comprendía un punto, no podía avanzar al siguiente, como por ejemplo la traza: álgebra utilizada en los retículos → qué son los retículos → problema Module-LWE. Por esta razón he estado estancado varias veces en algunos puntos a lo largo de la realización del proyecto, hasta que por fin comprendiese los conceptos que estaba leyendo pero no era capaz de entender.

Un problema menor ha sido el idioma, y es que toda la documentación que existe y he utilizado al respecto está en inglés. En principio esto no ha supuesto un problema, pero si es verdad que, al utilizar palabras tan técnicas como se puede esperar de las utilizadas en el documento de un estándar oficial, cuando no comprendía alguna y no disponía de internet para buscar su significado, el resto del texto leído ya se me quedaba descolgado por ello.

Otro problema, aunque no lo parezca, es el haber tenido total libertad para utilizar los recursos que yo creyese oportunos para implementar este algoritmo, ya que en el documento todo viene dado en pseudocódigo. Esto me ha supuesto un problema porque a veces utilizaba demasiada abstracción y pensaba "¿Debería crear yo mismo el método en lugar de utilizar uno precreado?", o, en el caso contrario "¿Hace falta que implemente todo este algoritmo o podría utilizar una función precreada de alguna librería de Python?".

Finalmente, un problema que puede parecer no tan evidente es que todos estos algoritmos, incluso la computación cuántica, no es algo de nuestro día a día. Esto hace que apenas haya información al respecto y que la que hay esté en constante cambio a medida que avanzan los descubrimientos. Además, no hay casi forma de comprobar si un algoritmo es mejor que otro, ya que no se usan todavía; ni de realizar muchas pruebas o siquiera que estas pruebas tengan certeza de estar ejecutándose correctamente ya que, de nuevo, los ordenadores cuánticos aún están en un punto bastante primitivo.

9.3. Conclusiones y aprendizaje obtenidos a raíz de este proyecto

Este es, probablemente, el punto más importante de todo el proyecto; y es que es la valoración personal de qué se ha aprendido y, sobre todo, de qué sirve o qué implica de cara al presente y al futuro todo ello.

Este trabajo me ha hecho afianzar los conceptos sobre la criptografía actual, conociendo un poco más acerca de los estándares utilizados hoy día, sus principales algoritmos y cómo funcionan estos.

He aprendido qué es la computación cuántica, que, en mi opinión personal, es un campo tan interesante de estudiar como complicado de crear. Hemos visto que la creación de ordenadores cuánticos es aún una tarea extremadamente compleja al alcance de muy pocas súper-compañías. Sin embargo, este campo está en constante mejora y nunca se sabe si de un día para otro puede haber un gran descubrimiento que revolucione todo esto, como ha pasado este año 2024 con, por ejemplo, la inteligencia artificial.

Realmente, en parte, es una suerte que este campo no esté todavía muy avanzado, ya que, como hemos visto en este proyecto, puede acarrear grandes problemas a nivel mundial. La verdadera carrera no está en conseguir ordenadores cuánticos lo antes posible (que también), sino en conseguir proteger todo lo vulnerable frente a ellos antes de que esto pase.

En pos de entender y desarrollar este objetivo principal por nosotros mismos, hemos desarrollado el algoritmo post-cuántico ML-KEM. Gracias a este análisis e implementación, se ha podido aprender mucho más sobre la criptografía post-cuántica, ver cuán avanzada está y sobre qué principios se basa.

El algoritmo que yo he desarrollado se estandarizó el pasado 14 de agosto de 2024 y es una derivación del algoritmo CRYSTALS-Kyber, que fue estandarizado en 2022. Esto es un claro ejemplo de que ya existen algoritmos que se consideran seguros, por lo que se lleva bastante tiempo investigando en ellos; así como que no por haber estandarizado algunos se ha dejado de investigar o dado la tarea por finalizada ni mucho menos, sino que se sigue buscando mejorar y reforzar la seguridad en este campo.

Solo es cuestión de tiempo que estos nuevos algoritmos post-cuánticos estandarizados empiecen a implementarse a gran escala, sustituyendo los actuales. Para cuando esto pase, yo y cualquier persona que se lea este proyecto fin de grado, contará con una idea general bastante buena sobre qué tratan dichos algoritmos. Además, aunque sea pequeña, cabe la posibilidad de que sea este mismo algoritmo, ML-KEM, el que se acabe implantando en muchos protocolos de seguridad.

Cierro este trabajo dando las gracias a todas las personas que hayan llegado hasta este punto, sea cual sea la razón por la que comenzaron a leer. Espero que este trabajo les haya resultado útil y les haya servido para conocer un poco más sobre este campo.

Bibliografía

[1] Aprende con Alf. *Manual de NumPy en Python*. https://aprendeconalf.es/docencia/python/manual/numpy/. 2024.

[2] Amazon Web Services. *What is Cryptography?* 2024. URL: https://aws.amazon.com/es/what-is/cryptography/.

[3] Jesús Cea Avión. *Registros de Desplazamiento con Realimentación Lineal (LFSR)*. 2024. URL: https://www.jcea.es/artic/lfsr.htm.

[4] Michel Boyer et al. "Tight bounds on quantum searching". En: *Fortschritte der Physik* 46.4-5 (1998), págs. 493-505.

[5] CEUPE. *Cifrado Simétrico: Qué es, Usos y Técnicas*. 2023. URL: https://www.ceupe.com/blog/cifrado-simetrico.html.

[6] COSIC - KU Leuven. *Introduction to Lattices*. 2023. URL: https://www.esat.kuleuven.be/cosic/blog/introduction-to-lattices/.

[7] COSIC, KU Leuven. *Introduction to Lattice-based Cryptography (Part 1)*. https://www.esat.kuleuven.be/cosic/blog/introduction-to-lattices/. 2020.

[8] CRYSTALS. *CRYSTALS-Dilithium: A Post-Quantum Digital Signature Scheme*. 2024. URL: https://pq-crystals.org/dilithium/index.shtml.

[9] CRYSTALS. *CRYSTALS-Kyber: A Post-Quantum Cryptography Project*. 2024. URL: https://pq-crystals.org/kyber/index.shtml.

[10] Derivando. *¿Es verdad que los ORDENADORES CUÁNTICOS van a romper nuestras CLAVES SECRETAS de internet?* 2024. URL: https://www.youtube.com/watch?v=hh1fHVzlqOQ.

[11] Derivando. *¿Qué es y cómo funciona la COMPUTACIÓN CUÁNTICA?* 2024. URL: https://www.youtube.com/watch?v=YpYuBEzfRlM.

[12] Python Software Foundation. *hashlib — Algoritmos seguros de hash y resumen de mensajes*. https://docs.python.org/es/3/library/hashlib.html. 2024.

[13] GMV. *Qué es la criptografía y para qué se utiliza*. 2023. URL: https://www.gmv.com/es-es/node/5731/printable/print.

[14] Lov K. Grover. "A fast quantum mechanical algorithm for database search". En: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*. ACM. 1996, págs. 212-219.

[15] IBM. *¿Qué es la computación cuántica?* 2024. URL: https://www.ibm.com/es-es/topics/quantum-computing.

[16] IONOS. *Cifrado Asimétrico: Definición y Ejemplos de la Criptografía Asimétrica*. 2023. URL: https://www.ionos.es/digitalguide/servidores/seguridad/cifrado-asimetrico/.

[17] Juncotic. *Criptografía Simétrica: Conceptos clave*. 2023. URL: https://juncotic.com/criptografia-simetrica-conceptos-clave/.

[18] Veritas Technologies LLC. *What Is RSA Encryption?* 2024. URL: https://www.veritas.com/es/es/information-center/rsa-encryption.

[19] Vadim Lyubashevsky, Chris Peikert y Oded Regev. "On Ideal Lattices and Learning with Errors over Rings". En: *Journal of the ACM (JACM)* 60.6 (2010), págs. 1-35.

[20] Daniele Micciancio y Chris Peikert. "Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller". En: *Advances in Cryptology–EUROCRYPT 2012*. Springer. 2012, págs. 700-718.

[21] Daniele Micciancio y Oded Regev. "Lattice-Based Cryptography". En: *Post-Quantum Cryptography*. Ed. por Daniel J. Bernstein, Johannes Buchmann y Erik Dahmen. Supported in part by NSF Grant CCF 0634909 and the Binational Science Foundation, Israel Science Foundation, European Commission under the Integrated Project QAP, and European Research Council (ERC) Starting Grant. Springer, 2009, págs. 147-191.

[22] Victor S. Miller. *Use of elliptic curves in cryptography*. Springer Berlin Heidelberg, 1985.

[23] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Inf. téc. National Institute of Standards y Technology (NIST), 2024. URL: https://csrc.nist.gov/pubs/fips/203/final.

[24] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (Initial Public Draft)*. Inf. téc. National Institute of Standards y Technology (NIST), 2023. URL: https://csrc.nist.gov/pubs/fips/203/ipd.

[25] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Inf. téc. FIPS 203. Gaithersburg, MD: U.S. Department of Commerce, ago. de 2024. URL: https://doi.org/10.6028/NIST.FIPS.203.

[26] National Institute of Standards and Technology (NIST). *Advanced Encryption Standard (AES)*. 2001. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf.

[27] National Institute of Standards and Technology (NIST). *Data Encryption Standard (DES)*. 1977. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.46-3.pdf.

[28] National Institute of Standards and Technology (NIST). *Post-Quantum Cryptography Standardization*. 2024. URL: https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization.

[29] National Institute of Standards and Technology (NIST). *Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher*. 1999. URL: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-67r1.pdf.

[30] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge University Press, 2010.

[31] Chris Peikert. "A Decade of Lattice Cryptography". En: *Foundations and Trends® in Theoretical Computer Science* 10.4 (2016), págs. 283-424.

[32] Falcon Project. *Falcon: A Post-Quantum Digital Signature Scheme*. 2024. URL: https://falcon-sign.info/.

[33] The Wild Project. *EL GRAN PELIGRO DE LOS ORDENADORES CUÁNTICOS - Matemático detalla un posible problema mundial*. 2024. URL: https://www.youtube.com/watch?v=bfYH7JmHQnk.

[34] The Wild Project. *Programador informático deja alucinado a Jordi Wild contándole cómo funciona un Ordenador Cuántico*. 2024. URL: https://www.youtube.com/watch?v=xLL6ly830dA.

[35] Red Seguridad. *Cifrado Simétrico: Qué es, Usos y Técnicas*. 2023. URL: https://www.redseguridad.com/actualidad/cifrado-simetrico-que-es-usos-tecnicas_20231104.html.

[36] Reglamento General de Protección de Datos (RGPD). *Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo de 27 de abril de 2016 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos*. 2016. URL: https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=celex%3A32016R0679.

[37] Ronald L. Rivest, Adi Shamir y Leonard Adleman. "A method for obtaining digital signatures and public-key cryptosystems". En: *Communications of the ACM* 21.2 (1978), págs. 120-126.

[38] Peter W. Shor. "Algorithms for quantum computation: Discrete logarithms and factoring". En: *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE. 1994, págs. 124-134.

[39] William Stallings. *Cryptography and Network Security: Principles and Practice*. 7th. Boston: Pearson, 2016. ISBN: 978-0134444284.

[40] National Institute of Standards y Technology (NIST). *About NIST*. 2024. URL: https://www.nist.gov/about-nist.

[41] National Institute of Standards y Technology (NIST). *Digital Signature Algorithm (DSA)*. 2013. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf.

[42] National Institute of Standards y Technology (NIST). *Post-Quantum Cryptography Standardization: Call for Proposals*. 2024. URL: https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/Call-for-Proposals.

[43] National Institute of Standards y Technology (NIST). *Post-Quantum Cryptography Standardization: Round 1 Submissions*. 2024. URL: https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions.

[44] National Institute of Standards y Technology (NIST). *Post-Quantum Cryptography Standardization: Round 4 Submissions*. 2024. URL: https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions.

[45] National Institute of Standards y Technology (NIST). *Post-Quantum Cryptography: Digital Signature Project*. 2024. URL: https://csrc.nist.gov/projects/pqc-dig-sig.

[46] National Institute of Standards y Technology (NIST). *Post-Quantum Cryptography: Selected Algorithms 2022*. 2024. URL: https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[47] Bike Suite Team. *Bike Suite: A Comprehensive Solution for Bike Sharing*. 2024. URL: https://bikesuite.org/.

[48] Classic McEliece Team. *Classic McEliece: A Post-Quantum Cryptography Candidate*. 2024. URL: https://classic.mceliece.org/.

[49] HQC Team. *HQC: A Post-Quantum Cryptography Scheme*. 2024. URL: https://pqc-hqc.org/.

[50] SIKE Team. *SIKE: Supersingular Isogeny Key Encapsulation*. 2024. URL: https://sike.org/.

[51] SPHINCS Team. *SPHINCS: A Stateless Hash-Based Signature Scheme*. 2024. URL: https://sphincs.org/.

[52] Jianfeng Zhang y Zhenfeng Zhang. “Lattice-Based Cryptosystems”. En: *Lattice-Based Cryptosystems*. Springer Nature Singapore Pte Ltd, 2020. DOI: 10.1007/978-981-15-8427-5_2. URL: https://doi.org/10.1007/978-981-15-8427-5_2.

Apéndice A

Enlace al código disponible en un repositorio de GitHub

El código correspondiente a implementación de la clase ML-KEM, así como el código utilizado para la ejecución de la interfaz y las pruebas de medición de tiempo realizadas, se encuentra en el siguiente repositorio de GitHub: `https://github.com/DenisSValentin/Algoritmo_ML-KEM`